

Deep Learning Methods for Corporate Finance¹

Zhaoxuan Wang

2026-04-27

¹Email - wxuan.econ@gmail.com or zxwang13@student.ubc.ca.

Table of contents

Introduction	2
1 Solving Dynamic Models	3
1.1 Definition	3
1.2 Application to the Basic Investment Model	6
1.3 Application to the Risky Debt Model	7
1.4 Overview of Solution Methods	9
1.5 Data Generation, Reproducibility, and Workflow	12
1.6 Validation of Solution	13
1.7 Results: Basic Investment Model	14
1.8 Issues in Neural Network Architecture and Training	18
1.9 Risky Debt Model	20
2 Structural Estimation	25
2.1 GMM and SMM	25
2.2 Applying SMM to the risky debt model	29
2.3 Defects of the risky debt model and potential solutions	31
3 Bayesian Estimation of the Strebulaev (2012) Models	33
3.1 Short answers to the bonus question #1	33
3.2 Overview	34
3.3 Bayesian Inference with Neural Surrogate	38
3.4 Validation Results	44
3.5 Future Extensions	51
4 Reserved	53
5 Nikolov, Schmid, and Steri (2021): Three Theoretical Models	54
5.1 Technology and Notation	54
5.2 Timing (common across all three models)	55
5.3 Trade-off Model	56
5.4 Limited Enforcement Model	60
5.5 Moral Hazard Model	63
5.6 Structural Estimation and Model Comparison	68
A Solution Methods to Dynamic Models	77
A.1 Value and Policy Function Iterations	77

A.2	Lifetime Reward Maximization	78
A.3	Euler Residual Minimization	81
A.4	Short-Horizon Actor-Critic (SHAC)	83
A.5	Bellman Residual Minimization	84
A.6	Nested VFI (Risky Debt)	87
B	Implementation Details	90
B.1	Input Normalization	90
B.2	Hidden-Layer Activation	90
B.3	Output Head Transformation	91
C	Structural Estimation Methods	92
C.1	Generalized Method of Moments (GMM)	92
C.2	Simulated Method of Moments (SMM)	95
D	Moment fit: TO Model	99
	References	101

Abstract. This report examines quantitative methods for solving and estimating structural corporate finance models. On the solution side, I implement and evaluate the three deep-learning methods proposed in Maliar, Maliar, and Winant (2021) (Lifetime Reward Maximization (LRM), Euler Residual Minimization (ERM), and Bellman Residual Minimization (BRM)) and benchmark them against classical Value Function Iteration and Policy Function Iteration. I also develop a new solution method based on Short-Horizon Actor-Critic (SHAC) reinforcement learning (Xu et al. 2022) and show that it converges to the analytical solution in settings where Maliar, Maliar, and Winant (2021)'s methods are inapplicable. On the estimation side, I implement Generalized Method of Moments (GMM) and Simulated Method of Moments (SMM), validate both via Monte Carlo on the basic investment model, and apply SMM to the Hennessy and Whited (2007) endogenous default (risky debt) model. I document the practical and structural challenges in these applications.

Introduction

This report collects work on solving and estimating structural corporate finance models. Chapter 1 solves dynamic investment models using deep-learning and classical methods, and benchmarks them on a basic investment model and a risky-debt model. Chapter 2 estimates these models with the Generalized Method of Moments (GMM) and the Simulated Method of Moments (SMM). Chapter 3 develops a Bayesian estimation pipeline that uses a pre-trained neural surrogate policy to keep repeated model solves tractable. Chapter 5 solves the three Nikolov, Schmid, and Steri (2021) financing models by linear programming and estimates the trade-off model by indirect inference on Hong Kong data. Chapter 4 is reserved for future material. Each chapter is supported by an appendix collected at the end of the report.

Chapter 1

Solving Dynamic Models

To keep this report clear and concise, I focus on presenting the findings and the direct answers to the interview questions. In appendix, I provide a more detailed description of the models, solution methods (algorithms), additional results, and other implementation details.

The corporate finance model considered in this report are generally represented as a dynamic programming problem with the following features:

- Discrete-time
- Continuous state and action spaces,
- Policy function is deterministic and known
- State transition function with random noise is known

For continuous-time deep learning methods for solving dynamic models, Duarte, Duarte, and Silva (2024) proposed a “deep policy iteration” method and demonstrated its application to corporate finance and other quantitative finance models.

1.1 Definition

1.1.1 Markov Decision Process

A Markov Decision Process (MDP) is defined as a collection $(\mathcal{S}, \mathcal{A}, \mathcal{E}, f, r, \gamma)$ that subsumes all relevant information for decision-making.

Symbol	Definition
$\mathcal{S} \subseteq \mathbb{R}^n$	State space (continuous). A state $s \in \mathcal{S}$ is a vector encoding all information the agent observes. When the environment involves multiple variables (e.g., productivity z and capital k), they are stacked into a single vector $s = (z, k)^\top$.
$\mathcal{A} \subseteq \mathbb{R}^d$	Action space (continuous). An action $a \in \mathcal{A}$ is a vector of controls the agent selects (e.g., investment, consumption).

Symbol	Definition
$\mathcal{E}$	Shock space. The space from which exogenous shocks ε are drawn. When dynamics are deterministic, $\mathcal{E} = \emptyset$.
$f : \mathcal{S} \times \mathcal{A} \times \mathcal{E} \rightarrow \mathcal{S}$	State transition function/dynamics. Given current state s , action a , and exogenous shock $\varepsilon \in \mathcal{E}$, the next state is $s' = f(s, a, \varepsilon)$. When dynamics are deterministic, the shock argument is absent and $f : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$. When the dynamics involve both exogenous and endogenous components, they are combined into a single vector-valued function.
$r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$	Reward function. A scalar signal $r(s, a)$ received after taking action a in state s (e.g., cash flow or utility).
$\gamma \in (0, 1)$	Discount factor. Controls the trade-off between immediate and future rewards.

A full sequence of actions and states is defined as a **trajectory** or **rollouts**:

$$(s_0, a_0, s_1, a_1, \dots)$$

where the initial state s_0 is randomly sampled from some distribution p_0 . The state transition is given by:

$$s_{t+1} = f(s_t, a_t, \epsilon_t)$$

where ϵ_t is a random noise (e.g., productivity shock) but the function f is deterministic. Note that this is different from a stochastic transition function in RL where s_{t+1} is a draw from a distribution $s_{t+1} \sim P(\cdot | s_t, a_t)$.

The **reward function** $r(s, a)$ is assumed to be known exactly and it maps current state and actions (s, a) to a scalar value. In corporate finance, this is typically firm's cash flow. The **discounted lifetime reward** over an infinite-horizon trajectory is summarized as:

$$\sum_{t=0}^{\infty} \gamma^t \cdot r(s_t, a_t)$$

1.1.2 Optimal Policy Function

A **deterministic policy** π is a mapping from $\mathcal{S} \rightarrow \mathcal{A}$. In most applications there do not exist a closed-form analytical solution to π , thus researchers either directly solve for it numerically, or use a function approximator with parameterization. Denote the parameters as θ and the parameterized deterministic policy as π_θ . Given a state s , the policy outputs a specific action:

$$a = \pi_\theta(s)$$

The approximator can be as simple as linear, $\pi_\theta(s) = \theta_1 + \theta_2 s$. However, to better capture non-linearities of the unknown mapping, I use a deep neural network approximator where θ is a vector that collects all trainable weights and biases. Here π_θ is a deterministic function (e.g., the neural network), and a is a realized scalar or vector (the network's output for input s).

The **optimal policy** π^* is defined by

$$\pi^* \equiv \arg \max_{\pi} \mathbb{E}_{\epsilon} \left[\sum_{t=0}^{\infty} \gamma^t \cdot r(s_t, \pi(s_t)) \right]$$

subject to dynamics $s_{t+1} = f(s_t, \pi(s_t), \epsilon_{t+1})$ with random noise $\epsilon_{t+1} \sim P_{\epsilon}$.

With NN approximator π_{θ} , the objective is equivalent to finding the optimal NN parameters θ^* such that

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\epsilon} \left[\sum_{t=0}^{\infty} \gamma^t \cdot r(s_t, \pi_{\theta}(s_t)) \right]$$

1.1.3 Value Functions

Given a policy π_{θ} and starting state s_0 , define the trajectory $\tau \equiv (s_0, a_0, s_1, a_1, \dots)$ where $a_t = \pi_{\theta}(s_t)$ and $s_{t+1} = f(s_t, a_t, \epsilon_{t+1})$ with shocks ϵ_t drawn i.i.d. from some distribution.

1.1.3.1 On-Policy Value function

The expected return if you start in state s and always act according to policy π is given as:

$$V^{\pi_{\theta}}(s) = \mathbb{E}_{(\epsilon_1, \epsilon_2, \dots)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi_{\theta}(s_t)) \right]$$

where $s_{t+1} = f(s_t, \pi_{\theta}(s_t), \epsilon_{t+1})$. This is the expected cumulated discounted reward from state s when following policy π_{θ} , with the expectation taken over all future shock realizations.

1.1.3.2 Optimal Value Function

The optimal value function $V^*(s)$ gives the maximum expected lifetime reward if agent start from s and always act according to the *optimal* policy π^* :

$$V^*(s) = \max_{\pi} V^{\pi}(s) = \mathbb{E}_{(\epsilon_1, \epsilon_2, \dots)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi^*(s_t)) \right]$$

1.1.4 Bellman Equations

Let the ‘tick’ denote next time step variables, e.g., $s' \equiv s_{t+1}$, the Bellman equations for the on-policy value functions are

$$V^{\pi}(s) = r(s, \pi(s)) + \gamma \mathbb{E}_{\epsilon} [V^{\pi}(s')]$$

where the expectation is taken over ϵ that governs the state transition functions $s' = f(s, a, \epsilon)$.

The Bellman equations for the optimal value functions are

$$V^*(s) = \max_a \{r(s, a) + \gamma \mathbb{E}_{\epsilon} [V^*(s')]\}$$

where the max operator ensures that at optimality, the agent will pick the action that maximizes the Bellman right-hand-side (RHS).

1.2 Application to the Basic Investment Model

This section briefly describes the basic investment model, in which firm chooses the optimal investment given current states (capital stock and realized productivity). The solution to this model is the optimal policy function that maps $(k, z) \rightarrow I$ or k' .

State Space: State variables are capital k and productivity shock z , which is stacked into a single state vector:

$$s = (k, z)^\top$$

with exogenous bounded space $[k_{\min}, k_{\max}]$ and $[z_{\min}, z_{\max}]$.

Action Space: Action variable is investment $a = I$ that can be either positive or negative. The state spaces $0 \leq k' \leq k_{\max}$ and $0 \leq k \leq k_{\max}$ implies:

$$-(1 - \delta)k \leq I \leq k_{\max}$$

so that the action space is also bounded by $[I_{\min}, I_{\max}]$ with $I_{\min} = -(1 - \delta)k < 0$ and $I_{\max} = k_{\max}$.

State Transition Function. The productivity shocks follow auto-regression (AR-1) process with stationary mean μ , persistent coefficient ρ , and variance σ :

$$\log z' = \mu + \rho \log z + \sigma \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1)$$

Capital stock depends on depreciation rate δ and investment I (action):

$$k' = (1 - \delta)k + I$$

They are stacked into a single vector-valued state transition function:

$$f(s, a, \epsilon) \equiv \begin{pmatrix} z' \\ k' \end{pmatrix} = \begin{pmatrix} \exp\{\mu + \rho \log z + \sigma \epsilon\} \\ (1 - \delta)k + I \end{pmatrix}$$

Reward is defined by the net cash flow $e(k, z, I)$:

$$r(s, a) \equiv e(k, z, I) = \Pi(k, z) - \Psi(I, k) - I$$

where the production function is Cobb-Douglas with parameter β :

$$\Pi(k, z) = z \cdot k^\beta, \quad \beta \in (0, 1)$$

and the capital adjustment cost is

$$\Psi(I, k) = \phi_0 \cdot \frac{I^2}{2k} + \phi_1 \cdot k \cdot \mathbf{1}\{I \neq 0\}$$

where ϕ_0 is the smooth adjustment cost coefficient and ϕ_1 is the fixed adjustment cost coefficient. The indicator $\mathbf{1}\{I \neq 0\}$ triggers whenever the firm invests or disinvests.

Objective: The solution of the model is to find the optimal investment policy that maximizes expected discounted lifetime cash flows:

$$\max_{\{I_t\}_{t=0}^{\infty}} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t \cdot r(k_t, z_t, I_t) \right]$$

where $\gamma = 1/(1 + \bar{r})$ is the discount factor and $\bar{r}$ is the risk-free interest rate. The solution is an optimal parameterized policy function $\pi_\theta^*(s)$ that maps states (k, z) to investment I .

Frictionless analytical solution. When both adjustment costs are zero ($\phi_0 = \phi_1 = 0$), the model admits a closed-form optimal policy that depends only on z : capital is fully reversible each period, so the firm resets k to the static optimum every period. The frictionless first-order condition

$$1 = \gamma \mathbb{E} \left[\beta z' (k')^{\beta-1} + (1 - \delta) \mid z \right]$$

solves to

$$k^*(z) = \left[\frac{\beta \cdot \mathbb{E}[z' \mid z]}{\bar{r} + \delta} \right]^{1/(1-\beta)}, \quad \mathbb{E}[z' \mid z] = \exp((1 - \rho)\mu + \rho \log z + \frac{1}{2}\sigma^2).$$

The conditional expectation is given by the log-normal mean in closed form: no numerical quadrature is needed, because $\log(z' \mid z)$ is exactly normal with mean $(1 - \rho)\mu + \rho \log z$ and variance σ^2 . This makes $k^*(z)$ an exact, parameter-free anchor that I use repeatedly throughout the paper:

- k^* used as anchor for the space bounds of k and b .
- k^* as the ground-truth policy for [benchmarking solution methods](#);
- k^* used for computing the baseline steady-state value V^* for normalizing the reward and Bellman in SHAC and BRM methods;
- k^* as the closed-form optimal policy in [Part II SMM validation](#) that isolates estimation error from solver error.

State-space bounds. All three bounds are pinned to the AR(1) stationary distribution and the frictionless anchor k^* :

- $z \in [\exp(\mu - 3\sigma_{\text{erg}}), \exp(\mu + 3\sigma_{\text{erg}})]$ with ergodic standard deviation $\sigma_{\text{erg}} = \sigma/\sqrt{1 - \rho^2}$ truncates the log-normal stationary distribution at ± 3 ergodic std-devs. Initial z_0 is drawn uniformly in level space within this range.
- $k \in [\underline{c}^k \cdot k^*, \bar{c}^k \cdot k^*]$, where $k^* = k^*(\bar{z})$ is the frictionless capital at the stationary mean $\bar{z} = \exp(\mu)$. The asymmetric upper and lower bound multiplier $\underline{c}^k, \bar{c}^k$ are supplied by user and are set to be generous so the box covers the optimal k' at the upper end of z . Default values are $[0.25k^*, 6k^*]$.

These are solver / training parameters calibrated once at construction time from $(\mu, \rho, \sigma, \beta, \bar{r}, \delta)$.

1.3 Application to the Risky Debt Model

The risky debt model extends the basic model by allowing firms to borrow at an endogenous risky interest rate, with the option to default. Then risky interest rate is determined by the lender's zero profit condition with rational expectation of default probability. Firm's optimal investment and leveraging in turn depends on the equilibrium risky interest rate.

The solution to this model consists of (i) an optimal policy function π^* mapping states $(k, b, z) \rightarrow (k', b')$; and (ii) the optimal value function $V^{\pi^*}(k, b, z)$ satisfying the Bellman equation for policy π^* .

State Space The state variables are current capital k , productivity z , and debt b :

$$s \equiv (k, b, z)$$

with bounded state space $k \in [k_{\min}, k_{\max}]$, $z \in [z_{\min}, z_{\max}]$, and $b \in [0, b_{\max}]$. Here $b_{\max}$ is an exogenously determined upper bound and should be set generously large enough to avoid binding frequently.

Action Space Firm chooses investment I and next-period debt b' (new borrowing):

$$a \equiv (I, b')$$

with action space bounds $[I_{\min}, I_{\max}]$ and $[0, b_{\max}]$.

State Transition Function is a vector-valued function defined as:

$$f(s, a, \epsilon) \equiv \begin{pmatrix} z' \\ k' \\ b' \end{pmatrix} = \begin{pmatrix} \exp\{\mu + \rho \log z + \sigma \epsilon\} \\ (1 - \delta)k + I \\ b' \end{pmatrix}$$

Reward is defined as the net cashflow (payouts) minus cost of external financing:

$$e(k, b, z; I, b') - \Omega(e(\cdot))$$

Cash flow is given as

$$e(k, b, z; I, b') \equiv (1 - \tau)\Pi(k, z) - \Psi(I, k) - I - b + \frac{b'}{1 + \tilde{r}(\cdot)} + \frac{\tau \tilde{r}(\cdot) b'}{[1 + \tilde{r}(\cdot)](1 + r)}$$

where

- τ is the corporate tax rate
- b is repayment of last-period debt
- r is risk-free interest rate
- $\tilde{r}(\cdot)$ is the endogenous risky interest rate that depends on states
- $b'/(1 + \tilde{r})$ is proceeds from issuing new risky debt
- The last term is the tax shield from debt interest

When cash flow is negative, the firm must raise costly external equity:

$$\Omega(e) = (\omega_0 + \omega_1 |e|) \cdot \mathbf{1}\{e < 0\}$$

As in the basic model, the production function is Cobb-Douglas with parameter β :

$$\Pi(k, z) = z \cdot k^\beta, \quad \beta \in (0, 1)$$

and the capital adjustment cost is

$$\Psi(I, k) = \phi_0 \cdot \frac{I^2}{2k} + \phi_1 \cdot k \cdot \mathbf{1}\{I \neq 0\}$$

where ϕ_0 is the smooth adjustment cost coefficient and ϕ_1 is the fixed adjustment cost coefficient. The indicator $\mathbf{1}\{I \neq 0\}$ triggers whenever the firm invests or disinvests.

Endogenous Risky Interest Rate The bond price $q = 1/(1 + \tilde{r})$ is determined by the lender's zero-profit condition:

$$b'(1 + r) = (1 + \tilde{r})b' \mathbb{E}_\epsilon[1 - D] + \mathbb{E}_\epsilon[D \cdot \text{Recovery}(k', b', z')]$$

where:

- LHS: Opportunity cost of lending at risk-free rate
- RHS: Expected return accounting for default probability and recovery

Endogenous Default The firm defaults when its continuation (latent) value is negative:

$$D(k', b', z') = \mathbf{1}\{\tilde{V}(k', b', z') < 0\}$$

Shareholders walk away with zero under limited liability:

$$V(k', b', z') = \max\{0, \tilde{V}(k', b', z')\}$$

Recovery Under Default

$$\text{Recovery}(k', z') = (1 - \alpha) [(1 - \tau)\pi(k', z') + (1 - \delta)k']$$

where $\alpha \in [0, 1]$ is the deadweight loss from liquidation.

Bellman Equation for the latent firm value $\tilde{V}$ is given by

$$\begin{aligned} \tilde{V}(k, b, z) &= \max_{k', b'} \{e(k, b, z; I, b') - \Omega(e) + \gamma \mathbb{E}_\epsilon[V(k', b', z')]\} \\ &= \max_{k', b'} \{e(k, b, z; I, b') - \Omega(e) + \gamma \mathbb{E}_\epsilon[\max\{0, \tilde{V}(k', b', z')\}]\} \end{aligned}$$

where the RHS continuation value encodes limited liability (firm can walk away with zero).

The Nested Fixed-Point Problem A key computational challenge is that the latent value $\tilde{V}$ depends on the risky rate $\tilde{r}$, but solving for $\tilde{r}$ requires knowing the default probability $\mathbb{E}[D]$, which depends on $\tilde{V}$. Traditional methods solve this via nested iteration. The neural network approach trains policy, value, and pricing networks jointly, avoiding explicit nested loops.

State-space bounds. Productivity bounds are identical to the basic model ($z \in [\exp(\mu \pm 3\sigma_{\text{erg}})]$). Capital and debt bounds are anchored to a tax-adjusted frictionless capital

$$k_{\text{ref}} = \left[\frac{(1 - \tau)\beta \mathbb{E}[z']}{\tilde{r} + \delta} \right]^{1/(1-\beta)},$$

which is the frictionless optimum after corporate profits are reduced by the tax rate τ . Similar to the default bounds used in basic model, capital is then bounded as $k \in [0.25 k_{\text{ref}}, 6 k_{\text{ref}}]$. Debt is pinned to capital via: $b_{\text{max}} = 3k_{\text{max}}$ with $b_{\text{min}} = -0.2b_{\text{max}}$ allowing cash holdings ($b < 0$). The form mirrors a standard collateral constraint where debt capacity scales with capital, but the multipliers are set by user and should be generous enough so the bounds never bind.

1.4 Overview of Solution Methods

The solution to the model is given by the optimal policy function $\pi^*(s)$ that maps states to actions. Optionally, solution also include the optimal state-value function $V^*(s)$. This section provides a high-level, brief summary of the solution methods using generic notations. I discuss the key ideas of each method, their main strength and limitations, and improvements. A more detailed and comprehensive documentation of each method (algorithm) are provided in [Appendix.A](#).

I implemented four main solution methods in Python and Tensorflow:

1. Value and policy function iteration (VFI/PFI)

2. Lifetime reward maximization (LRM) with terminal value correction
3. Euler residual minimization (ERM)
4. Short horizon actor critic method (SHAC)
5. Nested value function iteration (specific to the risky debt model)

VFI and PFI are the classical discrete dynamic programming solvers. They discretize the continuous state and action spaces onto finite grids, estimate Markov transitions from data, and iterate the Bellman operator to convergence. The resulting value function and policy are exact on the grid (up to discretization error) and serve as ground-truth benchmarks for the NN-based methods. Both methods are robust but they suffers from the curse of dimensionality. I use linear interpolation to reduce the grid approximation error and show that it avoids the overestimation bias due to the arg max operation on coarse grids. But the real “curse of dimensionality” is the number of state and action variables, and this is the main motivation for alternative methods. The [VFI/PFI appendix](#) describes my algorithm and implementation details.

Maliar, Maliar, and Winant (2021) introduces three **deep learning methods** that uses neural networks (NN) to approximate policy and value function: Lifetime reward maximization (LRM), Euler residual minimization (ERM), and Bellman residual minimization (BRM). I implemented original version of all these methods and find that only ERM is reliable for actual production. The LRM is plagued by systematic bias from finite horizon truncation, and the BRM usually failed to converge in practice. These defects and their “fixes” are discussed in next section in detail.

The **ERM method** minimizes violations of the first-order conditions (Euler equations) that characterize optimality. Rather than simulating full trajectories, it enforces an intertemporal necessary condition between (s, a) and (s', a') at each observation independently. At the optimum, the policy π_θ satisfies the Euler equation:

$$\mathbb{E}_\varepsilon [F(s, \pi_\theta(s), s', \pi_\theta(s'))] = 0$$

where $F : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the Euler residual function derived analytically from the first-order conditions of the Bellman equation, and $s' = f(s, \pi_\theta(s), \varepsilon)$ is the known state transition function. Two important implementation details:

1. The empirical loss is the squared Euler residual $\frac{1}{B} \sum_{i \in B} F(s_i, a_i, s'_{i,1}, a'_{i,1}) \cdot F(s_i, a_i, s'_{i,2}, a'_{i,2})$ using two i.i.d. draws $s'_{i,1}$ and $s'_{i,2}$, which is an unbiased estimator for $\mathbb{E}_\varepsilon [F(\cdot)^2]$.
2. Both Maliar, Maliar, and Winant (2021) and Fernández-Villaverde (2025) use a single policy network π_θ inside the Euler residual function F . In practice, this creates a moving-target problem that prevents convergence: the gradient of θ flows through both the current policy $\pi_\theta(s)$ and the next-period policy $\pi_\theta(s')$, which appear on both side of the Euler equation. To fix this, I uses a separate target network π_{θ^-} for the next-period action, which stablizes training and faciliates convergence.

The **LRM method** directly maximizes discounted cumulative rewards by simulating trajectories under the current policy. Given initial state s_0 and a shock sequence $\{\varepsilon_1, \dots, \varepsilon_T\}$, the policy π_θ generates a trajectory $\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$ where $a_t = \pi_\theta(s_t)$ and $s_{t+1} = f(s_t, a_t, \varepsilon_{t+1})$. Gradients flow backward through the entire trajectory via backpropagation through time (BPTT), requiring both the reward r and the endogenous transition f^{endo} to be differentiable with respect to the action. The objective is

$$\max_{\theta} V^\pi(s_0) = \mathbb{E} \left[\sum_{t=0}^{T-1} \gamma^t r(s_t, \pi_\theta(s_t)) \right] + \gamma^T \mathbb{E} [V^\pi(s_T)]$$

where the terminal value is truncated and implicitly set to $\mathbb{E}[V^\pi(s_T)] = 0$. This truncation is only valid when the finite-horizon rollout T is sufficiently large. For example, with discount factor $\gamma = 0.95$ and $T = 100$, the terminal value contribution of $0.6\% \approx (0.95)^{100}$ is negligible. In practice, the **LRM method** faces an important trade-off between bias and computational cost: If we set large T rollout, the computational cost of BPTT is huge and LRM is practically much slower than any other methods (including VFI/PFI). BPTT is sequential so it cannot be parallelized by Tensorflow. In contrast, LRM is feasible when rollout is moderate (e.g., $T \leq 30$) but the truncation bias would be large enough to systematically bias the solution π_{θ^*} .

I also implemented and tested the **Bellman residual minimization (BRM) method** following Maliar, Maliar, and Winant (2021). However, I find that this method is very unstable and may converge to a spurious, self-consistent fixed point different from the optimal policy. Maliar, Maliar, and Winant (2021) concludes that the main defect of BRM is it is less precise and requires careful tuning of hyperparameters to match the scale of the Bellman equation residual and the first order condition (FOC) residual. I show that the defect is structural and cannot be solved by fine tuning and warm start (pre-training). In short, the Bellman residual can be minimized for any arbitrary policy and only the first-order necessary conditions are providing useful gradient directions. The BRM method converges only when FOC dominates the Bellman residual in training and when the initiation of the value function network is around the “right” basin of the local optimum. This makes it dependent on pre-training and fine-tuning, and less useful in practice. Therefore I remove BRM from the main production methods and discuss the more fundamental defects in the [BRM appendix](#).

Finally, I introduced and implemented a new method, **Short-Horizon Actor Critic (SHAC)**, based on a revision of the reinforcement learning (RL) algorithm developed by Xu et al. (2022). This method requires four neural networks: a policy network parameterized by π_θ and a value function network parametrized by V_ϕ , and two polyak updated copies $\bar{\pi}_\theta$ and $\bar{V}_\phi$.

The basic design of SHAC is we can slice the full T horizon into T/H windows, each with length H , then exploits the H -step rollout and BPTT to accurately trained π_θ and V_ϕ within each window. Each gradient update consists of two steps: For each window $j = 0, \dots, T/H$, the actor step update the policy network π_θ that maximizes

$$\max_{\theta} V_\phi(s_j) = \mathbb{E} \left[\sum_{t=j}^{H-1} \gamma^t r(s_t, \pi_\theta(s_t)) \right] + \gamma^H \mathbb{E} [\bar{V}_\phi(s_H)]$$

then the critic step is supervised learning of the value network:

$$\min_{\phi} \mathbb{E} [V_\phi - y]^2$$

where the target label $y = \text{Stop Gradient}(\sum r^H + \gamma^H \bar{V}_\phi)$ is the Bellman right-hand-side value from the actor step. The intuition underlying actor critic method is very similar to VFI/PFI, where the policy evaluation find the policy that maximizes the Bellman and the tabular value function is updated using the improved policy.

The advantages of **SHAC** method are clear:

- it does not requires the existence of closed-form Euler equation
- it introduces a value network to precisely learned the terminal value omitted by LRM

- it uses short-horizon rollout and BPTT to reduce bootstrap error for the value network

I show that SHAC and ERM achieves the significantly better accuracy and robustness compared with LRM and BRM. Unlike ERM, SHAC does not require closed-form Euler equation and thus can be applied to more general set of models. The main cost is the computational expense of BPTT, but it can be fine-tuned to achieve a balance between stability (longer T) and speed (shorter T).

1.5 Data Generation, Reproducibility, and Workflow

1.5.1 Synthetic Data

Standard deep learning applications typically use three datasets:

1. Training data: splited into mini-batches to trained the NNs via SGD
2. Validation data: used to evaluate the quality of solution and convergence criteria
3. Test data: sealed and only used once to benchmark the results after the entire training is finished

I adopted a similar approach when training the NNs. The key design principles are:

- All different methods are trained on the same fixed training dataset
- Convergence/early stopping criteria are evaluated on a separate validation dataset

Unlike Maliar, Maliar, and Winant (2021) who simulated data on-the-fly during NN training, my approach **strictly separates data generation and NN training**. Because different methods are applied to the exact same datasets, their results can be fully reproduced, compared and benchmarked. Any discrepancies in results must be due to the effectiveness of solution methods rather than potential randomness in data simulation.

I simulated datasets in two general structures (with i denotes observation): 1. Full trajectory data: $\{(s_{it}, a_{it}, s_{1,i,t+1}, s_{2,i,t+1})\}_{t=0}^{T-1} \equiv \{(s_{it}, \pi_\theta(s_{it}), f(s_i, \pi(s_i), \epsilon_{1,it}), f(s_i, \pi(s_i), \epsilon_{1,it}))\}_{t=0}^{T-1}$ 2. One-period transition data: $(s_i, a_i, s'_{1i}, s'_{2i}) \equiv (s_i, \pi_\theta(s_i), f(s_i, \pi_\theta(s_i), \epsilon_{1,i}), f(s_i, \pi_\theta(s_i), \epsilon_{1,i}))$

where the one-period transition data is flattened and randomly shuffled from the full trajectory data. Full trajectory data is used by LRM and SHAC methods, and the one-period transition data is used by VFI/PFI, ERM and BRM methods. This design ensures that all these six methods are trained on the same data points even if their required data structure is different.

Note that for each period t , I take two iid draws $(\epsilon_{1,it}, \epsilon_{2,it})$ which is necessary to construct the unbiased estimator for the loss function in ERM and BRM methods. Another point is that the action a_{it} and next-period states $s_{1,i,t+1}$ depends on the current policy π_θ , so they must be generated during training. In practice, I separate exogenous states (e.g., AR(1) productivity shocks) that does not depend on θ from endogenous states (e.g., capital stock k) that depends on action π_θ (e.g., investment). Trajectory of exogenous states can be fully unrolled before training, and endogenous states are on-policy rollouts during training.

Application: To apply to both the basic investment model and the risky debt model, the data generation is (suppressing i index): 1. Build bounded state spaces $\mathcal{S} = [\underline{z}, \bar{z}] \times [\underline{k}, \bar{k}] \times [\underline{b}, \bar{b}]$ and action spaces $\mathcal{A} = [\underline{l}, \bar{l}] \times [\underline{b}, \bar{b}]$ from model environment 2. Sample initial states $z_0, k_0, b_0 \sim \text{Uniform}(\mathcal{S})$ at $t = 0$ 3. For $t = 0, \dots, T-1$, sample M independent shock sequences $\{\epsilon_{1,t}, \dots, \epsilon_{M,t}\}$ from $\mathcal{N}(0, 1)$ 4. Separate exogenous states z_t and endogenous states (k_t, b_t) , start from z_0 and unroll the full trajectories of $\{z_{1,t}, \dots, z_{M,t}\}_{t=1}^{T-1}$ using the state transition function $z_{t+1} = \rho \log z_t + \sigma \epsilon_{m,t}$ for $m = 1, \dots, M$. 5. Store the full trajectory data:

$$\mathcal{D}^{\text{traj}} = (k_0, b_0, z_0, \{z_{1,t}\}_{t=1}^{T-1}, \dots, \{z_{M,t}\}_{t=1}^{T-1})$$

6. Take the full trajectory data, flatten the exogenous states to only keep one-step sample (z_m, z'_m) between t and $t + 1$ for a given $m = 1, \dots, M$. Sample a new current-period endogenous state $k, b \sim \text{Uniform}(\mathcal{S})$, merge them, drop the t subscript and use $'$ to denote next-period variable. Randomly permutate the data to break the serial correlation (ordering) and store it as the one-step transition data:

$$\mathcal{D}^{\text{flat}} = (k, b, z, \{z'_m\}_{m=1}^M)$$

1.5.2 Reproducibility

All randomness in the project flows from a single integer pair of **master seed** (m_0, m_1) . TensorFlow stateless RNGs derive deterministic sub-seeds for three independent streams: **data generation** (the initial draws k_0, z_0, b_0 , the shock paths $\varepsilon^{(1)}, \varepsilon^{(2)}$, and the post-flatten row permutation that break the serial correlation of the $N \cdot T$ one-step transitions), **NN initialization** (policy and critic weights), and **SGD mini-batch ordering** (the `tf.data` shuffle iterator inside each trainer). Together these guarantee that two runs with the same master seed produce bit-identical data, identical initial parameters, and identical mini-batch order on the same machine.

The data-generation stream is the most structured. Each random quantity has a fixed integer identifier $\text{ID}(x)$, and for training step $j = 1, 2, \dots$:

$$\mathbf{s}_{x,j}^{\text{train}} = (m_0 + 100 + \text{ID}(x), m_1 + j), \quad \mathbf{s}_x^{\text{val}} = (m_0 + 200 + \text{ID}(x), m_1).$$

Training seeds advance with j so each round draws fresh shocks; validation seeds are fixed and shared across rounds and methods. The split offsets together with the per-variable IDs guarantee all streams are pairwise disjoint.

ID	Variable	Description
1	k_0	Initial endogenous capital
2	z_0	Initial exogenous productivity
3	b_0	Initial debt (risky-debt model only)
4	$\varepsilon^{(1)}$	Main AR(1) shock path
5	$\varepsilon^{(2)}$	Second draw of AR(1) shock path (for AiO cross product)
6	flatten	Post-flatten permutation of the $N \cdot T$ one-step transitions

The design delivers two benefits: **reproducibility** (same master seed $\rightarrow$ bit-identical experiment) and **common random numbers** (different methods trained at the same step j see the same data, so cross-method comparisons are paired and free of Monte-Carlo noise). Note that a separate **strict mode** (`strict_reproducibility=True`) additionally pins down kernel-level non-determinism inside TensorFlow itself (parallel reductions, GPU / Metal ops); it is only reserved for strict replication and debugging.

1.6 Validation of Solution

To verify the effectiveness and correctness of the solution methods, I benchmark them on a separate validation dataset that is fixed and identical across methods. I consider three main metrics for effectiveness:

1. **Policy MAE** against the true analytical optimal policy (only for frictionless basic model).
2. **Mean absolute Euler residual**: requires a closed-form Euler equation.
3. **Mean lifetime reward**: always feasible, the standard RL evaluation metric.

Policy MAE. For the frictionless basic model, the analytical anchor $k^*(z) = [\beta \mathbb{E}[z' | z] / (\bar{r} + \delta)]^{1/(1-\beta)}$ is exact. I evaluate the policy in next-period capital space $\pi_\theta(k, z) = k'$, where the analytical action $I^*(s)$ has been transformed back via the known endogenous transition:

$$\text{MAE}(\pi_\theta) = \frac{1}{N} \sum_{i=1}^N |k'_{\pi_\theta}(k_i, z_i) - k^*(z_i)|$$

with $\{(k_i, z_i)\}_{i=1}^N$ drawn from the flattened validation dataset and k^* clipped to the state-space bounds $[k_{\min}, k_{\max}]$. This is a direct correctness proof when the optimal policy is known.

Mean absolute Euler residual. When the model admits a closed-form Euler residual $F(s, a, s', a')$ (e.g., the basic model with smooth adjustment costs only), I report

$$|\overline{F}|(\pi_\theta) = \frac{1}{N} \sum_{i=1}^N |F(s_i, \pi_\theta(s_i), s'_i, \pi_\theta(s'_i))|,$$

on the same fixed validation dataset, where s'_i is computed via the known state transition under the validation-side AR(1) shock draw.

Mean discounted reward. When neither the analytical policy nor a closed-form Euler residual is available (e.g., the risky-debt model), I evaluate the policy by rolling it out on the validation trajectory dataset and computing

$$\bar{V}_T(\pi_\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \gamma^t \cdot r(s_t^i, \pi_\theta(s_t^i)),$$

where every trajectory uses the same pre-simulated exogenous shock path from the validation dataset, and T is set large enough that the truncation tail is negligible (default $T = 200$, giving $\gamma^T \approx 4 \times 10^{-5}$ at $\gamma = 0.95$). Because every method sees the same shocks, differences in $\bar{V}_T$ reflect policy quality rather than Monte-Carlo noise.

1.7 Results: Basic Investment Model

This section present results from solving the basic model using different methods. To validate the correctness and effectiveness of solution, I set capital adjustment costs to zero (frictionless) $\psi_0 = \psi_1 = 0$ so that we can benchmark the solution against a ground-truth optimal policy k^* . This experiment can be reproduced by running `docs/01_basic_investment_benchmark.ipynb`.

1.7.1 Convergence Curve

Figure 1.1 shows the three validation metrics from the [Validation of Solution](#) section across wall-clock training time (seconds), all evaluated on the same held-out validation dataset: mean absolute Euler residual (left), mean discounted reward (middle), and policy MAE against the analytical anchor (right). I do not use

the training loss as a convergence diagnostic, because a low loss does not necessarily imply a high-quality policy due to overfitting. The held-out metrics avoid this confound and let me compare every method on the same evaluation surface. Of the three, policy MAE (under frictionless basic model) is the strongest measure: the analytical true policy solution is exact, so zero MAE means the learned policy coincides with the closed-form optimum.

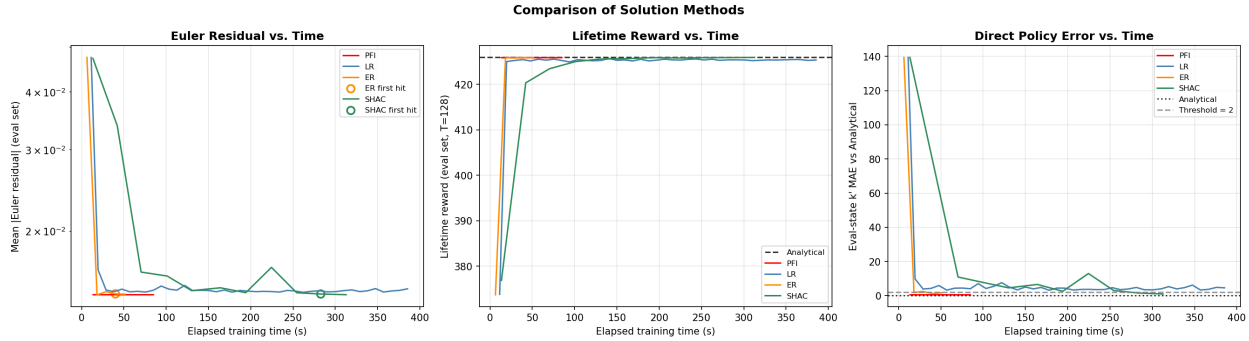


Figure 1.1: Convergence of policy MAE across methods.

The dashed line in the right panel marks the fixed MAE threshold ($= 2$) that defines convergence here. The threshold is held fixed across methods, so the relevant comparison is wall time to threshold rather than absolute loss. I find three key patterns:

- **PFI converges fastest and reaches the lowest MAE.** This is expected for a low-dimensional model. The Bellman operator is a γ -contraction on the discrete grid, so classical iteration remains very efficient at this scale.
- **ERM is the second-best method.** Its policy MAE drops below the threshold within a comparable wall-time budget, and the plateau sits close to the analytical solution.
- **SHAC also converged but takes longer.** This is consistent with its higher BPTT cost per gradient step.
- **LRM obtains close approximate but did not converged.** This is a structural bias due to terminal value truncation, as discussed below.

The most interesting case is LRM. Its policy MAE drops quickly in the early phase and then plateaus just above the threshold without ever crossing it. This is not a training artifact such as insufficient steps, a poor learning rate, or a small batch size. It is the structural defect of LRM described in [the LRM appendix](#). Even with the geometric-perpetuity terminal value correction, **LRM cannot recover the true on-policy continuation value up to precision $V^\pi(s_T)$** . The perpetuity can at-best approximate the stochastic future value with a deterministic steady state and ignores the firm's optimal response to future shocks. This leaves an $O(\sigma_\varepsilon^2)$ approximation error that is small but does not vanish with longer training. LRM therefore always have an approximation bias, regardless of training budget and sample size.

1.7.2 Learned Policy vs True Policy

Figure 1.2 provides a visual validation of the solutions. It plots two slices of the next-period capital policy $k'(k, z)$ for each method, against the closed-form analytical anchor $k^*(z) = [\beta E[z' | z] / (\bar{r} + \delta)]^{1/(1-\beta)}$:

- **Left panel: k' as a function of z , holding k fixed.** The analytical anchor depends only on z and rises monotonically with z , so the plotted curves are upward sloping.

- **Right panel: k' as a function of k , holding z fixed.** Because k^* is independent of k , the analytical anchor is a horizontal line. A correctly learned policy is also flat in this panel.

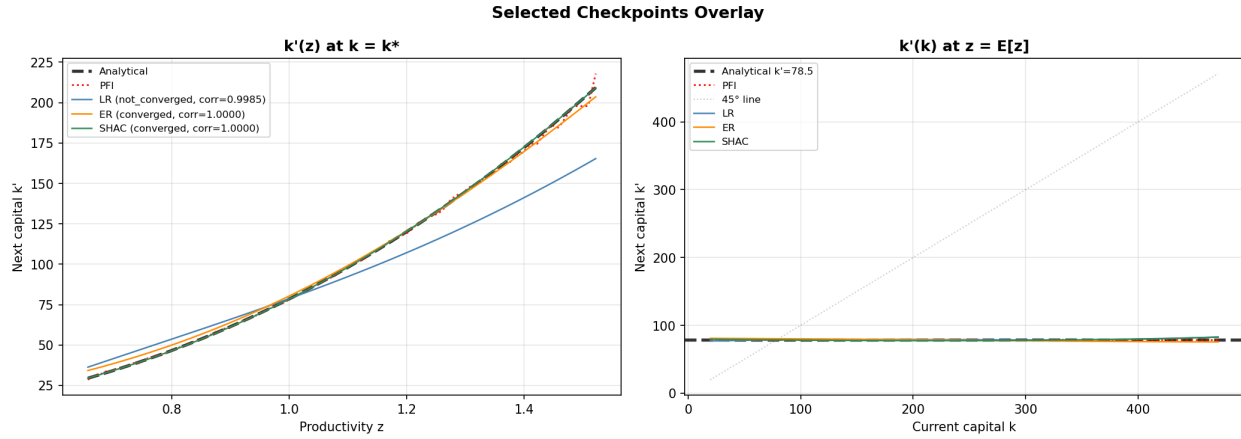


Figure 1.2: Learned policy against true policy by methods

The black dashed line is the analytical solution. The red dotted line is the PFI solution, which serves as the discrete-grid anchor. PFI, ERM, and SHAC all match the analytical solution to within the convergence threshold of policy $\text{MAE} \leq 2$ established in Figure 1.1, and their learned policies tightly coincide with the true policy.

The only exception is LRM. In the left panel, **LRM systematically underestimates k' when z is high.** This is exactly the terminal-value truncation error described in the LRM appendix. It is worth emphasizing that this gap remains nontrivial after the terminal value approximation is already implemented by adding $\hat{V}_T^{\text{term}}(s_T^{\text{endo}})$ to the LRM objective function. Without this correction, the downward bias would be much larger.

The underlying intuition is that the perpetuity deliberately pins the exogenous component at the stationary mean $\bar{z}$ instead of conditioning on the realized z_T . AR(1) persistence means the rollout typically ends at high z_T when the trajectory started from high z , but the perpetuity ignores this conditioning and therefore underestimates the true continuation $V^\pi(s_T)$. The actor's gradient then underweights the long-run benefit of investing at high z , and the learned k' is pulled toward the interior. **This bias cannot be reduced by more training, more data, or a smaller learning rate**, because the gap lives in the analytic terminal correction itself rather than in the optimization. The only way to close it is to replace the perpetuity approximation with a learned value network, which is exactly what SHAC does.

1.7.3 Reproducing the Original Maliar et al. (2021) Methods

Now I present three additional experiments that strictly reproduce the original methods in Maliar, Maliar, and Winant (2021): Euler Residual Minimization (ERM), Lifetime Reward Maximization (LRM), and Bellman Residual Minimization (BRM). All three are applied to the frictionless basic investment model. The goal is to evaluate the methods as they are published, identify their key defects, and explain why we need patches and/or better alternative methods like SHAC to correctly solve the corporate finance models.

- **Figure 1.3** reports each method's **training loss** as a function of training step.
- **Figure 1.4** reports the three **held-out validation metrics** from the Validation of Solution section: mean absolute Euler residual, mean discounted lifetime reward, and policy MAE against the analyt-

ical anchor.

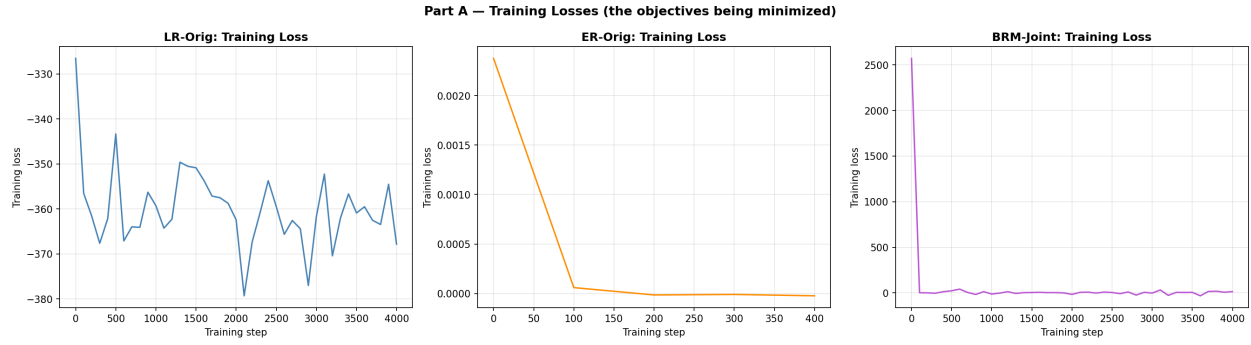


Figure 1.3: Loss function of original Maliar, Maliar, and Winant (2021) methods

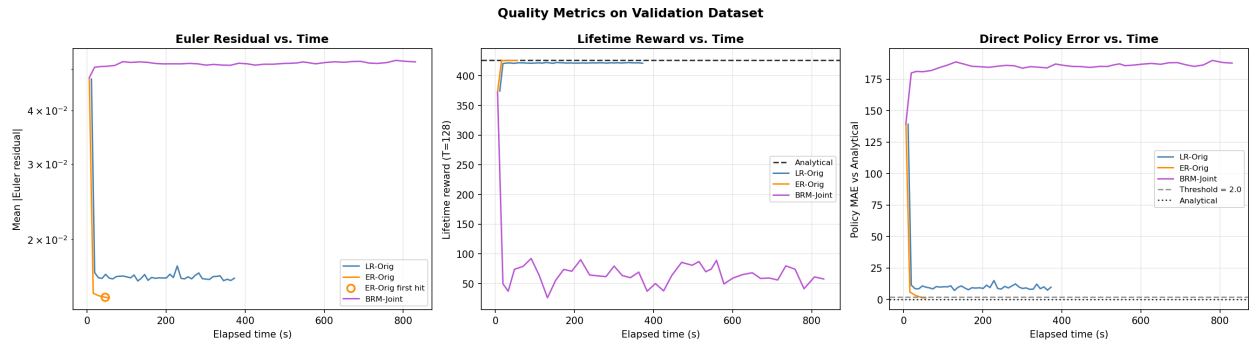


Figure 1.4: Validation metrics of original Maliar, Maliar, and Winant (2021) methods

Reading these two figures side by side reveals an important point: **a low training loss does not imply a correct solution**. Specifically, the BRM loss is “correctly” minimized as visualized in Maliar, Maliar, and Winant (2021), but the actual solution is wrong and the training shows no sign of improvement.

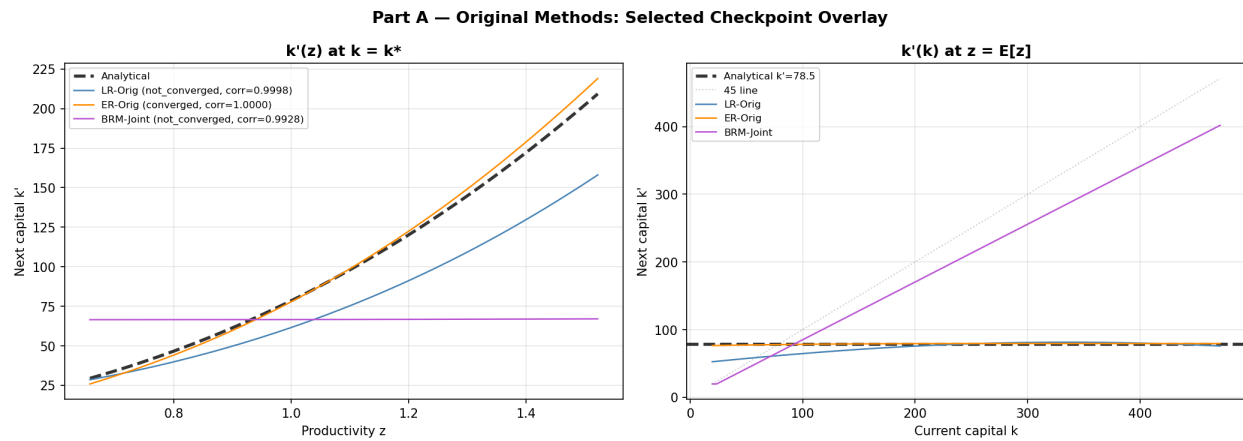


Figure 1.5: Learned policy against true policy of original Maliar, Maliar, and Winant (2021) methods

ERM is the only method that converges. ERM’s training loss in Figure 1.3 and its validation metrics

in Figure 1.4 drop together: the held-out policy MAE crosses the convergence threshold and the Euler residual settles near zero. It is the only one of the three Maliar, Maliar, and Winant (2021) methods that converged when applied to the basic model.

LRM converges in training but under-invests at every z . LRM’s training loss in Figure 1.3 descends steadily, yet Figure 1.4 shows the held-out policy MAE plateaus well above the convergence threshold. This is the terminal value truncation bias detailed in the LRM appendix. The original LRM drops every period of cash flow beyond the rollout horizon T from the objective, which under-weights the long-run benefit of investment at every state. The resulting policy under-invests uniformly in z . My patched LRM applies a deterministic-perpetuity correction that reduces this to a high- z -only bias.

BRM diverges despite a near-zero training loss. BRM’s training loss in Figure 1.3 decreases monotonically and reaches near-zero, matching Maliar, Maliar, and Winant (2021)’s training-loss curves. Yet Figure 1.4 shows BRM’s held-out policy MAE never actually improved and the learned policy is qualitatively wrong.

This refutes an implicit claim in Maliar, Maliar, and Winant (2021): that joint minimization of the Bellman residual and the FOC residual is sufficient to identify the optimal policy. The paper argues this by analogy without a formal proof, and my experiment shows the analogy fails.

The mechanism is intuitive and detailed in the BRM appendix. In economic environments, the Bellman residual is orders of magnitude larger than the FOC residual, so gradient descent drives down the Bellman loss first. The value network therefore learns to satisfy the Bellman equation for whatever policy emerges, not for the optimal policy. The policy network then minimizes the FOC against this wrong value function. Both losses become small, but the system locks into a self-consistent fixed point that depends on the value network’s initialization. Maliar, Maliar, and Winant (2021) proposes fine-tuning the exogenous weights to balance the two, but they can only partially mitigate the symptom and fail to prevent divergence.

This is why I reject BRM method for production. The new SHAC method is a direct fix to the BRM method and it adopts the canonical actor-critic method design in RL to ensure the convergence to the correct and unique fixed point.

1.8 Issues in Neural Network Architecture and Training

Standard VFI and PFI methods are “simple” and robust because its convergence is guaranteed by contraction mapping theorem. In contrast, I find that there are many details that are critical for Neural Network (NN) based methods to work, and these practical issues are often omitted by the higher-level algorithm summary in original papers Fernández-Villaverde (2025).

Table below summarizes the issues specific to the three methods introduced by Maliar, Maliar, and Winant (2021):

Method	Major Defects	Minor Defects	Usability	Reference
Euler Residual Min	None	Single policy network is unstable. Solved by adding a target policy network	Fast and robust for production, but requires existence of a closed-form Euler equation	Fernández-Villaverde (2025)

Method	Major Defects	Minor Defects	Usability	Reference
Lifetime Reward Max	Terminal value truncation bias	Long-horizon backpropagation through time (BPTT) is slow and costly	Can be used as rough baseline, but not ideal for production when accuracy/unbiasedness matters	(Maliar, Maliar, and Winant 2021)
Bellman Residual Min	Can easily converge to “wrong” but self-consistent fixed point	Conflicting gradients due to scale mismatch of loss functions; Require existence of closed-form first order condition	High-risk and strongly rejected for production	Fernández-Villaverde (2025)
Short Horizon Actor Critic	None	Slower than ERM due to BPTT, but can be fine-tuned to improve speed	Unbiased. Most generalized and flexible method. Does not require closed-form Euler equation or FOC.	(Xu et al. 2022)

In addition, I rank the general issues (shared by all methods) based on their importance in practice:

General Issues	Description	Solutions	Results
Smooth and differentiable reward and dynamics	This is a fundamental prerequisite for gradient-based training to work	Kinks can still be handled, but discrete choice or jump discontinuities can only be approximated with error	For the basic model with fixed adjustment cost, NN-based methods cannot learn the inaction regions; Soft-surrogate suffers nontrivial approximation error; VFI/PFI is strictly better
Input Normalization	Raw data are measured in level and large units can easily de-stabilize training	Normalize input to z-score and re-scale it back to economic levels as NN output head	Hidden layer only see normalized inputs and is agnostic to environment
Network output head	Sigmoid, Tanh, and other activation can suppress gradient and prevent learning at extreme values	Use raw linear (no activation function) with affine transformation	Gradient is uniformly “strong” across state/action space; Output variable converted back to original unit

General Issues	Description	Solutions	Results
Hidden layer activation	For economic models, ReLU is not stable, Sigmoid and Tanh cause vanishing gradient	SiLU swish always perform better in practice	Gradient is stable and nonzero
Full reproducibility	Comparison across methods should be fair and fully reproducible	Separate data generation and training; Full schedule of random number generator (RNG)	All methods are trained on exactly the same fixed dataset and results are fully controlled by master seeds
Convergence metric	Objective and loss function are NOT the correct metrics for the quality of solution	Measure effectiveness of learned policy in a separate validation dataset	Avoid overfitting, enable early stopping based on same criteria, fair comparison across methods

The architecture-level choices in this table are documented in detail in [Implementation Details](#): input normalization, hidden-layer activation, output head transformation.

1.9 Risky Debt Model

1.9.1 Solution method: Nested VFI

To solve the risky debt model described in Strebulaev and Whited (2012, sec. 3.6), I implemented a **nested VFI algorithm**. I find that nested VFI is still the best method for this model in terms of speed, robustness, and accuracy. In contrast, I find that all three of Maliar, Maliar, and Winant (2021)’s methods **cannot be applied to solve the risky debt model** because the model has a nested fixed-point structure: the firm’s value function V depends on the bond’s risky rate $\tilde{r}$, and $\tilde{r}$ depends on V because the lender prices the bond using the default states implied by V .

I solve the model with a two-level VFI iteration. The **inner loop** is a standard VFI that solves V on the discrete grid for a fixed pricing schedule. The **outer loop** updates $\tilde{r}$ to be consistent with the default partition implied by the latest V . The algorithm terminates when both loops converge, so the value function and the pricing schedule are mutually consistent.

On top of the standard algorithm described in Strebulaev and Whited (2012) and used by Hennessy and Whited (2007), I developed two algorithm refinements. I argue that these two refinements significantly **improve the speed of nested VFI without sacrificing accuracy**. The saving in compute can be especially large for applications of the simulated method of moment (SMM) where the computational bottleneck is re-solving the model repeatedly over many optimization steps.

1. **Linear interpolation on z-grids**, which significantly increases the speed without hurting precision. The refined nested VFI algorithm can solve a $25 \times 25 \times 15$ in just a few minutes on standard CPU. Currently none of the deep learning methods I’ve tried can achieve similar performance.
2. **Adaptive b -bound around the default boundary**: VFI is expensive and we want to spend the compute on the economically meaningful regions of (k', b') near the default boundary. I added a pre-training stage to VFI that pins down the default boundary first with coarse grid configurations (e.g., $10 \times 10 \times 5$) before running the full algorithm on finer grids.

Below is a brief summary of the algorithm. The full details are in the [nested-VFI appendix](#).

Input: Discrete grids for capital, debt, and productivity, Markov transition matrix for the productivity shock, model primitives, risk-free rate r , convergence tolerances $\varepsilon_{\text{inner}}$ and $\varepsilon_{\text{outer}}$.

Output: A mutually consistent value function V^* and risky-rate schedule $\tilde{r}^*$.

1. Initialize the pricing schedule at the risk-free rate: $\tilde{r} = r$ everywhere.
2. **For** $n = 1, 2, \dots$ **do**:
3. **(a) Inner VFI.** With $\tilde{r}$ held fixed, run standard value function iteration on the discrete grid. Cap the value at zero whenever the Bellman maximand is negative, so the firm defaults exactly when $V \leq 0$. Iterate to convergence in sup-norm and store the result as $V^{(n)}$.
4. **(b) Outer convergence check.** If $\|V^{(n)} - V^{(n-1)}\|_{\infty} < \varepsilon_{\text{outer}}$, stop and return $(V^{(n)}, \tilde{r})$.
5. **(c) Default partition.** For each next-period choice (k', b') , partition the future productivity grid into default states ($V^{(n)} = 0$) and solvent states ($V^{(n)} > 0$).
6. **(d) Update pricing.** For each (z, k', b') , solve the lender's zero-profit condition for the new $\tilde{r}(z, k', b')$ given the default partition. This pins down the risk premium that compensates lenders for default probability and recovery loss.
7. **End for.**

1.9.2 Results

The solution of the risky debt model are visualized in three set of plots: 1. Value function slice $V(z, k, b)$ against each one of the argument 2. Policy function slice $k'(z, k, b)$ and $b'(z, k, b)$ against each of the argument 3. Critical value of next-period shock z'_d determining default boundary: for a given optimal action (k', b') , any realization $z' < z'_d$ means default, vice versa.

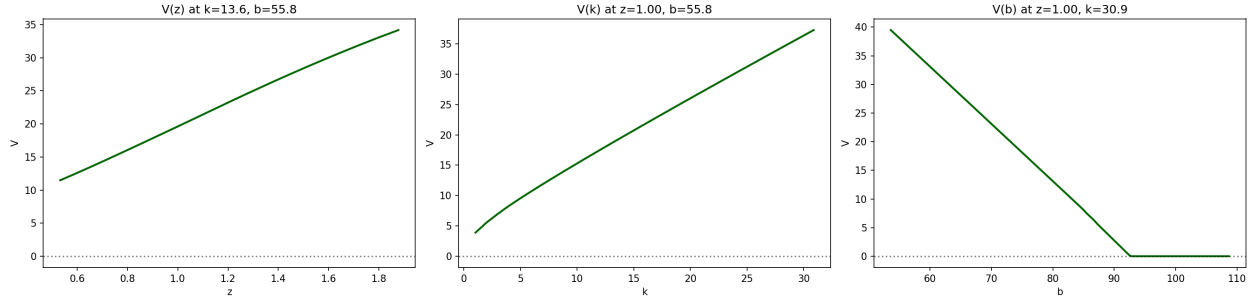
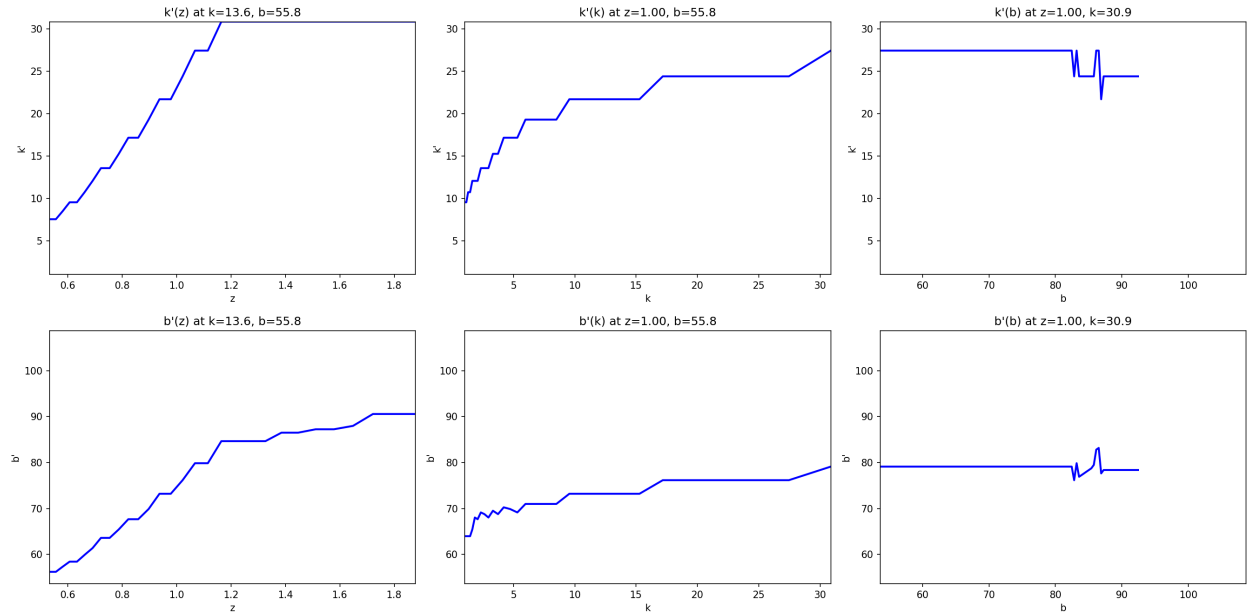
The risky debt model does not have closed form Euler equation or analytical solution under special cases (e.g., frictionless), the only feasible validation metric is the sum of lifetime reward (higher means better) — but it is only useful for comparison across methods. Instead, I rely on an **economic diagnostic test** to check the solutions. Specifically, I verify whether the comparative statics are consistent with the theoretical propositions proved in Hennessy and Whited (2007):

- $V(z, k, b)$: Firm value should be increasing in k , decreasing in b , and non-decreasing in z (Proposition 3 and 4)
- Critical value z'_d should be increasing in b' and decreasing in k' (Proposition 6).

My results in Figure 1.6 and Figure 1.8 are consistent with these implications. In addition, we know from the basic model that the optimal investment policy k' should be increasing in z and k when there exist adjustment costs. This also matches the first two panels in Figure 1.7.

Neither Hennessy and Whited (2007) or Strebulaev and Whited (2012) derive any comparative statics regarding $k'(z, k, b)$ and $b'(z, k, b)$, so I interpret them as ambiguous. That said, from the basic model we know that $\partial k' / \partial z > 0$ and $\partial k' / \partial k > 0$ when there exist adjustment costs, which are consistent with Figure 1.7. The remaining results are also economically reasonable:

- b' is increasing in z and k : firm can borrow more without default risk when productivity is high or when capital stock is large (buffer)
- k' and b' is independent of current b : this is consistent with the model where issuing a one-period corporate bond does not entail any frictional costs

Figure 1.6: Value function slices $V(z, k, b)$ of the nested-VFI risky-debt solution.Figure 1.7: Policy function slices $k'(z, k, b)$ and $b'(z, k, b)$ of the nested-VFI risky-debt solution.

Finally, Figure 1.8 provides a great summary of the results. The top left panel visualizes the default boundary governed by a critical value of z'_d . The economic intuition is that at current period given (z, k, b) , firm takes the optimal action govern by policy (k', b') and rational expectation of dynamics $P(z'|z)$. When next period shock z' realized, there exist non-zero probability that even the most optimistic realization $z' = z_{\max} \approx 1.8$ cannot prevent default. This forms the default boundary (dark navy) across the (k', b') space. Moreover:

- Higher k' reduces default probability, so z'_d is monotonically decreasing in k'
- Higher b' increases default probability, so z'_d is monotonically increasing in b'

The bottom panels of Figure 1.8 summarizes the relationship between endogenous bond yield (debt discount) $1/(1 + \tilde{r})$ and (k', b', z) . Higher default risks increase $\tilde{r}$ and thus reduce bond yield, therefore:

- Debt discount is increasing in z and k' because of lower default risk
- Debt discount sharply declines in b' between $b' \approx 70 - 80$. This cliff captures the default boundary and the default risks are priced into $\tilde{r}$

- When default risks are low (e.g., high z , k' and/or high b'), the debt discount is close to risk-free rate $1/(1+r) \approx 0.96$ (dotted line)

In summary, the results are consistent with economic rationals of the risky debt model and both the inner and outer loop of VFI converged up to high precision (error $< 10^{-6}$). I consider this as strong evidence of the correctness and effectiveness of the solution.

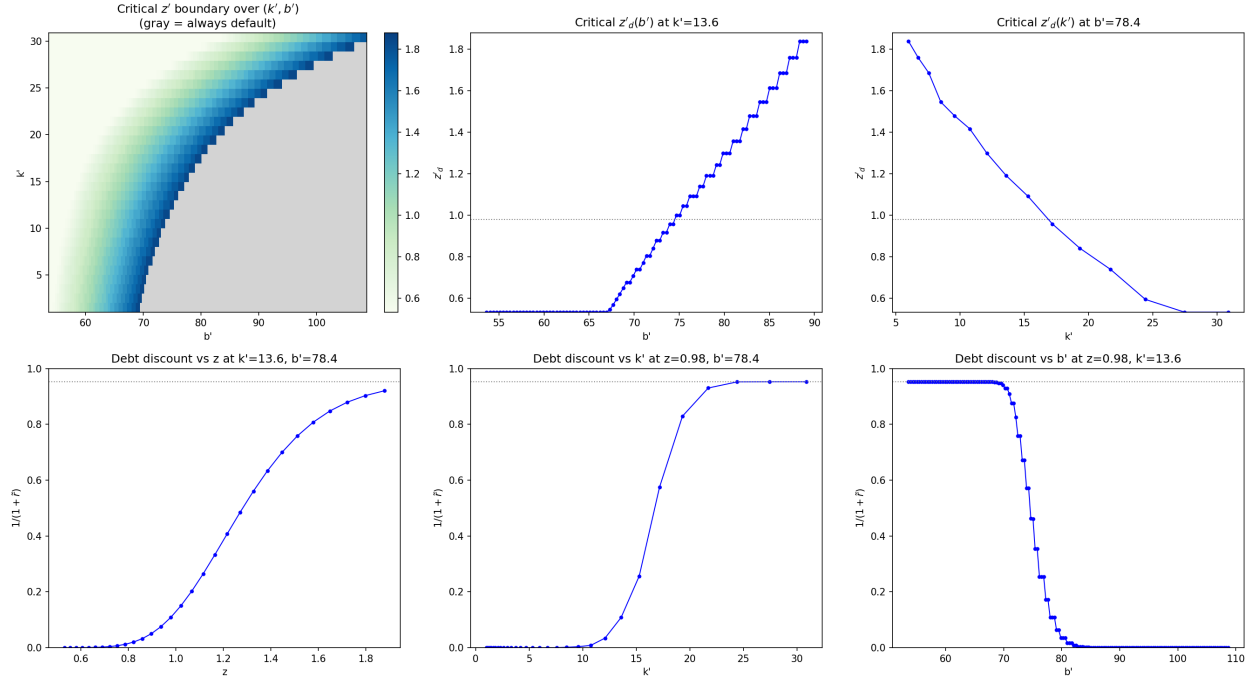


Figure 1.8: Default boundary z'_d over (k', b') and endogenous debt discount $1/(1+\tilde{r})$ across states.

1.9.3 Why deep learning methods failed for this model?

For the three methods developed by Maliar, Maliar, and Winant (2021): LRM does not learn V and $\tilde{r}$, ERM requires closed-form Euler equation that is not feasible for this model, and BRM is rejected due to structural defect. None of them are suitable for this nested fixed-point problem.

One promising solution methods to this model is Short-Horizon Actor Critic (SHAC), which include a separate value NN training and does not require closed-form Euler. However, I benchmarked the solution of SHAC to nested VFI and find that **SHAC systematically learned a more conservative policy (lower leverage-to-asset ratio)**. This is a consequence of the theoretical model's structure and should be **common to all actor-critic methods**:

- During training, value network V is initially inaccurate and gradually improving
- But the initially biased V directly determine the default set and interest rate, which affect the policy learning and the target V in next iteration
- Both policy and value network converged to a self-consistent but over-pessimistic (low leverage) or over-optimistic (high leverage) equilibrium, which is sensitive to the NN initiation weights ϕ for the value function V_ϕ .

In practice, my experiments show that SHAC solutions are usually pessimistic (low leverage) because initial V network tends to underestimate $\partial V / \partial b$ and lead to conservative policy (low leverage and less default

states), which leads to a more conservative target value network in next iteration and self-reinforcing policy updates.

There are two promising fixes: (1) use stochastic policy method instead of a deterministic policy to explore off-policy and with scalar cash flow (reward) acting as a score function. This allows the training to explore high-leverage states (default) off-policy. (2) switch to a standard trade-off model as in Nikolov, Schmid, and Steri (2021) where default states (and endogenous interest rate) does not depend on the value function and can be written down analytically only in terms of current states.

However, stochastic policy methods are usually not sample-efficient. For our model, nested VFI method is clearly faster and more robust. Fix (2) is a practical choice if we are willing to deviate from this version of the endogenous default model, but the standard VFI still performs better for this low-dimensional problem with few states.

Chapter 2

Structural Estimation

2.1 GMM and SMM

I implemented and tested both GMM and SMM methods to structurally estimate model parameters. I measure effectiveness using the basic investment model because it is computationally cheaper. The basic idea of the Monte Carlo (MC) validation is:

For MC replication $j = 0, \dots, J$: 1. Set replication count $j = j + 1$. Select a set of *true* parameters β_j^* , solve for the optimal policy $\pi^*(\cdot|\beta_j^*)$, use it to simulate a “target” panel dataset of N i.i.d. firms over T periods. 2. Start with a random guess $\beta_j^0 \neq \beta_j^*$, apply GMM or SMM to the target dataset, obtain a set of estimated params $\hat{\beta}_j$ and variance-covariance matrix. 3. Conduct t-test $H_0 : \hat{\beta}_j = \beta_j^*$ and expect failure to reject the null. Conduct over-identifying test and verify if we fail to reject the hypothesis of model mis-specification.

When all J replications completes, compute diagnostics including the average bias $\frac{1}{J} \sum_j (\hat{\beta}_j - \beta_0^*)$, Root Mean Square Error (RMSE), average rejection rate of over-identifying test, etc. The detailed diagnostic formulas are defined in the [SMM appendix](#).

If the pipeline is correctly implemented, three properties must hold. (i) The optimizer must reach an interior minimum, so moment errors at $\hat{\beta}$ should be near zero. (ii) On a single panel, $\hat{\beta}$ should fall within its sandwich SE of the truth, and the t-tests and J-test should fail to reject at 5%. (iii) Across MC replications, the empirical bias should be small relative to the within-replication SE, the empirical SD across replications should match the average SE (so confidence intervals built from one panel have approximately correct coverage), and the J-test rejection rate at the 5% nominal level should not exceed roughly 0.05. A failure on any one localizes the defect: failure of (i) implies the optimizer; failure of (ii) the point estimate or SE formula; failure of (iii) finite-sample anti-conservativeness in the asymptotic SE. The three result tables in each subsection below test these three properties in turn.

I consider the implementation to be correct only if our MC replication can consistently estimate $\hat{\beta}_j$ close to the true β_j^* . The shock realization of replication is controlled by master seeds and are fully reproducible.

For the actual application, we can replace step 1 with one real-world “target” dataset such as the Compustat firm panel data, and we only apply Step 2-3 once.

There are several important implementation issues:

- GMM uses Euler equation to form the moment condition, so it does not require solving the model

- SMM typically requires re-solving the model for optimal policy in Step 1 for each candidate β for evaluation. This is the main computational bottleneck. For this validation, I use the frictionless basic model with analytical solution to the optimal policy to avoid the cost. This validates the correctness of the entire SMM pipeline and separate potential errors of model solver (e.g. VFI/PFI) from estimation.
- Both GMM and SMM requires choosing appropriate global and local optimizer to find β^* that minimizes the moment condition error. I implemented both the simulated annealing optimizer used by Hennessy and Whited (2007) and the “differential evolution” optimizer search for β^* . These optimizers are built in `scipy.optimize`.

2.1.1 GMM Validation on the Basic Investment Model

I validate the GMM implementation on the basic investment model with smooth (convex) capital adjustment cost, where the Euler equation has a closed form. The Euler conditions provide moment restrictions that are evaluated directly from the observable panel (π, k, I) , so each $Q(\beta)$ evaluation in the optimizer is arithmetic on the data and the model is never re-solved inside the optimizer loop. The results can be reproduced by running `docs/04_gmm_validation.ipynb`.

Validation design.

- **Model.** Basic investment with convex adjustment cost only ($\phi_1 = 0$).
- **Parameters.** $\beta = (\alpha, \psi_1, \rho, \sigma)$: production elasticity, convex-cost coefficient, AR(1) persistence, and AR(1) shock std-dev. Interest rate ($r = 0.04$) and depreciation rate ($\delta = 0.15$) are calibrated externally.
- **Truth and initial guess.** True parameters $\beta^* = (0.60, 0.10, 0.70, 0.15)$. Optimizer starts from $\beta_0 = (0.480, 0.500, 0.475, 0.500)$.
- **Moments (6).** Three Euler-orthogonality conditions (Euler $\times 1$, Euler $\times I/k$ lag, Euler $\times \pi/k$ lag), two shock-orthogonality conditions (Shock $\times 1$, Shock $\times \ln z$ lag), and one variance condition.
- **Two-step GMM.** Stage 1 minimizes the moment distance with $W = I$ using `dual_annealing` for global search and Powell refinement. Stage 2 warm-starts from the Stage-1 estimate and minimizes with $W = \hat{\Omega}^{-1}$ using Powell only. Standard errors come from the Stage-2 sandwich formula.
- **“Real” data simulation.** I first run a single PFI solve at β^* on a dense grid (exogenous = 50, endogenous = 100, action = 100), then simulate $N = 100$ firms over $T = 25$ periods after a 275-period burn-in to ensure ergodic sampling. The GMM estimator treats (π, k, I) as observed data and never re-solves the model. PFI approximation error in the policy enters the Euler residuals as a small systematic bias, kept below the sampling noise floor by the dense grid.
- **Monte Carlo.** $J = 20$ independent panels, each generated from β^* under a different master seed; the full two-step procedure is run on each.

Result 1: optimizer reaches an interior minimum. If GMM is correctly implemented, the six moment conditions at $\hat{\beta}$ should be near zero, well below the sampling noise floor. Table 2.1 confirms this: five of six conditions are below 10^{-3} and the largest residual (Shock $\times 1$) is 3.5×10^{-3} . The Stage-1 global search and Stage-2 Powell refinement together find an interior minimum.

Moment	$g(\hat{\beta})$
Euler $\times 1$	-9.23×10^{-4}
Euler $\times I/k$ lag	-1.70×10^{-4}
Euler $\times \pi/k$ lag	-2.66×10^{-4}
Shock $\times 1$	-3.55×10^{-3}
Shock $\times \ln z$ lag	6.87×10^{-4}

Moment	$g(\hat{\beta})$
Var $\times 1$	-3.71×10^{-7}

Table 2.1: Moment-condition vector $g(\hat{\beta})$ on a single representative panel.

Result 2: point estimate and sandwich SE are correctly calibrated on a single panel. If they are, every estimate should be within roughly 1 SE of the truth and the parameter t-tests should fail to reject at 5%. Table 2.2 confirms both: every estimate is within 1 SE of β^* , all four parameter t-tests have $p > 0.10$ (the smallest is σ at $p = 0.11$), and the over-identification J-test is also insignificant ($J = 4.11$, $p = 0.13$, $df = 2$). The single-panel pipeline behaves correctly.

Parameter	True	Estimate	SE	t -stat	p -value
α	0.60	0.5999	0.0006	-0.14	0.89
ψ_1	0.10	0.0861	0.0176	-0.79	0.43
ρ	0.70	0.6817	0.0154	-1.18	0.24
σ	0.15	0.1466	0.0021	-1.60	0.11

Table 2.2: Parameter estimates, sandwich SEs, and t-tests against the true parameter. J-statistic = 4.11, $p = 0.13$, $df = 2$.

Result 3: point estimate is unbiased on average and asymptotic SE is correctly calibrated. This requires three things together: bias should be much smaller than the within-replication SE, the empirical SD across replications should match the average SE, and the J-test rejection rate at 5% nominal should be ≈ 0.05 . Table 2.3 partially confirms this. The point estimates are economically close to the truth (relative bias is below 4% for every parameter). However, the empirical SD across replications is systematically 1.3–2 $\times$ the average within-replication SE, so the asymptotic sandwich formula understates the true sampling variability. This propagates into J-test over-rejection at 0.20 vs the nominal 0.05. Both failures are consistent with PFI grid approximation in the panel-generation step entering the Euler residuals as a small systematic bias.

Parameter	True	Mean estimate	Bias	RMSE	SD across MC	Avg SE
α	0.60	0.6015	0.0015	0.0021	0.0016	0.0008
ψ_1	0.10	0.1345	0.0345	0.0576	0.0473	0.0332
ρ	0.70	0.6856	-0.0144	0.0303	0.0273	0.0188
σ	0.15	0.1506	0.0006	0.0028	0.0028	0.0022

Table 2.3: Monte Carlo summary across $J = 20$ replications. J-test reject rate at 5% is 0.20.

Summary. Predictions (i) and (ii) hold cleanly. Prediction (iii) is partially confirmed: the point estimate is essentially unbiased, but the asymptotic SE is mildly anti-conservative and the J-test correspondingly oversized. Both shortfalls are attributable to PFI grid approximation in the panel-generation pipeline rather

than the GMM core. With this caveat, the GMM machinery (moment construction, two-step weighting, optimizer, sandwich SE, J-test) is correctly implemented on a model with a closed-form Euler equation.

2.1.2 SMM Validation on the Frictionless Basic Investment Model

I validate the SMM implementation on the frictionless basic investment model, where the analytical policy is exact and there is no model-solve error. Any deviation between the estimated and true parameters in this experiment must come from the SMM machinery itself: the moment construction, the two-step weighting, the global / local optimizer, or the standard-error formula. This isolates SMM correctness from model-solution correctness. The results reported below can be reproduced by running `docs/05_smm_validation.ipynb`.

Validation design.

- **Model.** Frictionless basic investment ($\phi_0 = \phi_1 = 0$). The optimal policy is $k^*(z)$ given analytically. This eliminates any iteration error from VFI / PFI / NN solvers.
- **Parameters.** $\beta = (\alpha, \rho, \sigma)$. Interest rate ($r = 0.04$) and depreciation rate ($\delta = 0.15$) are calibrated externally.
- **Truth and initial guess.** True parameters $\beta^* = (0.60, 0.70, 0.15)$. Optimizer starts from $\beta_0 = (0.525, 0.475, 0.480)$.
- **Moments (4 in total, overidentified by 1).** Mean and variance of I/k , serial correlation of I/k , and the residual std of an AR(1) regression on log income.
- **Two-step SMM.** Stage 1 minimizes the moment distance with $W = I$ using `differential_evolution` for global search and Powell refinement. Stage 2 warm-starts from the Stage-1 estimate and minimizes with $W = \hat{\Omega}^{-1}$ using Powell only. Standard errors come from the Stage-2 sandwich formula.
- **Simulation budget.** $N = 500$ firms, horizon $T = 25$, burn-in = 75, and $S = 50$ simulated panels per moment evaluation.
- **Monte Carlo.** $J = 10$ independent fake-real panels, each generated from β^* under a different master seed.

Result 1: optimizer reaches an interior minimum. If SMM is correctly implemented, the simulated moments at $\hat{\beta}$ should match their data-side targets to within Monte-Carlo noise. Table 2.4 confirms this: every fitted moment is within $\sim 1\%$ of its target, so the Stage-1 global search and Stage-2 Powell refinement find an interior minimum.

Moment	Target	Fitted
Mean I/k	0.1927	0.1916
Var I/k	0.0925	0.0926
Serial corr I/k	-0.1427	-0.1432
AR(1) resid std	0.2124	0.2123

Table 2.4: Moment fit at $\hat{\beta}$ on a single representative panel.

Result 2: point estimate and sandwich SE are correctly calibrated on a single panel. If they are, every estimate should fall within roughly 1 SE of the truth and the parameter t-tests should fail to reject at 5%. Table 2.5 confirms both: every estimate is within 0.2 SE of β^* and all parameter t-tests have $p > 0.85$. The over-identification J-test is also insignificant ($J = 0.62$, $p = 0.43$, $df = 1$).

Parameter	True	Estimate	SE	t -stat	p -value
α	0.60	0.6007	0.0079	0.09	0.93
ρ	0.70	0.7018	0.0156	0.12	0.91
σ	0.15	0.1502	0.0010	0.19	0.85

Table 2.5: Parameter estimates, sandwich SEs, and t-tests against the true parameter. J-statistic = 0.62, $p = 0.43$, $df = 1$.

Result 3: point estimate is unbiased on average and asymptotic SE is correctly calibrated. This requires bias much smaller than the within-replication SE, empirical SD $\approx$ average SE, and J-test rejection rate at 5% nominal not exceeding 0.05. Table 2.6 confirms all three. Bias is more than an order of magnitude below SE for every parameter, the empirical SD agrees with the average SE within $\sim 30\%$ (consistent with the small MC sample of $J = 10$), and the J-test never rejects across 10 replications; 0/10 is statistically consistent with any size at or below 0.05.

Parameter	True	Mean estimate	Bias	RMSE	SD across MC	Avg SE
α	0.60	0.6003	0.0003	0.0084	0.0089	0.0085
ρ	0.70	0.6969	-0.0031	0.0166	0.0172	0.0159
σ	0.15	0.1505	0.0005	0.0015	0.0016	0.0012

Table 2.6: Monte Carlo summary across $J = 10$ replications. J-test reject rate at 5% is 0.00.

Summary. All three predictions hold. The SMM machinery (moment construction, two-step weighting, optimizer, sandwich SE, J-test) is correctly implemented on a model whose ground truth is known. This confirms the correct implementation of the SMM pipeline except for the model solver (e.g., VFI) itself.

2.2 Applying SMM to the risky debt model

Applying SMM to the basic investment model identifies four parameters: production-function curvature (α), smooth adjustment cost (ψ_1), AR(1) persistence (ρ), and AR(1) shock variance (σ). Adding costly equity issuance from section 3.3 of Strebulaev and Whited (2012) brings two more parameters into scope: the fixed and proportional cost components (η_0 and η_1). The full endogenous-default model adds one final parameter, the deadweight bankruptcy cost c_{def} , the fraction of firm value lost when the firm defaults. From a pure estimation perspective, the endogenous-default extension only adds one parameter; the rest can be estimated from the simpler frictional model in section 3.3.

The cost of applying SMM to risky debt model is computational: each candidate β in the optimizer's inner loop requires a fresh nested-VFI solve on the discrete (k, b, z) grid. A Monte-Carlo replication study at this scale is infeasible (on my current device), so I report a single representative run rather than MC summary statistics. The full SMM target is $\beta = (\alpha, \psi_1, \eta_0, \eta_1, c_{\text{def}}, \rho, \sigma)$ with $K = 7$, matched against $R = 11$ moments following Hennessy and Whited (2007)'s selection (see Appendix B). The results can be reproduced from `docs/06_risky_debt_smm_workflow.ipynb`. It took about 40 hours to run the full SMM on my 2020 Macbook Pro (M1).

Result 1: moment fit. Fitted moments deviate noticeably from their targets. The conditional-issuance, AR(1)-shock-std, and variance-of-investment moments miss by 50%+ of their target value, indicating the optimizer cannot find a β that matches all 11 moments simultaneously.

Moment	Target	Fitted
Avg Iss/k	0.0168	0.0235
Cond Iss	0.0881	0.1917
AC Iss	-0.0507	-0.0518
Corr(Lev, Iss)	0.2723	0.3008
Avg Lev	0.9099	0.8635
Std Lev	0.0308	0.0485
AC I/k	0.0023	0.0023
Var I/k	0.0287	0.0395
AR(1) β	0.3399	0.3639
AR(1) σ	0.2158	0.3139
Default freq	0.000488	0.000400

Table 2.7: Moment fit at $\hat{\beta}$ on a single representative panel (11 moments, 7 estimated parameters).

Result 2: parameter estimates. Five of seven t-tests reject $H_0 : \hat{\beta}_k = \beta_k^*$ at the 5% level. The two that fail to reject (c_{def}, ρ) do so only because their standard errors are abnormally large.

Parameter	True	Estimate	SE	t -stat	p -value
α	0.70	0.6213	0.0174	-4.53	$< 10^{-5}$
ψ_1	0.05	0.0691	0.0071	2.69	0.007
η_0	0.10	0.1207	0.0418	0.50	0.62
η_1	0.05	0.0851	0.0049	7.13	$< 10^{-12}$
c_{def}	0.45	0.3844	0.3896	-0.17	0.87
ρ	0.60	0.6411	0.0624	0.66	0.51
σ	0.15	0.2206	0.0306	2.31	0.02

Table 2.8: Parameter estimates at $\hat{\beta}$, sandwich SEs, and t-tests against the truth. The J-statistic is NaN because $\hat{\Omega}$ is numerically singular.

Two failure modes are visible. First, the optimal weight matrix is not available: the J-statistic is NaN because $\hat{\Omega} = \frac{1}{S} \sum_s E_s E_s^\top$ is numerically singular at the Stage-1 estimate. With 11 moments and a moderate S , several rows of the per-panel error matrix are nearly collinear, so Stage 2 cannot use $W = \hat{\Omega}^{-1}$ as the optimal weighting. The sandwich SE for c_{def} (0.39) is essentially the width of its prior range, the typical fingerprint of a singular weight matrix collapsing onto a single parameter. Second, the point estimates of α , ψ_1 , η_1 , and σ are biased away from the truth by 4 to 7 standard errors. Combined with the moment-fit gaps, this is consistent with weak identification: the moment vector cannot distinguish β^* from nearby parameter values that happen to fit a subset of moments better.

Diagnostic step. To localize the failure I run two checks on the same fixed simulation seed:

- **Oracle test.** Compute $Q(\beta^*)$ at the true parameters: $Q(\beta^*) \approx 1.4 \times 10^{-3}$, well below the optimization tolerance. The moment-construction pipeline is correctly wired and the truth is reachable in principle; the optimizer simply does not converge to it.
- **Jacobian audit.** Finite-difference the 11×7 Jacobian $\partial g / \partial \beta$ at β^* (2K extra solves). Two findings: (i) the moments *conditional issuance size*, *autocorrelation of equity issuance*, and *cross-correlation leverage-issuance ratio* load almost entirely on α and ψ_1 rather than the parameters they were meant to identify. They are redundant with the cleaner Hennessy and Whited (2007) moments already in the active set; (ii) the column for c_{def} has a small norm relative to the other parameter columns, confirming weak identification.

2.2.1 Attempted fixes

I tried two remedies in addition to dropping the three redundant moments identified by the Jacobian audit. Neither restored the estimator.

1. Calibrate c_{def} at the prior value. Motivated by the Jacobian audit’s weak-identification finding, I fix $c_{\text{def}} = 0.45$ and remove it from the estimated parameter vector, dropping K from 7 to 6. With 8 remaining moments and 6 parameters, the Jacobian achieves full rank (6/6). The single-panel estimates of the remaining six parameters are still biased: α , ψ_1 , ρ , and σ all reject at the 5% level, η_0 and η_1 have standard errors above 10^5 (a numerical artifact of $\hat{\Omega}^{-1}$ inverting near-zero singular values), and the J-statistic becomes computable but rejects strongly ($J = 44.9$, $p < 10^{-9}$, $\text{df} = 2$). Calibrating c_{def} removes the most visible weak-identification artifact but does not restore the estimator.

2. Replace summary moments with empirical-policy-function (auxiliary-model) moments Following Nikolov, Schmid, and Steri (2021), I run an auxiliary regression on both real and simulated panels and match the *coefficients* of that regression. Concretely, I regress each observed outcome (I/k , b'/k , $\text{max-equity-issuance} / k$) on a small set of lagged states and controls ($\log k_{t-1}$, $\log z_{t-1}$, b_{t-1}/k_{t-1}), and treat the coefficient vector as the moment vector. This is the indirect-inference approach: the auxiliary model serves as a richer summary of the policy than scalar moments. The richer moment vector improves the conditioning of $\hat{\Omega}$ and the J-statistic is computable, and the pathological standard errors on η_0 and η_1 disappear. Yet the point estimates of α , ψ_1 , η_0 , η_1 remain biased compared with the truth, and the J-test still rejects.

Summary. Neither attempt solved the weak identification problem when applying SMM to the risky debt model. Since we have validated that the SMM pipeline itself is correct, the issue is the model structure itself and would require a better selection of moment conditions beyond the ones used in Hennessy and Whited (2007). Unfortunately, I currently do not have a clean solution to it.

2.3 Defects of the risky debt model and potential solutions

I focus on the defects of the model’s core economic mechanisms and discuss their theoretical and practical (empirical) implications. To be fair and constructive, I do not discuss critique that are either too board or general, or those that require adding new features beyond the original focus of the model.

To clarify, the core mechanism of the risky debt model is that firm’s financing decisions reflect (i) optimal investment under frictional adjustment cost; and (ii) the opportunity to exploit the tax shield benefit of debt (in the form of a one-period corporate bond). Lenders (bank) with rational expectation charge a risk premium on the yields of the corporate bond based on anticipated default probability. Default threshold is determined by the realization of next-period productivity shock conditional on current states and actions, for example, higher debt requires higher realization of future productivity to repay, and thus expand the

default set (probability). This is anticipated by the lender and priced into the endogenously-determined risk premium.

Given this core mechanism, I find two critical defects in the specific risky debt model presented in Strebulaev and Whited (2012, sec. 3.6):

1. Timing of tax shield benefit create “always-default” strategy
2. Assumption of perfect managerial information

Both are structural defects that can only be “solved” by major revisions to the model and estimation strategy.

2.3.1 Timing of tax shield benefit

Defect #1 is problematic because the present value of tax shield benefit, $\frac{\tau \tilde{r} b'}{(1+\tilde{r})(1+r)}$ is obtained by firm upfront and is unconditional on the next-period solvent/default states (Equation 3.26). Since the model intentionally does not impose a borrowing limit, firm could exploit an optimal “always-default” strategy that borrow as much as possible and default in next period. The lender rationally rationalize this in pricing the interest rate $\tilde{r} \rightarrow \infty$, but the tax shield benefit is still large and positive:

$$\frac{\tau \tilde{r} b'}{(1+\tilde{r})(1+r)} \rightarrow \frac{\tau b'}{(1+r)} > 0 \quad \text{as } \tilde{r} \rightarrow \infty \text{ and } b' \rightarrow \infty$$

This is confirmed empirically when any naive implementation with large upper bound $b_{\max}$ relative to the $k_{\max}$ will cause the optimal policy to converge to “borrow as much as possible then default” with $b' = b_{\max}$ and $k' \approx 0$. For model solve itself this can be mitigated with well-calibrated parameters and bounds, but the true risk is for SMM estimation when the optimizer re-solved the policy under different parameters and a non-trivial fraction of the parameter combinations will lead to this unintended strategy.

One simple solution is to use the same time schedule as in the trade-off model in Nikolov, Schmid, and Steri (2021).

2.3.2 Imperfect managerial information

Defect 2 is directly related to the critique by DeAngelo (2022): Can manager and lender precisely estimate the continuation value V and default states given by $\{z : V(\cdot, z) \leq 0\}$? This is the key important assumption of the model: the endogenous default decision is a going-concern and manager would only default when the firm’s continuation value is negative. The pricing of bonds is from a bargaining between the firm and the lender based on $E_{z'|z}[V(k', b', z')]$ where there exist a critical value of shock z'_d such that all $z' < z'_d$ are default states (Hennessy and Whited 2007, Proposition 6). However, if manager and lender cannot learn V with any realistic precision, this core mechanism is broken.

DeAngelo (2022, sec. VI) reviews direct evidence of imperfect manager knowledge. There are two key takeaways. First, large-scale surveys of CFOs indicated that “most managers have nothing close to the knowledge assumed in extant dynamic capital structure models, which posit a complete understanding of investment opportunities and capital-market conditions over an infinite horizon” (Graham 2022). Second, a number of studies have estimated a near-flat relationship between firm value and leverage, suggesting that “real-world managers are unable to pin down a uniquely optimal capital structure with any real precision” (Korteweg 2010).

Chapter 3

Bayesian Estimation of the Strebulaev (2012) Models

3.1 Short answers to the bonus question #1

(a) What are your prior assumptions on the parameters? A prior encodes belief about a parameter before seeing data. It is specified as a probability density function of the parameter, $p(\beta)$. Practical specification depends on domain-knowledge. I discuss the specific priors I choose for different parameters in later section.

(b) What filtering method do you use, and what are its pros and cons vs alternatives? By default, I use Kalman Filter when the model can be cast as a Linear Gaussian state-space model (LGSSM). When the likelihood is differentiable but non-linear, I consider Extended Kalman Filter or Unscented Kalman Filter as alternatives. When the likelihood is non-differentiable, I consider the Particle Filter and Random Walk Metropolis–Hastings (RW-MH) as fallback. I discuss specific filtering choice later when applying to different models.

(c) Which MCMC method (in TFP), and how is it chosen? I built and tested two main options: (1) No-U-Turns (NUTS) Hamiltonian Monte Carlo with Extended Kalman Filter; (2) Random-Walk Metropolis–Hastings (RW-MH) with Extended Kalman Filter. NUTS are more efficient when gradient info $\nabla L(\beta)$ is available, RW-MH is used when gradient is not available or too computationally costly.

(d) What tests assess the validity of the estimation method? For generic diagnostic metrics, I report split- $\hat{R}$ and effective sample size (ESS), as well as several method-specific metrics such as diverged transitions and leapfrog tree depth (for NUTS-HMC). For posterior analysis, I report visualization of the posterior marginals, summary statistics (median and mean), and trace plot. For more complete tests, I implement a minimal version of coverage check in the spirit of [Simulation-Based Calibration](#), as well as posterior predictive checks (simulate data to match real data) and prior sensitivity analysis (robustness of result when varying prior).

(e) How does Bayesian estimation compare to GMM and SMM? GMM matches model-implied moments (analytical functions of θ) to sample moments. It requires the optimal policy function itself to be expressible analytically in θ . In this project only the basic model without adjustment cost satisfies that. SMM replaces the analytical moments with simulated ones: one model solve per parameter evaluation, so any model that can be simulated is in scope. SMM gives a point estimate with asymptotic-sandwich

standard errors, no full posterior; the analyst must pick the moments (a modeling choice that affects identification); and it is prone to weak identification when the chosen moments don't pin down θ .

Bayesian estimation uses the full likelihood (no moment selection), delivers the joint posterior distribution, and quantifies parameter uncertainty. It often worked better in identification of high-dimensional models compared with SMM. For models more complicated than the basic model, I introduce a Neural Surrogate approach to cut per-evaluation model solve cost for both Bayesian MCMC and SMM. This is done by pre-training a neural network (NN) to approximate the optimal policy function in state-by-parameter space. Thus, the choice between Bayesian and SMM reduces to a methodological one: full posterior characterization (Bayesian) versus a frequentist point estimate (SMM).

3.2 Overview

3.2.1 Target

Treat the parameter vector β as a random variable. Given observed data y , the target is the posterior distribution

$$p(\beta | y) \propto \underbrace{p(\beta)}_{\text{prior}} \cdot \underbrace{p(y | \beta)}_{\text{likelihood}}.$$

Markov Chain Monte Carlo (MCMC) operates on the **log-target**

$$L(\beta) := \log p(\beta) + \log p(y | \beta)$$

Since $\log p(y)$ does not depend on β , it cancels in every Metropolis-Hastings (MH) acceptance ratio and never needs to be computed. The pipeline reduces to two tasks: specify a prior, and evaluate the log-likelihood $\log p(y | \beta)$ at any candidate β .

3.2.2 Model specification

The Bayesian pipeline has one upstream modeling choice that shapes everything downstream: Do we assume that observed actions (e.g., investment, leverage) are generated by the firms acting by optimal policy? If so, what structural form shall we impose on the residual gaps?

In principle, there are many ways to cast the structural model into a set of empirical specifications for Bayesian estimation. This specification choice largely depend on (1) the complexity of the model itself (e.g., risky debt); (2) the target parameters to be estimated (e.g., firm's deviation from optimal leverage); and (3) the additional assumptions we are willing to impose on top of the original theoretical model (e.g., Gaussian noise or systematic error).

To illustrate this concretely, I use the simple basic model of optimal investment as example. Consider that we observe a panel data of firms. The basic optimal investment model has a solution defined as an optimal policy function φ mapping states to actions.

Observed data. We observe a firm-year panel of capital, leverage, and revenue:

$$\text{Panel data: } (k_{it}, b_{it}, \Pi_{it}).$$

Structural model. A vector $\beta \equiv (\alpha, \rho, \sigma_\epsilon)$ of structural parameters underlying the data-generating process (DGP):

$$\begin{aligned} \text{Cobb-Douglas production:} \quad & \log \Pi_{it} = \log z_{it} + \alpha \cdot \log k_{it} + \eta_{it} \\ \text{Latent AR(1) productivity:} \quad & \log z_{it} = \rho \log z_{i,t-1} + \sigma_\epsilon \cdot \mathcal{N}(0, 1) \\ \text{Investment policy:} \quad & \log k_{it} = \varphi_k(z_{it}, k_{it}, b_{it} \mid \beta) + \xi_{it}^k \\ \text{Leverage policy:} \quad & \log b_{it} = \varphi_b(z_{it}, k_{it}, b_{it} \mid \beta) + \xi_{it}^b \end{aligned}$$

where z is the latent productivity shock, φ_k and φ_b are the optimal investment and leverage policies implied by the model at β , and the unconditional mean of $\log z$ is normalized to zero so the production residual carries all level information.

Residual specification. The three residuals absorb the gap between model implications and observed data:

$$\eta_{it} \sim \mathcal{N}(\mu_\eta, \sigma_\eta^2), \quad \xi_{it}^k \sim \mathcal{N}(\mu_{\xi^k}, \sigma_{\xi^k}^2), \quad \xi_{it}^b \sim \mathcal{N}(\mu_{\xi^b}, \sigma_{\xi^b}^2).$$

The Gaussian shape is a likelihood assumption that can be relaxed (Student-t, mixture). I take Gaussian as the baseline. The means and variances are additional parameters to be estimated.

Economic interpretation. Each residual has important economic meanings:

- η is production misspecification or measurement error in revenue. A nonzero μ_η indicates the production function omits a systematic factor (e.g., labor input). σ_η^2 measures production-equation fit quality.
- ξ^k and ξ^b are *policy deviations*: the systematic gap between observed investment / leverage and the model-implied optimal choices. A nonzero μ_{ξ^k} indicates firms invest more or less on average than the model-implied optimal decisions; $\sigma_{\xi^k}^2$ measures dispersion of that deviation. Analogously for ξ^b . This is the cross-sectional analog of the investment and labor “wedges” in DSGE business-cycle accounting (Chari, Kehoe, McGrattan 2007).

Identification. Generally, we need to constraints for identification:

1. The unconditional mean of $\log z$ is normalized to zero. Without this, $E[\log z]$ and μ_η are observationally equivalent in the production equation. The normalization is harmless: any nonzero unconditional mean of latent productivity is absorbed by μ_η .
2. The policy intercept of $\varphi_k(\cdot \mid \beta)$ is a deterministic function of β . A free additive μ_{ξ^k} is partially collinear with shifts in β that move this intercept. Depending on the specific models, identification is enabled by (i) informative priors on μ_{ξ^k} centered at zero, and (ii) β enters η , ξ^k , and ξ^b jointly, so the posterior anchors β from production and AR(1) variations, leaving the residual policy gap to μ_{ξ^k} .

These choices capture the argument for identifying firm’s deviation from optimal actions: μ_{ξ^k} captures “policy deviation that cannot be explained by a different β within the model class.”

Baseline choice of priors

I consider the following baseline choice of priors:

Parameters	Prior	Support	Rationale
α	Beta(2, 2)	(0, 1)	Moderate return to scale
ρ	Beta(2, 2)	(0, 1)	Moderate persistence

Parameters	Prior	Support	Rationale
σ_ε	HalfNormal(0.3)	$(0, \infty)$	More mass on low variance
σ_η	HalfNormal(0.1)	$(0, \infty)$	More mass on less measurement error
$\mu_\eta, \mu_{\xi^k}, \mu_{\xi^b}$	Normal(0, 0.1)	$(-\infty, \infty)$	Center around zero
$\sigma_\eta, \sigma_{\xi^k}, \sigma_{\xi^b}$	HalfNormal(0.5)	$(0, \infty)$	More mass on low variance

Special case: observation-only inference. When the only goal is to identify parameters that appear in the production equation $(\alpha, \rho, \sigma_\varepsilon, \mu_\eta, \sigma_\eta)$, the policy residuals can be dropped from the likelihood entirely. Observed (k, b) are then treated as exogenous covariates and no policy solve is needed. This case applies to the frictionless basic model (§3.1), where the policy adds no extra identifying information beyond what the production equation provides. For every other model in the project (frictional basic, risky debt, extensions), cost and default parameters enter only through the policy, so the policy residuals must be kept in the likelihood. This is the central motivation for the neural-surrogate work in §3: solving the model at every MCMC step requires the policy to be amortized in β so each evaluation is one forward pass.

3.2.3 Generic Algorithm

The pipeline is generic across choices of filter and MCMC sampler. The Evaluate step branches by approach.

Setup.

- Specify priors $p(\cdot)$
- Specify a filtering algorithm that, given fixed data y and a candidate β , returns the scalar $\log p(y | \beta)$.
- Fix the number of MCMC iterations J and the number of chains.

Pre-training NN surrogate policy

This step may be skipped if the optimal policy can be written in closed-form formula, or if inference does not need optimal policy to form the observation equations.

- Takes firm panel dataset as input, extract state space bounds (e.g., min and max capital observed)
- Use the extracted state bounds from panel and parameter bounds from prior to simulate a panel of training dataset
- Train NN on the simulated data and obtain the parametrized policy surrogate, $\varphi_\theta(s, \beta)$, which is stored as NN weights and biases. Discard the training and validation set.
 - Model solve method: Short-horizon Actor Critic (default) and Euler residual minimization (special case)
- Pass the stored NN surrogate to the likelihood estimation in next stage

Bayesian Inference: MCMC + Filtering Algorithm

For each MC chain, per iteration $j = 1, \dots, J$:

1. **Propose** a candidate β' by perturbing the current β . I built three options:
 - *Random-Walk Metropolis-Hastings (RW-MH)*: draw $\beta' = \beta + \text{Gaussian noise}$. Gradient-free default.

- *Robust Adaptive Metropolis (RAM)*: Adaptive version of RW-MH that avoids manual tuning of the search step size.
 - *No-U-Turn Sampler with Hamiltonian Monte Carlo (NUTS-HMC)*: use $\nabla_{\beta} L(\beta)$ to simulate Hamiltonian dynamics on L , then slice-sample a candidate from the trajectory. Gradient-based; requires L to be differentiable in β .
2. **Evaluate** $L(\beta') = \log p(\beta') + \log p(y | \beta')$. The likelihood step depends on the approach:
 - *Option 1: Closed-form Policy*. Form the observation equation using closed-form formula of optimal policy. Run the filter.
 - *Option 2: Neural Surrogate Policy*. Take the pre-trained NN surrogate policy $\varphi_{\theta}(s, \beta)$, plug into observation and transition equations, run the filter.
 3. **Accept or reject**. MH test accepts β' with probability $\min(1, \exp(L(\beta') - L(\beta)))$ under symmetric proposal. NUTS-HMC uses the joint Hamiltonian ratio.
 4. **Record** the current chain position as sample $\beta^{(j)}$.

Output. A pooled set of samples $\{\beta^{(j)}\}$ across chains. Their empirical distribution approximates $p(\beta | y)$. Posterior means, quantiles, and credible intervals are computed from this set.

Two principles guide method choice:

- Filter and sampler are independent components and can be swapped separately.
- The MH accept/reject step guarantees correctness. The proposal rule controls only efficiency.

3.2.4 Validation and Diagnostics

Below are the metrics and tests I use to validate the inference implementation.

Per-run MCMC convergence. The following metrics are reported to test whether the sampler converged on this dataset?

- *Split R-hat* (Vehtari et al. 2021). Between- vs within-chain variance ratio after splitting each chain in half. Target $\hat{R} < 1.01$ at production budget; loosened to $\hat{R} < 1.05$ at smoke budget.
- *Effective Sample Size (ESS)*. Autocorrelation-corrected count of independent samples,

$$\text{ESS} = \frac{MS}{1 + 2 \sum_{k \geq 1} \rho_k}$$

for M chains and S post-warmup samples each, with ρ_k the lag- k autocorrelation truncated at the first negative estimate. Target $\text{ESS} > 400$ (about 5% Monte Carlo error on credible-interval quantiles).

- *NUTS-specific signals*. Zero divergences and no max-tree-depth saturation across post-warmup iterations. Both indicate the sampler is exploring the target geometry without numerical breakdown or premature trajectory termination.
- *Trace plots*. Overlapping chains, no drift, no stuck plateaus. Necessary but not sufficient; complements the numerical diagnostics.

TFP modules: `tfp.mcmc.potential_scale_reduction`, `tfp.mcmc.effective_sample_size`. Divergence counts and per-iteration tree depth are returned in the trace metadata of `tfp.experimental.mcmc.windowed_adaptive_nuts`

Calibration of the inference machinery. Do the credible intervals attain their nominal coverage on data drawn from the model? This validates the implementation against the data-generating process. - *Coverage check*. Draw R parameter vectors β_0 from the prior; for each, simulate a panel under the model

at β_0 , run the full inference pipeline, and record per-parameter 95% credible intervals. Pass when the empirical hit rate per parameter falls inside the binomial 95% interval around true coverage 0.95 at the chosen R . - Future extension: *Simulation-Based Calibration* (Talts et al. 2018). Rank-based formal test at $R \geq 100$ with rank histograms; the rigorous version of the coverage check. Not implemented in current version

Model-data fit. The last set of tests aim to examine how the fitted model describe the real data?

- *Posterior predictive checks (PPC).* Compare summary statistics of replicated panels $Y^{\text{rep}} \sim p(\cdot | \beta^{(s)})$ to the observed Y . Systematic mismatch in a summary that the model should reproduce flags misspecification.
- *Prior sensitivity analysis.* Vary the prior within a defensible range and confirm that posterior summaries move by less than their credible-interval widths. Bounded movement indicates the posterior is data-driven rather than prior-driven on the parameters of interest.

3.3 Bayesian Inference with Neural Surrogate

The goal is to verify the Bayesian inference pipeline end-to-end on the basic model with no adjustment costs. This is a special case of the toy model because the optimal policy has a closed-form formula, which allows me to validate and test the pipeline with ground-truth.

The main innovation of my implementation is the use of a pre-trained “Neural Surrogate” to approximate the optimal policy function in inference. The benefit is substantial: - Typical MCMC + Filtering requires at least **thousands of model solve** (via VFI/PFI/NN-based methods) when evaluating different parameter candidates - My approach only needs **one model solve**: pre-train a NN surrogate over the entire States $\times$ Parameter space, then pass it to the inference pipeline for each evaluation.

I illustrate my pre-training + inference pipeline below using the toy basic investment model.

3.3.1 Environment: Basic Model

Following Strebulaev and Whited (2012, sec. 3.1), each firm $i = 1, \dots, N$ solves an infinite-horizon investment problem with no adjustment costs and no debt.

The log-productivity follows a first-order autoregressive process (AR1):

$$\log z_{i,t+1} = \rho \cdot \log z_{i,t} + \sigma_\varepsilon \cdot \varepsilon_{i,t+1}$$

where $\varepsilon_{i,t+1} \sim \mathcal{N}(0, 1)$, $\rho \in (0, 1)$, $\sigma_\varepsilon > 0$. Shocks $\varepsilon_{i,t+1}$ are iid across firms and time.

Firm’s observed revenue and capital stock is assumed to be generated by:

$$\begin{aligned} \log \Pi_{it} &= \log z_{it} + \alpha \cdot \log k_{it} + \eta_{it} \\ \log k_{it} &= \varphi_k(z_{it}, k_{it} | \beta) + \xi_{it}^k \end{aligned}$$

where as before I assume that model specification error are Gaussian:

$$\eta_{it} \sim N(\mu_\eta, \sigma_\eta), \quad \xi_{it}^k \sim N(\mu_\xi, \sigma_\xi)$$

Using capital accumulation identity, $k_{i,t+1} = (1 - \delta) k_{i,t} + I_{i,t}$, the control (action) variable can be re-written from investment I to k_{t+1} , that is, firm directly choose the optimal capital level of next period.

Without adjustment costs, firm's optimal policy function has closed-form solution:

$$\begin{aligned}\varphi_k(z_{it}, k_{it} \mid \beta) &= \frac{\rho}{1-\alpha} \log z_{i,t} + \frac{1}{1-\alpha} \left[\log \alpha + \frac{\sigma_\varepsilon^2}{2} - \log(r + \delta) \right] \\ &= \frac{\rho}{1-\alpha} \log z_{i,t} + \kappa(\alpha, \sigma_\varepsilon, r, \delta)\end{aligned}$$

where we denote the second intercept term as $\kappa(\cdot)$ for simplicity. This formula can be derived easily using the Euler equation $E_t[\alpha z_{i,t+1} k_{i,t+1}^{\alpha-1}] = r + \delta$ and $E_t[z_{i,t+1}] = \exp(\rho \log z_{i,t} + \sigma_\varepsilon^2/2)$ for log-normal variable z .

This means capital is log-linear in $\log z_{i,t}$, enabling the linear Gaussian state-space form.

Target parameters to be estimated include economic parameters $\beta \equiv (\alpha, \rho, \sigma_\varepsilon, \sigma_\eta)$ and the mean $(\mu_\eta, \mu_{\xi k})$ and variances $(\sigma_\eta, \sigma_{\xi k})$ for the model-specification errors.

Calibrated parameters: $r = 0.04$ (risk-free rate) and $\delta = 0.10$ (depreciation rate).

3.3.2 Pre-training the Neural Surrogate

The pre-training part uses two NN-based methods to solve for firm's optimal policy: - Default: Short-Horizon Actor Critic (SHAC) - Optional: Euler Residual Minimization (ER)

These methods have been introduced and tested in part 1. The key point is instead of mapping from state space to action space, the NN surrogate is trained on the higher-dimensional state $\times$ parameter space.

Concretely, let $\beta \equiv (\alpha, \rho, \sigma_\varepsilon)$ denote the structural economic parameters of the basic model. The pre-trained NN surrogate here is $\varphi(z, k; \beta)$ mapping to the optimal next-period capital k' . This is different from pure model solve (Part 1) where parameters are fixed and the NN policy is only 2-dim over (z, k) .

This approach is only feasible with NN-based policy approximator. Traditional numerical methods like VFI, PFI, or Linear Programming are grid-based and cannot be solved once over a high-dimensional parameter space. It would require repeatedly solving the model under different parameters, which is intractable even for toy model of this scale (with 8 parameters to be estimated).

To validate the quality of the pre-trained NN surrogate policy, I use the same set of metrics computed on held-out validation dataset. I also plot the NN solution against true analytical solution over k slices. Figure 3.2 shows that SHAC learned a highly precise NN surrogate (orange) with mean absolute error (MAE) lower than 1% when compared against the true closed-form formula (dash). The pre-trained NN surrogate maps $(z, k, \alpha, \rho, \sigma_\varepsilon)$ to the optimal next-period capital k' . SHAC is also efficient as this training took about 30min on a CPU (Apple M1). In future, this can be scale up with GPU and more training budget to learn more complex models and to achieve lower mean absolute error. The results can be reproduced in `docs/08a_pretrain_nn_surrogate.ipynb`.

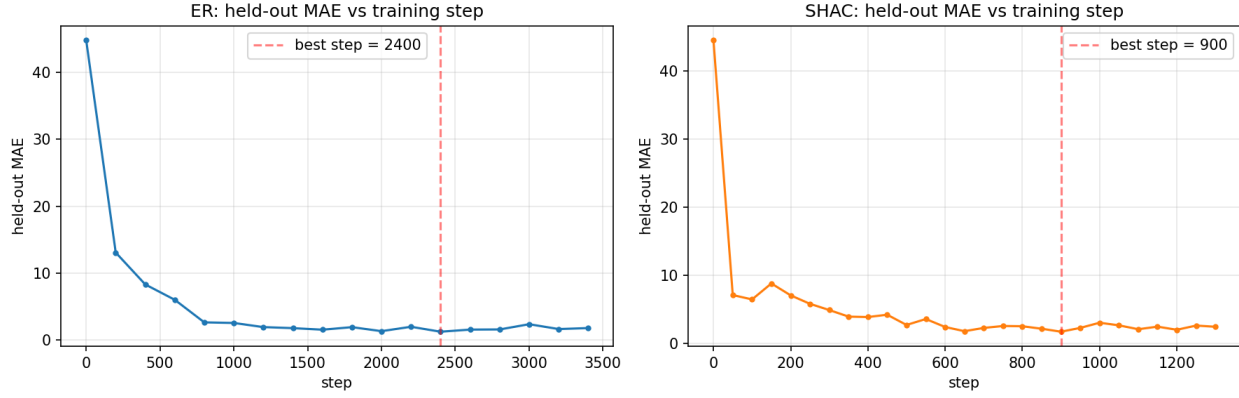


Figure 3.1: Training curves: held-out MAE vs SHAC/ER step. Red dashed line marks the best-checkpoint restore; the gap to final-step weights illustrates why best-step restoration matters.

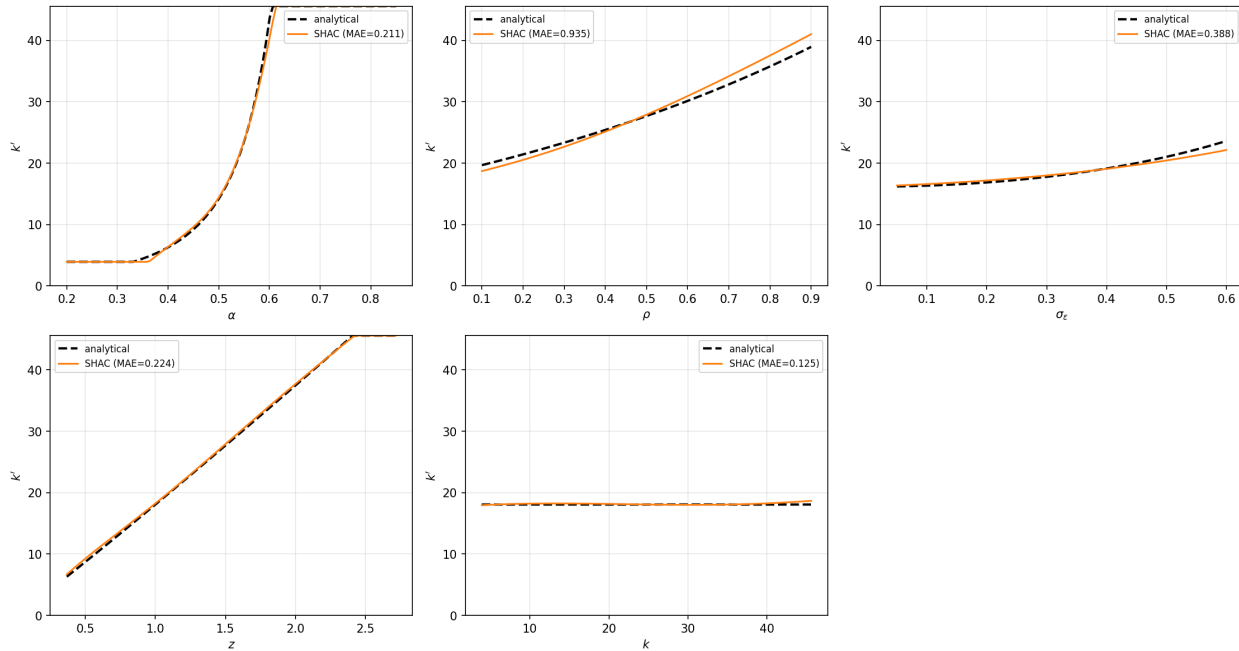


Figure 3.2: Pre-trained Neural Surrogate.

3.3.3 Likelihood: Extended Kalman Filter

I use Extended Kalman Filter (EKF) to compute likelihood. Independence across firms gives

$$\log p(Y | \beta) = \sum_{i=1}^N \sum_{t=1}^T \log p(y_{i,t} | y_{i,1:t-1}, \beta),$$

where $y_{i,t} = (\log \Pi_{i,t}, \log k_{i,t+1})^T$ stacks the two observations at firm-year (i, t) and β collects every estimable parameter from §2.3.

The latent state is scalar, $x_{i,t} \equiv \log z_{i,t}$. Let $m_{t|s}, V_{t|s}$ denote the conditional mean and variance of $x_{i,t}$ given $y_{i,1:s}$ (firm index suppressed).

Two observation equations. From §2.3,

$$\begin{aligned} y_{i,t}^{(1)} &= x_{i,t} + \alpha \log k_{i,t} + \mu_\eta + \eta_{i,t}, & \eta &\sim \mathcal{N}(0, \sigma_\eta^2), \\ y_{i,t}^{(2)} &= g(x_{i,t}, k_{i,t}; \beta) + \mu_{\xi^k} + \xi_{i,t}^k, & \xi^k &\sim \mathcal{N}(0, \sigma_{\xi^k}^2), \end{aligned}$$

where $g(x, k; \beta) := \log \phi_k(\exp x, k; \beta)$ is the log-policy prediction for $\log k_{t+1}$. Eq 1 is linear in x ; Eq 2 is **potentially nonlinear** in x through g .

When the EKF is appropriate. The EKF replaces $g(x; \beta)$ with its first-order Taylor expansion around the predicted latent mean,

$$g(x; \beta) \approx g(m_{t|t-1}; \beta) + H_2(t) \cdot (x - m_{t|t-1}), \quad H_2(t) := \left. \frac{\partial g}{\partial x} \right|_{x=m_{t|t-1}}.$$

The Taylor remainder is of order $\frac{1}{2}|g''(m_{t|t-1})| \cdot V_{t|t-1}$, so the linearization is accurate when *either* the policy is close to linear in x (small $|g''|$) *or* the predictive uncertainty is small (small $V_{t|t-1}$, i.e. the data anchors the latent state tightly). Two regimes apply here.

- **Closed-form policy.** $g(x; \beta) = \rho(1-\alpha)^{-1}x + \kappa(\alpha, \sigma_\epsilon)$ is **globally linear in x** , so $H_2 = \rho/(1-\alpha)$ is a constant, the Taylor expansion is exact, and EKF reduces to standard Kalman with zero linearization error. I use this as “ground-truth” for validation.
- **Neural surrogate policy.** g is the cached neural network ϕ_θ that is pre-trained and used to approximate the optimal policy function. Linearity holds only approximately; the surrogate’s slope against $\log z$ is checked offline via `diagnose_nn_linear_slope` and the residual variance $\sigma_{\xi^k}^2$ absorbs whatever gap survives.

Linearized state-space form. Stack the two equations at the linearization point:

$$y_{i,t} = H_t x_{i,t} + d_{i,t} + \epsilon_{i,t}, \quad \epsilon_{i,t} \sim \mathcal{N}(0, R),$$

with

$$H_t = \begin{pmatrix} 1 \\ H_2(t) \end{pmatrix}, \quad d_{i,t} = \begin{pmatrix} \alpha \log k_{i,t} + \mu_\eta \\ g(m_{t|t-1}, k_{i,t}; \beta) - H_2(t) m_{t|t-1} + \mu_{\xi^k} \end{pmatrix}, \quad R = \begin{pmatrix} \sigma_\eta^2 & 0 \\ 0 & \sigma_{\xi^k}^2 \end{pmatrix},$$

and AR(1) state transition $x_{i,t+1} = \rho x_{i,t} + \sigma_\epsilon v_{i,t+1}$, $v \sim \mathcal{N}(0, 1)$.

Initialize $m_{1|0} = 0$, $V_{1|0} = V_0 = 10$.

For $t = 1, \dots, T$:

1. **Predict the state** (AR(1) transition; at $t = 1$ use initialization):

$$m_{t|t-1} = \rho m_{t-1|t-1}, \quad V_{t|t-1} = \rho^2 V_{t-1|t-1} + \sigma_\epsilon^2.$$

2. **Evaluate the policy and its Jacobian** at $x = m_{t|t-1}$, $k = k_{i,t}$:

$$g_t := g(m_{t|t-1}, k_{i,t}; \beta), \quad H_2(t) := \left. \partial g / \partial x \right|_{x=m_{t|t-1}}.$$

Closed-form: $H_2(t) = \rho/(1-\alpha)$ analytically. NN surrogate: $H_2(t)$ via autodiff through the network.

3. **Innovation** (observed minus predicted):

$$v_t = \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} y_{i,t}^{(1)} - (m_{t|t-1} + \alpha \log k_{i,t} + \mu_\eta) \\ y_{i,t}^{(2)} - (g_t + \mu_{\xi^k}) \end{pmatrix}.$$

4. **Innovation covariance** $S_t = H_t V_{t|t-1} H_t^T + R$ (closed-form 2×2):

$$S_t = \begin{pmatrix} V_{t|t-1} + \sigma_\eta^2 & H_2(t) V_{t|t-1} \\ H_2(t) V_{t|t-1} & H_2(t)^2 V_{t|t-1} + \sigma_{\xi^k}^2 \end{pmatrix}.$$

5. **Likelihood contribution** (bivariate Gaussian):

$$\log p(y_{i,t} | y_{i,1:t-1}, \beta) = -\frac{1}{2} \left[2 \log(2\pi) + \log |S_t| + v_t^T S_t^{-1} v_t \right].$$

6. **Kalman gain and update.** $K_t = V_{t|t-1} H_t^T S_t^{-1}$ (a 1×2 row); then

$$m_{t|t} = m_{t|t-1} + K_t v_t, \quad V_{t|t} = V_{t|t-1} (1 - K_t H_t).$$

Sum likelihood contributions across t and i to obtain $\log p(Y | \beta)$. Cost per evaluation: $O(N \cdot T)$ filter steps, each requiring one g evaluation and one Jacobian.

Implementation. TFP’s `LinearGaussianStateSpaceModel` does not apply because g is nonlinear in x for the NN surrogate, so the EKF is hand-rolled in `src/v2/estimation/bayesian_basic_investment.py` (`_build_ekf_log_likelihood`). The per-step Jacobian $H_2(t)$ is computed inside `tf.GradientTape` (reverse-mode autodiff; at scalar latent state this matches forward-mode efficiency and is XLA-compatible). The whole filter loop is wrapped in `@tf.function(reduce_retracing=True, jit_compile=True)` so NUTS-HMC obtains $\nabla_\beta \log p(Y | \beta)$ end-to-end through one compiled graph, with no hand-coded backward pass. The same `_build_ekf_log_likelihood` factory serves both the closed-form path (§2, with `policy_callable = env.analytical_policy`) and the NN-surrogate path (§3.4, with `policy_callable = policy_nn`); only the policy callable differs.

3.3.4 MCMC Sampler: NUTS and RW-MH

I implemented two main samplers: NUTS-HMC and RW-MH. I also add an adaptive version of RW-MH known as Robust Adaptive Metropolis (RAM) introduced by Vihola (2012).

MCMC Sampler Algorithm

1. Start at β_0 (drawn from the prior).
2. At each iteration:
 - Propose β' from a proposal distribution $q(\cdot | \beta)$,
 - Accept it with probability $\min(1, \frac{p(\beta'|y)}{p(\beta|y)} \cdot \frac{q(\beta|\beta')}{q(\beta'|\beta)})$; otherwise stay at β .
3. After a warmup phase, the visited β values are samples from the posterior $p(\beta | y)$.

Sampler comparison. I summarize the three samplers by what they do and where each is preferred.

Sampler	What it does	Use case	Reason
NUTS-HMC	Uses the log-target's gradient $\nabla L(\beta)$ to simulate informed trajectories through parameter space. Trajectory length, step size, and per-axis scale are auto-tuned during warmup.	Closed-form policy function	Likelihood is differentiable and the gradient is cheap to compute (e.g., closed-form policy). Highest per-iteration efficiency.
RW-MH	Proposes the next β by adding Gaussian random noise to the current value. Uses no gradient or local geometry. Step size is fixed and tuned manually.	NN policy surrogate	Default fallback when the gradient is unavailable or too expensive (e.g., nonlinear NN surrogate). Simple and gradient-free, but needs many more iterations to mix.

Why gradient-free is the default for the NN-surrogate pipeline. NUTS achieves its efficiency by running many small inner steps per iteration, each requiring one gradient of the log-target. With a closed-form policy, this gradient flows through analytical formulas and is cheap to compute. With the NN surrogate, computing the same gradient requires backpropagation through the entire network, which is much more expensive than a single forward evaluation. In our pipeline this gap is dramatic: NUTS with the NN surrogate runs more than 30 hours of wall time on CPU for the basic model, while RW-MH or RAM with the same NN finishes in minutes. The trade-off is that gradient-free samplers need more iterations to reach the same posterior precision, but each iteration is so much cheaper that they remain practical where NUTS+NN does not.

3.3.5 Implementation Issues

There are several practical issues need to be noted:

- The NN surrogate (SHAC) must be trained over a region that contains the prior's bulk mass. If the prior puts non-negligible probability outside the trained box, MCMC will visit points where the NN extrapolates and the likelihood is junk. At the code level I strictly align the box for parameters between pre-training and inference, and I also let NN training extract the box range of observables (e.g., capital) from the panel data first and align the simulated training data with it.
- In my notebook demo, I slice the full dataset into smaller sub-samples to better control compute cost and wall time (specified by `n_firms` and `horizon`). For production we should obviously use the full panel dataset.
- For this specific model, μ_ξ is weakly identified because it is highly correlated with α . I did not patch this to keep the baseline algorithm simple and minimal. For future implementation, weak identification would need to be handled carefully with re-parameterization tricks or other treatment.

3.3.6 Reproducibility

The pipeline reuses the project’s existing stateless-seed infrastructure (`src/v2/data/rng.py`, `src/v2/utis/seeding.py`). A single master seed pair (`m0`, `m1`) controls every RNG-consuming step. Per-replicate child seeds are derived deterministically via `fold_in_seed(master, *tokens)`, where `tokens` are short namespace strings. Two key properties:

- Rerunning with the same master reproduces every β_0 , every panel, every η , and every chain trajectory bit-for-bit on the same hardware **and the same TFP version**. The NUTS warmup is delegated to `tfp.experimental.mcmc.windowed_adaptive_nuts`, which derives per-window seeds via `tfp.random.sanitize_seed`. The same stateless-seed model is used throughout TFP / JAX. The per-window split tree can change across TFP releases; posteriors remain statistically equivalent, but individual chain trajectories may differ byte-by-byte.
- Token-scoped folding means stages are isolated: re-running just MCMC on a fixed panel only changes `mcmc_seed`’s token, not the panel or β_0 draws.

No global RNG state is used. All randomness enters through explicit seed arguments.

3.4 Validation Results

This section reports results by applying Bayesian estimation on simulated panels and verify if it can recover the ground-truth. There are two layers to be validated:

- Correctness of the inference pipeline itself: MCMC + filtering algorithms
- Whether integrating the NN surrogate (instead of closed-form policy) into the inference pipeline lead to any bias and issues

For the first layer, I already implemented unit tests and integration tests, and the validation here is an additional replication to confirm that we can recover the true parameters (posteriors) of a toy model with closed-form optimal policy $k_p = \exp((\log \alpha + \log E[z] - \log(r+\delta))/(1-\alpha))$.

For the second layer, the only difference is one-line code change that inject the pretrained and cached neural network `policy_nn` to replace the closed-form policy formula inside the likelihood `_build_ekf_log_likelihood`.

More specifically, the validation has three steps:

1. Fix a set of ground-truth parameters $(\alpha, \rho, \sigma_\varepsilon, \sigma_\eta, \mu_\eta, \mu_{\xi k}, \sigma_\eta, \sigma_{\xi k})$, simulate a panel of firms using the observation equations of the model, drop latent variable z , the final data include (k_{it}, Π_{it})
2. Extract the range of observables k_{it} from panel, simulate new training and validation set on the same support, pre-train the NN surrogate policy using SHAC, save the learned NN φ_θ after convergence
3. Pass the panel (k_{it}, Π_{it}) and the policy (using either pre-trained φ_θ or closed-form) to MCMC sampler + filtering algorithm. Verify that the posterior median and credible intervals recover the ground-truth.

3.4.1 NUTS + Kalman Filter with closed-form policy

As the first validation exercise, I use the toy basic investment model to verify the code-level correctness of the inference pipeline for NUTS Sampler + Extended Kalman Filter. The implementation follow these steps:

1. Fix a set of ground-truth parameters $(\alpha, \rho, \sigma_\varepsilon, \sigma_\eta, \mu_\eta, \mu_{\xi k}, \sigma_\eta, \sigma_{\xi k})$, simulate a panel of firms using the observation equations of the model, drop latent variable z , the final data include (k_{it}, Π_{it})

2. Use closed-form optimal policy to form the LGSSM.
3. Run NUTS-HMC with Kalman filtering on the panel data (k_{it}, Π_{it}) , store and verify that the posteriors recover the ground-truth

By default, I use 4 chains, 1000 warmup steps, 500 post-warmup sample, and 0.9 MH test acceptance rate. I slice the panel to a small sample with 50 firms over 20 periods so that the estimation finished in one hour on Apple M1 (2020).

Table 1: Posterior summary at ground-truth β (single run).

Parameter	True	Median	95% CI	$\hat{R}$	ESS
α	0.50	0.495	[0.479, 0.511]	1.005	404
ρ	0.50	0.507	[0.494, 0.520]	1.002	832
σ_ϵ	0.24	0.232	[0.221, 0.244]	1.002	1240
μ_η	0.00	0.014	[-0.032, 0.065]	1.004	480
σ_η	0.05	0.051	[0.032, 0.065]	1.000	705
μ_{ξ^k}	-0.10	-0.048	[-0.197, 0.100]	1.006	408
σ_{ξ^k}	0.05	0.048	[0.025, 0.062]	1.001	624

All 7 parameters recover truth within the 95% credible interval. As rule of thumb, All $\hat{R} < 1.01$ and all ESS exceed 400. The three economic parameters $\alpha, \rho, \sigma_\epsilon$ are tightly identified. Out of the 4 remaining parameters, μ_{ξ^k} is of interest because it captures firm's deviation from the model-implied optimal policy, in this case the true deviation is set of be -10%. The μ_{ξ^k} posterior correctly cover the -20% to 10% range with median close to -5%, but the CI is wide due to weak identification as μ_{ξ^k} is partially collinear with α . For future production, we'll need larger sample, more training budget, and re-parameterization to solve the weak identification issue.

Split- $\hat{R}$ is the Gelman-Rubin ratio of between-chain to within-chain variance. $\hat{R}$ near 1 mean the chains have mixed to a common distribution. ESS is the autocorrelation-corrected count of effectively independent samples. For all posteriors, our ESS > 400 pass the minimal threshold. I interpret that both metrics pass the posterior diagnostic checks.

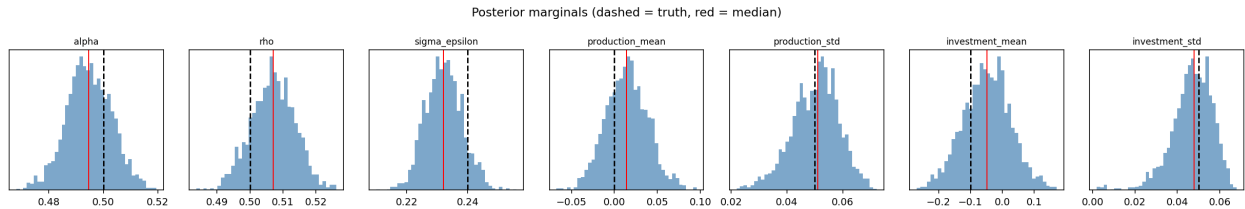


Figure 3.3: Plot Posterior marginals: dashed black = true value, red = posterior median. The x-axis is fixed at the posterior median ± 4 standard deviations, expanded to include the true value if it falls outside, clipped to support.

Figure 3.3 plots the posterior marginals for all parameters, where dashed line is ground-truth and the red line is the estimated posterior median. Figure 3.4 plots the trace of post-warmup draws. The visual evidence is clean and suggest that the code-level implementation of the Bayesian inference pipeline does not have major defects and bugs.

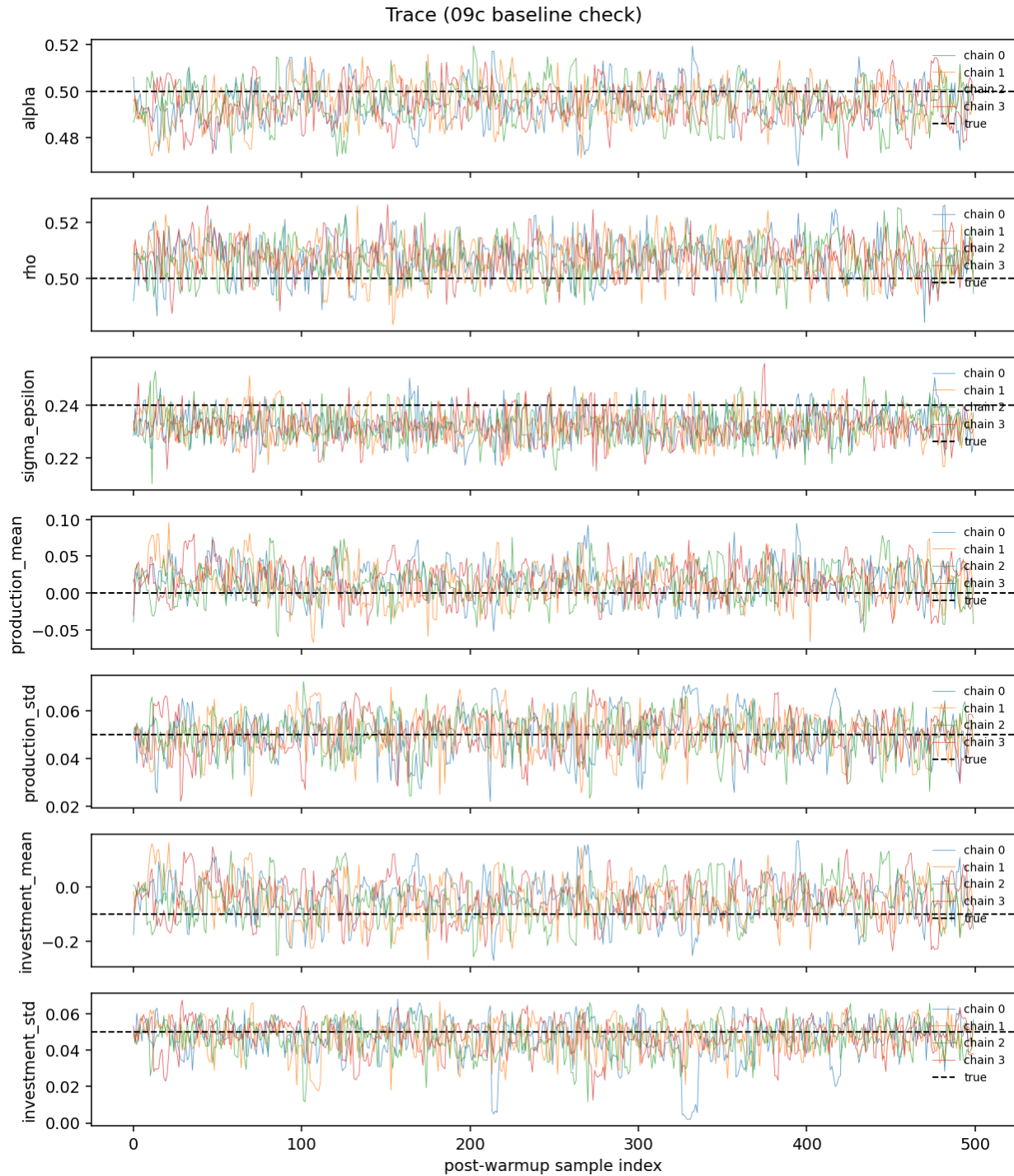


Figure 3.4: Plot Trace: two chains per parameter, overlaid with the truth line. Convergence shows as the two chains exploring the same region with no drift or stuck plateaus.

Coverage and Sensitivity Checks. To show additional checks, I re-run the estimation and the full post-inference analysis with smaller budget per replication: 2 chains, 500 warmup, 200 post-warmup samples, 15 firms and 10 periods (~90 min on Apple M1). This is obviously not sufficient, so the following results

should be interpreted as a *budget-limited demo* under time pressure. Future production will use much larger budget for credible validation results.

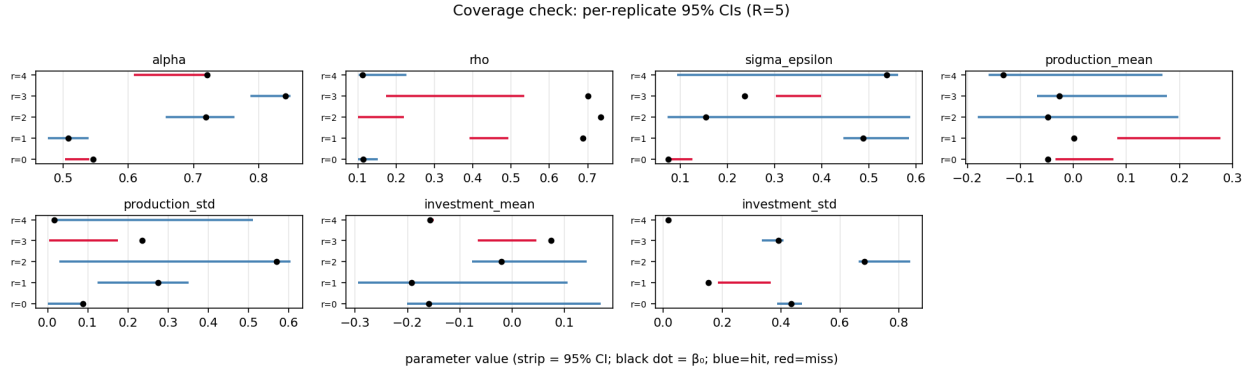


Figure 3.5: Plot coverage of 95% CI over R=5 replications.

Figure 3.5 shows whether each replicate's 95% credible interval contains its ground-truth β_0 . A well-calibrated estimator hits truth in roughly 95% of replicates. At the demo budget of $R = 5$ most parameters hit 3 or 4 times out of 5, just below the binomial pass-band and consistent with both small- R noise and modest per-replicate warmup. The pipeline runs end-to-end here. A production calibration claim requires $R \geq 30$ and longer per-rep adaptation.

Posterior predictive: replicated distribution (blue), observed value (red), replicated mean (dashed black); $n_{\text{draws}}=100$

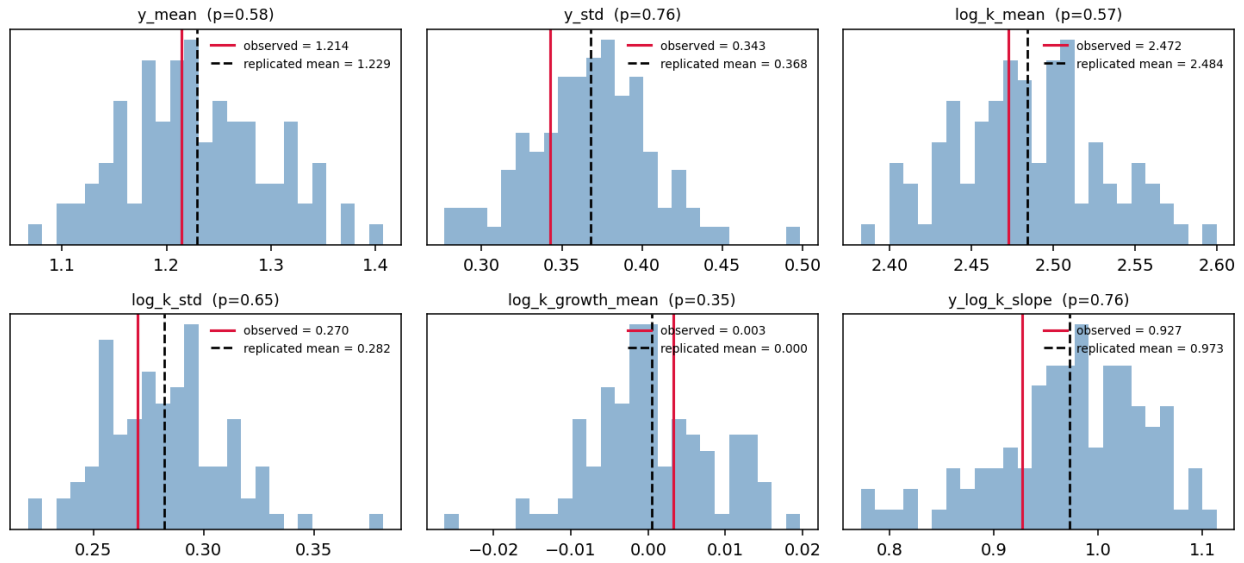


Figure 3.6: Plot coverage of 95% CI over R=5 replications.

Figure 3.6 compares six summary statistics of the observed panel against their distributions under panels simulated from the posterior. If the model fits the data well, each observed value should sit somewhere in the bulk of its simulated distribution. An extreme tail position flags a feature the model cannot reproduce. All six posterior median estimates sit in the central range. The fitted model reproduces these data features

without systematic misspecification.

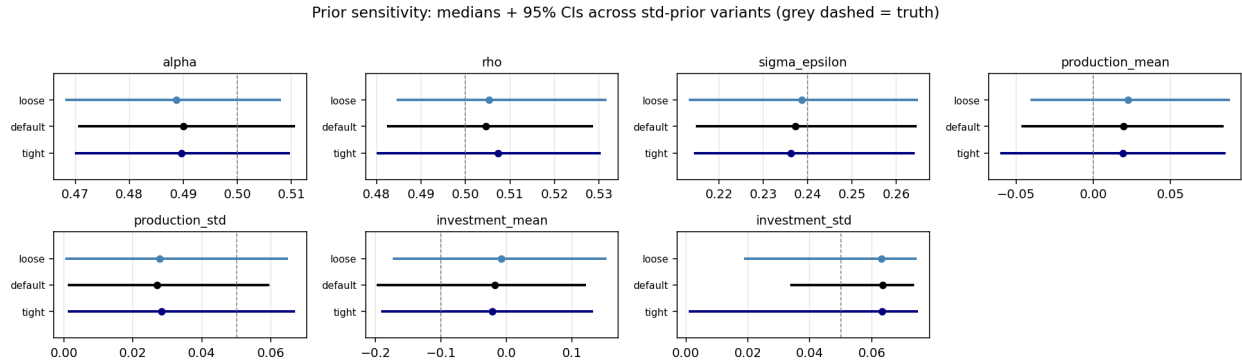


Figure 3.7: Plot coverage of 95% CI over R=5 replications.

Figure 3.7 overlays the posterior under three variants of the residual standard-deviation priors: tight, baseline, and loose. A posterior that barely moves across variants is data-driven, in contrast, one that tracks the prior is prior-driven. For all seven parameters the medians shift by a fraction of the credible-interval width and the intervals largely overlap. The data carries the identifying information here and the posterior estimates are generally robust.

3.4.2 RW-MH + Kalman Filter with Neural Surrogate Policy

The second validation exercise switches the gradient-based NUTS for a gradient-free RW-MH sampler, holding the Kalman filter unchanged. I argue that when using a pre-trained NN surrogate, RW-MH sampler is better than NUTS-HMC because backpropagation through the cached NN is still very slow per leapfrog iteration (>40 hours of wall time on Apple M1).

To make the attribution clean, I run three configurations side-by-side on the same observed panel at the same scalar proposal step size:

- **Closed-form RWMH** (control): RW-MH with the closed-form analytical policy. Isolates the sampler from any NN approximation error.
- **NN-RWMH** (test): same RW-MH, with the cached SHAC NN as the EKF's policy. Difference vs CF-RWMH attributes to NN approximation error.
- **NN-RAM** (adaptive variant): same NN spec, but with the proposal covariance adapted during warmup (Vihola 2012). Difference vs NN-RWMH attributes to the adaptive proposal.

All three are run at 4 chains $\times$ (20000 warmup + 5000 samples) on a 100 firms $\times$ 15 periods panel slice. CF finishes in about 30 seconds; the two NN methods take roughly 50 minutes each. The per-step NN evaluation accounts for the entire wall-time gap.

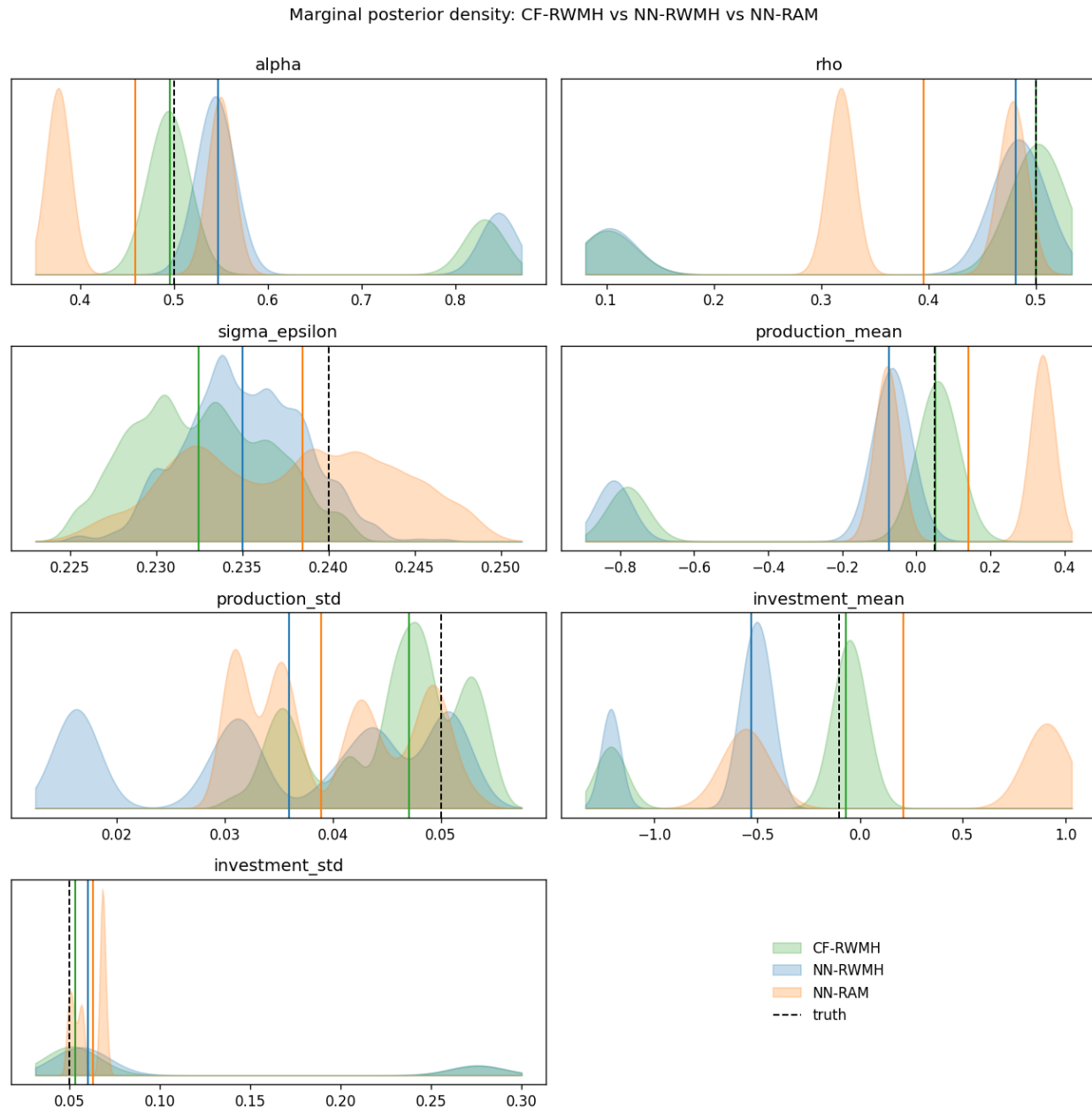


Figure 3.8: Posterior marginal densities under CF-RWMH (green, control with no NN), NN-RWMH (blue, baseline), and NN-RAM (orange, adaptive variant). Dashed black is truth; solid vertical lines are per-method medians.

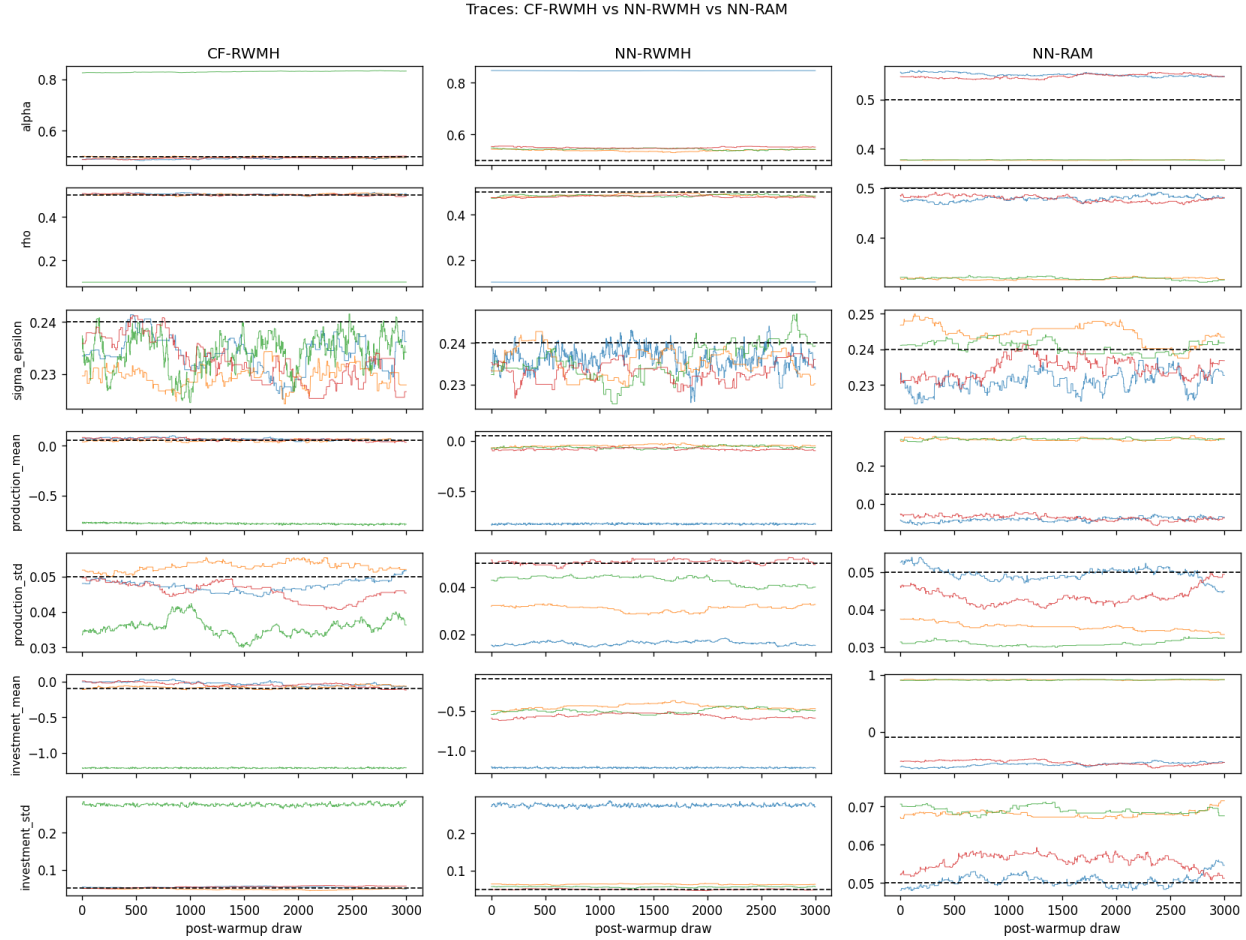


Figure 3.9: Trace plots: three columns (CF-RWMH, NN-RWMH, NN-RAM), one row per parameter, four chains overlaid in each cell. Well-mixed parameters show overlapping chains; ridge parameters show chains stuck in disjoint regions.

Does the evidence validate the pipeline? Partially. The results suggest the inference pipeline and the NN surrogate approach worked, but the identification is weak for several parameters which is attributed to the structural model specification.

Figure 3.8 overlays the marginal posterior density (instead of histogram) of the three configurations. The vertical lines mark the posterior mediana and the dashed line marks the true parameter value. It shows that:

- **Closed-form RWMH** (green) confirms the inference pipeline itself is correct, with posterior median very close to true value and a tight 95% CI
- **NN-RWMH** (blue) are noisier but the posterior density is still close to **Closed-form-RWMH**. This is important to confirm that the NN surrogate's approximation error does NOT break the inference. It just adds upfront pre-training cost for achieving better precision.
- **NN-RAM** (adaptive variant) rejects adaptive RW-MH as a usable variant at least for the basic model.

On the other hand, however, both the posterior density and the trace plot show failures in mixing. I view it as part of the model's specification issue. In particular, two parameters (ρ and μ_η) actively miss truth at

the 95% level, consistent with their broken chains and the inflated $\hat{R}$ and insufficient ESS.

The six poorly-mixed parameters all lie along the $\kappa(\alpha) + \mu_{\xi^k}$ ridge already flagged in the identification discussion in previous section: a tightly correlated subspace that a scalar Gaussian random walk cannot traverse efficiently within a practical wall budget. The CF-RWMH control reproduces the same failure pattern, which rules out NN approximation error as the dominant cause and isolates the bottleneck to the sampler class. In other words, **the NN surrogate is validated: it does not inject the mixing failure**. What fails is the gradient-free scalar random walk on a posterior geometry that the previous NUTS+CF baseline (with gradient information) handled cleanly. NN-RAM helps somewhat (max $\hat{R}$ 1670 vs 4138) but does not break the structural ceiling at this budget.

How to fix weak identification. Resolving the mixing failure requires two changes, both beyond the scope of the simple baseline reported here. First, the observation equations need to be reparametrized so that the ridge direction aligns with a single sampled quantity. Concretely, μ_{ξ^k} and $\kappa(\alpha, \sigma_\epsilon)$ both enter the log k equation additively as level terms, so sampling the composite offset $\zeta = \mu_{\xi^k} + \kappa(\alpha, \sigma_\epsilon)$ in place of μ_{ξ^k} removes the partial collinearity at sampler level; μ_{ξ^k} is recovered post-hoc as $\zeta - \kappa(\alpha, \sigma_\epsilon)$ from the joint posterior of $(\alpha, \sigma_\epsilon, \zeta)$. Second, the MCMC and filtering stack needs either a substantive extension or a replacement, for example particle filter with adaptive Metropolis (PMMH), sequential Monte Carlo with tempering, or full-covariance adaptive proposals that learn the ridge direction during warmup. Both directions are deferred to future implementation and testing.

Table 2: NN-RWMH posterior summary against truth (baseline NN + gradient-free path).

Parameter	True	Median	95% CI	$\hat{R}$	ESS
α	0.50	0.547	[0.534, 0.846]	4138	23
ρ	0.50	0.481	[0.103, 0.496]	2394	25
σ_ϵ	0.24	0.235	[0.229, 0.242]	1.3	65
μ_η	0.05	-0.073	[-0.824, -0.034]	2015	270
σ_η	0.05	0.036	[0.015, 0.052]	185	29
μ_{ξ^k}	-0.10	-0.528	[-1.219, -0.406]	250	347
σ_{ξ^k}	0.05	0.060	[0.048, 0.280]	3691	142

3.4.3 NUTS + Kalman Filter with Neural Surrogate Policy

3.5 Future Extensions

I consider the following directions as promising future extensions to better design and implementation of the final commercial product.

- **Neural Likelihood Estimation.** Train a neural network to replace the model-specific likelihood evaluation step, such as the Kalman filter likelihood in a state-space model. The MCMC sampler can remain unchanged, except that each likelihood evaluation inside MCMC is replaced by the neural likelihood surrogate. This is a major extension to our current policy surrogate approach. It shares the same motivation of **amortizing the cost of expensive model solves**: instead of resolving the structural model at every MCMC proposal, we solve/simulate the model many times upfront and train a neural network to approximate the likelihood. Once trained, the surrogate likelihood can be evaluated cheaply inside MCMC, making Bayesian inference feasible for models where repeated likelihood evaluation is computationally prohibitive.

- Benchmark the NN-surrogate inference under alternative filter + sampler pairings, especially **particle filter + Random Walk Metropolis-Hastings (PMMH)**. Although NUTS + Kalman is theoretically more efficient per iteration, the per-leapfrog gradient via autodiff through the NN is a measurable bottleneck on CPU; a gradient-free sampler paired with a non-Gaussian filter trades per-iteration efficiency for cheaper per-evaluation cost and parallelises more naturally on GPU. The cross-comparison would clarify whether the gradient-based pipeline remains the right default for NN-surrogate inference at scale.
- Address **weak-identification via re-parameterisation**. For example, in the current toy model μ_{ξ^k} (deviation) is weakly identified as it correlate strongly with α . Depending on the model and parameters of interest, we may need the re-parameterisation at algorithm level.
- **Hierarchical (multilevel) Bayesian model** for estimating firm-specific posteriors, which is more useful for the commercial product (e.g., firm's deviation from optimal capital structure). In this framework, each firm i has its own parameter vector β_i , linked through a shared population prior $\beta_i \sim p(\cdot \mid \beta_{\text{pop}})$ with β_{pop} drawn from a hyperprior. This sits between two extremes: full pooling (one β shared across all firms in the current setup) and full separation (independent per-firm MCMCs, which discard cross-sectional information). The mechanism is partial pooling: each firm's β_i deviates from β_{pop} where its own time series demands, but shrinks toward the population mean for parameters that are weakly identified within a single firm.

Chapter 4

Reserved

Reserved for future material.

Chapter 5

Nikolov, Schmid, and Steri (2021): Three Theoretical Models

5.1 Technology and Notation

This chapter implements linear programming (LP) methods to solve the three structural corporate finance models in Nikolov, Schmid, and Steri (2021):

- Trade-Off Model (TO)
- Limited Enforcement Model (LE)
- Moral Hazard Model (MH)

All three models share the same production technology, capital accumulation rule, and adjustment cost specification.

Operating profit (pre-tax):

$$\pi(k_{it}, z_{it}, \eta_{it}) = (z_{it} + \eta_{it})k_{it}^\alpha - f$$

After-tax operating cash flow is $(1 - \tau)\pi$. Paper Eq. 1 labels the after-tax expression $(1 - \tau)((z + \eta)k^\alpha - f)$ as π , but every subsequent equation uses $(1 - \tau)\pi$, so we treat π as pre-tax throughout this doc for consistency.

Capital accumulation (Eq. 2):

$$k_{it+1} = (1 - \delta)k_{it} + i_{it}$$

Adjustment cost (Eq. 3):

$$\Psi(k_{it+1}, k_{it}) = \frac{\psi}{2} \left(\frac{i_{it}}{k_{it}} \right)^2 k_{it} = \frac{\psi}{2} \left(\frac{k_{it+1} - (1 - \delta)k_{it}}{k_{it}} \right)^2 k_{it}$$

Variables and parameters: - k_{it} : capital stock of firm i at time t - i_{it} : investment at t - $z_{it} \in [\underline{z}, \bar{z}]$: persistent profitability shock, transition $Q(z'|z)$ - $\eta_{it} \in \{+\bar{\eta}, -\bar{\eta}\}$: i.i.d. disturbance, $P(\eta = +\bar{\eta}) = \kappa$ - $\tau \in (0, 1)$: corporate tax rate - $\alpha \in (0, 1)$: capital share (decreasing returns) - $f > 0$: fixed production cost - $\delta \in (0, 1)$: capital depreciation rate - ψ : adjustment cost parameter - r : risk-free interest rate - $\tau\delta k_{it}$: depreciation tax allowance

Capital cost convention. In the paper's Bellman and dividend formulas, the investment-related terms appear as $-k_{it} + (1 - \delta)k_{it} - \Psi(k_{it}, k_{it-1}) + \tau\delta k_{it}$, which simplifies to $-(1 - \tau)\delta k_{it} - \Psi$. The paper treats

capital as continuously maintained at level k_{it} (paying δk_{it} per period for depreciation replacement), with adjustment cost paid when changing the level. This is non-standard but internally consistent; the LP follows this convention.

5.1.1 Firm value W and equity value V

All three models maximize **firm value** $W(s_{it}, z_{it})$ at the end of period t : the PV of future cash flows to equity AND lenders combined, from $t + 1$ onward. In schematic form, W satisfies a Bellman recursion of the form

$$W(s, z) = \beta \max_a E[d(s, a, s', z', \eta') + W(s', z')]$$

where s is the model-specific state, a is the action, and d is the dividend or cash flow (with adjustments for default in TO).

Equity value V_{it} is the PV of dividends to shareholders alone, satisfying

$$V_{it} = \beta E[d_{z', \eta'} + V_{z', \eta'}]$$

In TO and LE, V is not tracked explicitly; firm value and equity value are linked by $W_{it} = V_{it} + V_{it}^{\text{debt}}$ where V_{it}^{debt} is the value of outstanding debt. In MH, incentive compatibility requires the firm to commit to a specific continuation equity value going forward, so V becomes a state variable in the MH Bellman.

5.2 Timing (common across all three models)

Within and across periods, events unfold sequentially as summarized below, following Figure 1 of the paper. The key convention is that $W(s_{it}, z_{it})$ is evaluated at the **end of period** t , after period- t shocks, production, repayments/transfers, and default checks have occurred, but before period- $(t + 1)$ shocks are realized. Decisions made at the end of period t determine capital and financing/contract terms for period $t + 1$.

State versus realized transfers. Persistent recursive states are $s = (k, b)$ in TO and LE and $s = (k, V)$ in MH. State-contingent payments such as p in LE and dividends d in MH are realized transfers chosen as part of the previous period's state-contingent contract; they are not persistent state variables except through the continuation balance b or continuation equity value V .

Point in time	Event(s)	TO	LE	MH
End of $t - 1$	Firm makes decisions to maximize $W(s_{i,t-1}, z_{i,t-1})$	Choose capital k_{it} and debt position b_{it} to be carried into period t ; lender break-even determines the spread attached to this debt contract.	Choose capital k_{it} and a state-contingent contract $\{b_{z_{it}, \eta_{it}}, p_{z_{it}, \eta_{it}}\}$ for period- t realizations.	Choose capital k_{it} and a state-contingent contract $\{V_{z_{it}, \eta_{it}}, d_{z_{it}, \eta_{it}}\}$ for period- t realizations.

Point in time	Event(s)	TO	LE	MH
Start/end of t	Shocks (z_{it}, η_{it}) realize; production and contractual transfers occur; the next recursive state is formed.	Production uses k_{it} ; the firm repays the debt obligation carried into period t if solvent, otherwise defaults and is liquidated. The next decision state, conditional on survival, is (k_{it}, b_{it}, z_{it}) .	The realized contract specifies payment $p_{z_{it}, \eta_{it}}$ and continuation balance $b_{z_{it}, \eta_{it}}$; no default occurs by design. The next decision state is $(k_{it}, b_{z_{it}, \eta_{it}}, z_{it})$.	Shareholders observe η_{it} , lenders do not; the realized contract specifies dividend $d_{z_{it}, \eta_{it}}$ and continuation equity value $V_{z_{it}, \eta_{it}}$; no default occurs by design. The next decision state is $(k_{it}, V_{z_{it}, \eta_{it}}, z_{it})$.
End of t	Firm makes decisions to maximize $W(s_{it}, z_{it})$	Choose capital $k_{i,t+1}$ and debt position $b_{i,t+1}$ to be carried into period $t+1$; lender break-even determines the spread attached to this debt contract.	Choose capital $k_{i,t+1}$ and a state-contingent contract $\{b_{z_{i,t+1}, \eta_{i,t+1}}, p_{z_{i,t+1}, \eta_{i,t+1}}\}$ for period- $(t+1)$ realizations.	Choose capital $k_{i,t+1}$ and a state-contingent contract $\{V_{z_{i,t+1}, \eta_{i,t+1}}, d_{z_{i,t+1}, \eta_{i,t+1}}\}$ for period- $(t+1)$ realizations.
Start/end of $t+1$	Shocks $(z_{i,t+1}, \eta_{i,t+1})$ realize; production and contractual transfers occur; the next recursive state is formed.	Production uses $k_{i,t+1}$; the firm repays the debt obligation carried into period $t+1$ if solvent, otherwise defaults and is liquidated. The next decision state, conditional on survival, is $(k_{i,t+1}, b_{i,t+1}, z_{i,t+1})$.	The realized contract specifies payment $p_{z_{i,t+1}, \eta_{i,t+1}}$ and continuation balance $b_{z_{i,t+1}, \eta_{i,t+1}}$; no default occurs by design. The next decision state is $(k_{i,t+1}, b_{z_{i,t+1}, \eta_{i,t+1}}, z_{i,t+1})$.	Shareholders observe $\eta_{i,t+1}$, lenders do not; the realized contract specifies dividend $d_{z_{i,t+1}, \eta_{i,t+1}}$ and continuation equity value $V_{z_{i,t+1}, \eta_{i,t+1}}$; no default occurs by design. The next decision state is $(k_{i,t+1}, V_{z_{i,t+1}, \eta_{i,t+1}}, z_{i,t+1})$.

The exact TO default condition, LE break-even/collateral constraints, and MH promise-keeping/incentive constraints are defined in the model-specific sections below. The timing table intentionally states the schedule in words rather than duplicating those equations.

5.3 Trade-off Model

5.3.1 Theoretical Model

In this setup, η_{it} is **public information**.

5.3.1.1 Financing

- Firms issue one-period bonds: cash inflow b_{it+1} at the beginning of period $t + 1$; previously issued bonds b_{it} are due with interest.
- Default premium Δ_{it} is charged above r , so the effective interest rate is $r + \Delta_{it}$.
- Interest payments are tax deductible: effective repayment due in period $t + 1$ is $(1 + (1 - \tau)(r + \Delta_{it}))b_{it}$.
- Tax shield: $\tau(r + \Delta_{it})b_{it}$.

Solvency condition (firm is solvent iff):

$$(1 - \tau)\pi(k_{it}, z_{it}, \eta_{it}) + (1 - \delta)k_{it} + \tau\delta k_{it} - (1 + (r + \Delta_{it-1})(1 - \tau))b_{it-1} \geq 0 \quad (4)$$

Default set:

$$D_{it} \equiv \{(z_{it}, \eta_{it}, k_{it}, \Delta_{it-1}) \in \bar{Z} \times \bar{N} \times \mathbb{R}^+ \times \mathbb{R}^+ : (4) \text{ does not hold}\}$$

- $\bar{D}_{it}$: set of solvency states (where Eq. 4 holds)
- $\mathcal{J}_{D,it}$: indicator function for default

Creditor break-even condition (risk-neutral pricing):

$$E_{t-1} \left[(1 + r + \Delta_{it-1})(1 - \mathcal{J}_{D,it}) + \frac{\xi(1 - \delta)k_{it}}{b_{it-1}} \mathcal{J}_{D,it} \right] = 1 + r$$

- ξ : recovery rate in bankruptcy

Payouts (limited liability, seasoned equity precluded):

$$d_{it} \equiv (1 - \tau)\pi(k_{it}, z_{it}, \eta_{it}) - k_{it} + (1 - \delta)k_{it} - \Psi(k_{it}, k_{it-1}) + \tau\delta k_{it} - (1 + (r + \Delta_{it-1})(1 - \tau))b_{it-1} + b_{it} \geq 0$$

5.3.1.2 Firm Problem

State variables: $(k_{it-1}, b_{it-1}, z_{it-1})$. Bellman equation:

$$\begin{aligned} W(k_{it-1}, b_{it-1}, z_{it-1}) \equiv & \frac{1}{1 + r} \max_{k_{it}, b_{it}} \left\{ -k_{it} + (1 - \delta)k_{it} - \Psi(k_{it}, k_{it-1}) + \tau\delta k_{it} \right. \\ & + \tau(r + \Delta_{it-1})b_{it-1}\mathcal{J}_{1-D,it} - ((1 - \xi)(1 - \delta)k_{it} + \tau\delta k_{it})\mathcal{J}_{D,it} \\ & \left. + E_{t-1}[(1 - \tau)\pi(k_{it}, z_{it}, \eta_{it}) + W(k_{it}, b_{it}, z_{it})] \right\} \end{aligned}$$

subject to:

$$(1 - \tau)\pi(k_{it}, z_{it}, \eta_{it}) - k_{it} + (1 - \delta)k_{it} - \Psi(k_{it}, k_{it-1}) + \tau\delta k_{it} - (1 + (r + \Delta_{it-1})(1 - \tau))b_{it-1} + b_{it} \geq 0, \quad \forall z_{it}, \eta_{it}$$

$$E_{t-1} \left[(1 + r + \Delta_{it-1})(1 - \mathcal{J}_{D,it}) + \frac{\xi(1 - \delta)k_{it}}{b_{it-1}} \mathcal{J}_{D,it} \right] = 1 + r$$

5.3.2 LP Solution Method

The baseline LP solutions (TO, LE, MH) use **NumPy** for array operations, **SciPy sparse matrices** for constraint assembly, and **scipy.optimize.linprog with the 'highs' backend** for the master LP solve. TensorFlow is not used: there is no neural-net training, no autodiff requirement, and no GPU benefit for the modest finite-grid baseline. The workload is dominated by grid construction, finite-action enumeration or action filtering, sparse matrix assembly, and LP solving, all of which NumPy and SciPy handle with mature, well-tested implementations.

LP state interpretation. The LP uses the paper's recursive timing. A state (k, b, z) (or (k, V, z) in MH) is the inherited end-of-period state: k is the capital level from the previous choice, b is the outstanding promised balance (or V is promised equity value), and z is the current public persistent shock. At this state the firm chooses the next capital level and financing contract, denoted (k', b') in TO, state-contingent $(b'_{z', \eta'}, p_{z', \eta'})$ in LE, and state-contingent $(V'_{z', \eta'}, d_{z', \eta'})$ in MH. The Bellman target then evaluates next-period shocks (z', η') , operating cash flow with the chosen capital k' , repayment/contract constraints, and continuation value at the chosen next state.

Baseline implementation modules. A coding agent should keep the implementation modular: (i) grids for K, B, V , payment/dividend grids, and shock transition matrices; (ii) primitives for π, Ψ , discount factors, and model parameters; (iii) timing/index helpers that map current state, action, shock realization, and continuation state; (iv) TO pricing for $\Delta(k_{\text{choice}}, b_{\text{old}}, z_{\text{old}})$ and default indicators; (v) LE/MH `action_enumeration` for fixed finite contract menus and feasibility filtering; (vi) `lp_assembly` for sparse Bellman inequalities; (vii) `lp_solve` for HiGHS; (viii) `policy_recovery` for argmax policies; and (ix) diagnostics for feasibility counts, LP residuals, pricing residuals, and boundary-hit warnings.

5.3.2.1 Primitives and grids

State $S = K \times B \times Z$ with discrete grids $K = \{k_1, \dots, k_{n_k}\}$, $B = \{b_1, \dots, b_{n_b}\}$, $Z = \{z_1, \dots, z_{n_z}\}$. Action $A = K \times B$ (choice of next-period (k', b')).

State (k, b, z) : capital in place, outstanding debt, current persistent shock. I.i.d. shock $\eta \in \{+\bar{\eta}, -\bar{\eta}\}$ with $P(\eta = +\bar{\eta}) = \kappa$. Persistent transition $Q(z' | z)$. Discount $\beta = 1/(1+r)$.

Model functions:

$$\pi(k, z, \eta) = (z + \eta)k^\alpha - f, \quad \Psi(k', k) = \frac{\psi}{2}((k' - (1 - \delta)k)/k)^2 k$$

Parameters: $\tau, \alpha, f, \delta, \psi, \xi, r, \kappa, \bar{\eta}$.

5.3.2.2 Step 1: Pre-compute lender pricing $\Delta(k_{\text{choice}}, b_{\text{old}}, z_{\text{old}})$

For every candidate chosen capital $k_{\text{choice}} \in K$, inherited debt $b_{\text{old}} \in B$, and inherited persistent shock $z_{\text{old}} \in Z$, compute the risky-debt premium that satisfies the lender break-even condition:

$$\sum_{z'} Q(z' | z_{\text{old}}) \sum_{\eta'} P(\eta') \left[(1 + r + \Delta)(1 - \mathcal{J}'_D) + \frac{\xi(1 - \delta)k_{\text{choice}}}{b_{\text{old}}} \mathcal{J}'_D \right] = 1 + r$$

with

$$\mathcal{J}'_D = \mathbf{1}\{(1 - \tau)\pi(k_{\text{choice}}, z', \eta') + (1 - \delta)k_{\text{choice}} + \tau\delta k_{\text{choice}} < (1 + (1 - \tau)(r + \Delta))b_{\text{old}}\}.$$

Important timing convention. In the Bellman constraint for state (k, b, z) and action (k', b') , the relevant premium on the outstanding debt b is $\Delta(k', b, z)$, not $\Delta(k, b, z)$. The premium is priced at the time the old debt is issued, using the capital level chosen for the period in which that debt will be repaid.

Robust finite-shock pricing solver. Do not rely on naive bisection as the baseline. Because $\mathcal{J}'_D$ changes with Δ on a discrete shock grid, the lender payoff is piecewise linear with possible jumps. Implement pricing as follows:

1. If $b_{\text{old}} = 0$, set $\Delta = 0$ and $\mathcal{J}'_D = 0$.
2. If $b_{\text{old}} > 0$, compute the default-switch threshold for each future shock realization:

$$\Delta^*(z', \eta') = \frac{(1 - \tau)\pi(k_{\text{choice}}, z', \eta') + (1 - \delta)k_{\text{choice}} + \tau\delta k_{\text{choice}}}{(1 - \tau)b_{\text{old}}} - \frac{1}{1 - \tau} - r.$$

A realization defaults when $\Delta > \Delta^*(z', \eta')$.

3. Sort the finite set of thresholds and examine the intervals over which the default set is fixed.
4. On each interval, solve the break-even equation analytically because the default set is fixed and the expected lender payoff is linear in Δ .
5. Keep the lowest nonnegative Δ whose implied default set is internally consistent and whose expected payoff equals $1 + r$ up to tolerance. If discreteness prevents exact equality, use the lowest nonnegative Δ for which expected lender payoff is weakly at least $1 + r$, and record the pricing residual.

Output: - 3D premium table $\Delta(k_{\text{choice}}, b_{\text{old}}, z_{\text{old}})$ of size $n_k \cdot n_b \cdot n_z$. - Optional default indicator table $\mathcal{J}_D(k_{\text{choice}}, b_{\text{old}}, z_{\text{old}}, z', \eta')$, evaluated at the stored premium.

In practice, materialize only the 3D premium table unless debugging. Given the stored $\Delta(k_{\text{choice}}, b_{\text{old}}, z_{\text{old}})$ and the analytic default condition, recompute the indicator on demand to save memory.

5.3.2.3 Step 2: LP

Variables. $W(k, b, z)$ for every $(k, b, z) \in S$. Count: $n_k n_b n_z$.

Objective.

$$\min_W \sum_{(k, b, z) \in S} W(k, b, z)$$

Bellman constraints. Following the paper's appendix Eq. (22), for every $(k, b, z) \in S$ and every feasible $(k', b') \in A$:

$$W(k, b, z) \geq \frac{1}{1+r} \left[\underbrace{-k' + (1 - \delta)k' - \Psi(k', k) + \tau\delta k'}_{\text{deterministic, paid regardless of default}} + \sum_{z', \eta'} Q(z' | z) P(\eta') \left(\underbrace{(1 - \tau)\pi(k', z', \eta')}_{\text{profit, always realized}} + \underbrace{\tau(r + \Delta(k', b, z))b}_{\text{tax shield if solvent}} \mathcal{J}_{1-D} - \underbrace{(1 - \tau)(r + \Delta(k', b, z))b}_{\text{tax shield if insolvent}} \right) \right]$$

where the default indicator $\mathcal{J}_D = \mathcal{J}_D(k', b, z, z', \eta')$ at the next-shock realization is

$$\mathcal{J}_D = \mathbf{1}\{(1 - \tau)\pi(k', z', \eta') + (1 - \delta)k' + \tau\delta k' < (1 + (1 - \tau)(r + \Delta(k', b, z)))b\}$$

and $\mathcal{J}_{1-D} = 1 - \mathcal{J}_D$. The tax shield is on the OUTSTANDING bond b with its premium $\Delta(k', b, z)$, per the paper's Section 2.2 financing description and the timing convention above.

Limited liability (feasibility filter). A pair (k', b') is feasible at (k, b, z) only if shareholder dividend $d \geq 0$ in every solvent realization (z', η') :

$$d = (1-\tau)\pi(k', z', \eta') - k' + (1-\delta)k' - \Psi(k', k) + \tau\delta k' - (1+(1-\tau)(r+\Delta(k', b, z)))b + b' \geq 0, \quad \forall (z', \eta') \text{ with } \mathcal{J}_D = 0$$

Drop the Bellman constraint at (k, b, z, k', b') if LL is violated. Default-state dividends are zero by limited liability (firm wiped out, equity holders receive nothing).

Constraint count (post-filter): bounded by $n_k^2 n_b^2 n_z$, sparser after filtering.

5.3.2.4 Step 3: Recover the policy

After the LP returns W^* :

$$(k', b')^*(k, b, z) = \arg \max_{(k', b') \in A \text{ feasible}} \text{RHS}(k, b, z, k', b'; W^*)$$

where RHS is the same Bellman target as in Step 2, with W^* plugged in for the continuation. Implied premium policy for the chosen next debt is $\Delta^*(k, b, z) = \Delta(k'^*(k, b, z), b'^*(k, b, z), z)$; within the Bellman RHS for repayment of inherited debt b , use $\Delta(k', b, z)$.

5.3.2.5 Implementation notes

- **Step 1:** finite-threshold pricing over the $(k_{\text{choice}}, b_{\text{old}}, z_{\text{old}})$ grid; vectorize threshold construction where possible, but keep explicit consistency checks for default sets and pricing residuals.
- **Step 2 LP construction:** NumPy broadcasting to build the $(n_k, n_b, n_z, n_k, n_b)$ Bellman RHS coefficient tensor and LL feasibility mask; assemble constraints as `scipy.sparse.csc_matrix`.
- **Step 2 LP solve:** `scipy.optimize.linprog` with `method='highs'`.
- **Step 3:** `np.argmax` over the RHS tensor with W^* plugged in.

5.4 Limited Enforcement Model

5.4.1 Theoretical Model

State-contingent payoffs are allowed. In this context, η_{it} is **public information**.

5.4.1.1 Financing

- Firms sell a portfolio of securities whose payoffs are contingent on next-period shocks z_{it+1} and η_{it+1} .
- Selling the portfolio at time t raises:

$$b_{it} \equiv \frac{1}{1+r} E_t [p_{z_{it+1}, \eta_{it+1}} + b_{z_{it+1}, \eta_{it+1}}]$$

- $p_{z_{it+1}, \eta_{it+1}}$: cash flow transferred to investors contingent on shocks
- $b_{z_{it+1}, \eta_{it+1}}$: residual present value of future promised repayments

Intuitively, the contract operates like a flexible credit line: b_{it} is the outstanding balance today; $p_{z_{it+1}, \eta_{it+1}}$ is the payment the firm makes next period contingent on the realized state; and $b_{z_{it+1}, \eta_{it+1}}$ is the new outstanding balance after that payment, carried forward as the firm's debt at $t+1$.

Collateral constraint (state-contingent debt must be fully collateralized):

$$p_{z_{it+1}, \eta_{it+1}} + b_{z_{it+1}, \eta_{it+1}} \leq \theta(1 - \delta)k_{it+1}, \quad \forall z_{it+1}, \eta_{it+1}$$

- θ : fraction of capital that can be pledged as collateral

Limited liability (payouts):

$$d_{it} \equiv (1 - \tau)\pi(k_{it}, z_{it}, \eta_{it}) - k_{it} + (1 - \delta)k_{it} - \Psi(k_{it}, k_{it-1}) + \tau\delta k_{it} + \tau r b_{it-1} - p_{z_{it}, \eta_{it}} \geq 0$$

5.4.1.2 Firm Problem

Bellman equation:

$$W(k_{it-1}, b_{it-1}, z_{it-1}) = \frac{1}{1 + r} \max_{k_{it}, b_{z_{it}, \eta_{it}}, p_{z_{it}, \eta_{it}}} \left\{ -k_{it} + (1 - \delta)k_{it} - \Psi(k_{it}, k_{it-1}) + \tau\delta k_{it} \right. \\ \left. + \tau r b_{it-1} + E_{t-1}[(1 - \tau)\pi(k_{it}, z_{it}, \eta_{it}) + W(k_{it}, b_{z_{it}, \eta_{it}}, z_{it})] \right\}$$

subject to:

$$b_{it-1} \equiv \frac{1}{1 + r} E_{t-1}[p_{z_{it}, \eta_{it}} + b_{z_{it}, \eta_{it}}] \quad (5)$$

$$(1 - \tau)\pi(k_{it}, z_{it}, \eta_{it}) - k_{it} + (1 - \delta)k_{it} - \Psi(k_{it}, k_{it-1}) + \tau\delta k_{it} + \tau r b_{it-1} - p_{z_{it}, \eta_{it}} \geq 0, \quad \forall z_{it}, \eta_{it} \quad (6)$$

$$p_{z_{it}, \eta_{it}} + b_{z_{it}, \eta_{it}} \leq \theta(1 - \delta)k_{it}, \quad \forall z_{it}, \eta_{it} \quad (7)$$

5.4.2 LP Solution Method

5.4.2.1 Primitives and grids

State $S = K \times B \times Z$ with the same K and Z grids as 1.2 and a nonnegative debt grid $B = \{b_1, \dots, b_{n_b}\} \subset \mathbb{R}_+$ with $0 \in B$. Additional parameter: θ (collateral fraction). Shock η and persistent shock z are as in 1.2.

State (k, b, z) : inherited capital, inherited promised balance, and current persistent shock. There is no default risk and no premium Δ in LE. Pricing is imposed by the risk-neutral break-even constraint (5).

5.4.2.2 Minimal finite-action baseline

To keep the baseline a true LP, use a **finite contract-action menu**. For each current state (k, b, z) and candidate next capital $k' \in K$, a contract action is a fixed collection

$$a = \{b'_{z', \eta'}, p_{z', \eta'}\}_{z' \in Z, \eta' \in N}$$

where each $b'_{z', \eta'} \in B$ and each $p_{z', \eta'} \in P$, with $P = \{p_1, \dots, p_{n_p}\} \subset \mathbb{R}_+$ a nonnegative payment grid. Because the contract components are fixed before a Bellman inequality is added, the continuation terms $W(k', b'_{z', \eta'}, z')$ enter linearly with known coefficients.

Do **not** include convex-combination weights such as $\lambda_j W(k', b_j, z')$ as free variables in the master LP. That would be bilinear, not linear. Continuous-state interpolation can be added later only through constraint generation, where interpolation weights are fixed constants when each Bellman inequality is added.

5.4.2.3 Step 1: Build feasible contract-action lists

For every (k, b, z, k') , enumerate candidate contract actions $a = \{b'_{z', \eta'}, p_{z', \eta'}\}$. Keep only actions satisfying all constraints below.

- **Break-even (Eq. 5):**

$$\frac{1}{1+r} \sum_{z', \eta'} Q(z' | z) P(\eta') [p_{z', \eta'} + b'_{z', \eta'}] = b.$$

In code, because P and B are discrete, use a tight tolerance `be_tol`:

$$\left| \frac{1}{1+r} \sum_{z', \eta'} Q(z' | z) P(\eta') [p_{z', \eta'} + b'_{z', \eta'}] - b \right| \leq \text{be_tol}.$$

- **Limited liability (Eq. 6):** for each (z', η') ,

$$p_{z', \eta'} \leq (1 - \tau)\pi(k', z', \eta') - k' + (1 - \delta)k' - \Psi(k', k) + \tau\delta k' + \tau r b.$$

- **Collateral (Eq. 7):** for each (z', η') ,

$$p_{z', \eta'} + b'_{z', \eta'} \leq \theta(1 - \delta)k'.$$

- **Sign restrictions:**

$$p_{z', \eta'} \geq 0, \quad b'_{z', \eta'} \geq 0.$$

A tuple (k, b, z, k') contributes no Bellman constraint if no feasible contract action exists for that particular k' . However, every state (k, b, z) must have at least one feasible action across all k' . If a state has no feasible action, treat this as a grid-design failure: enlarge the payment grid P , enlarge/shift the debt grid B , relax only the numerical tolerance if the miss is purely rounding error, or remove the state from the admissible state grid. Do not solve the LP with states that have an empty feasible action correspondence.

For the minimal baseline, this brute-force enumeration is intended for small grids only. A faster second-stage implementation can replace enumeration with constraint generation or an auxiliary search routine, but the master LP must still receive only fixed actions.

5.4.2.4 Step 2: Master LP

Variables. $W(k, b, z)$ for every $(k, b, z) \in S$. Count: $n_k n_b n_z$.

Objective.

$$\min_W \sum_{(k, b, z) \in S} W(k, b, z).$$

Bellman constraints. For every state (k, b, z) and every feasible fixed action (k', a) :

$$W(k, b, z) \geq \frac{1}{1+r} \left[-k' + (1-\delta)k' - \Psi(k', k) + \tau\delta k' + \tau r b + \sum_{z', \eta'} Q(z' | z) P(\eta') \left((1-\tau)\pi(k', z', \eta') + W(k', b'_{z', \eta'}, z') \right) \right].$$

The entire deterministic term is inside the discount factor $1/(1 + r)$, matching the paper's LE Bellman equation.

Constraint count depends on the number of feasible fixed contract actions. This can grow quickly with $n_z n_\eta$, so the minimal implementation should start with small grids.

5.4.2.5 Step 3: Recover the policy

After the LP returns W^* , choose the feasible fixed action that maximizes the RHS at each state:

$$(k', a)^*(k, b, z) = \arg \max_{(k', a) \in \mathcal{A}_{LE}(k, b, z)} \text{RHS}_{LE}(k, b, z, k', a; W^*).$$

The policy consists of $k'^*(k, b, z)$ and the associated state-contingent contract $\{b_{z', \eta'}^*, p_{z', \eta'}^*\}$.

5.4.2.6 Implementation notes

- **Baseline:** enumerate finite contract actions on small B and P grids, filter by break-even, limited liability, collateral, and sign restrictions, then assemble the sparse LP.
- **Correctness condition:** each Bellman inequality must correspond to a fixed feasible action. The continuation coefficient on each $W(k', b_j, z')$ is therefore a known probability weight, not a decision variable.
- **Scaling upgrade:** if enumeration becomes too large, use constraint generation. Given a current value vector $W^{(m)}$, solve a separate action-search problem for each state, freeze the selected action, add the corresponding linear Bellman inequality to the master LP, and repeat until no violated constraints remain.

5.5 Moral Hazard Model

5.5.1 Theoretical Model

Asymmetric information setup: - z_{it} follows a Markov chain that is **publicly observable** (also by the lender). - η_{it} is **observable by shareholders but unobservable by lenders**.

A lending contract is a sharing rule splitting firm resources between payments to the lender p_{it} and dividends d_{it} , in a fully state-contingent manner.

5.5.1.1 State variable choice

- Use equity value of the firm V_{it} as the state variable; debt value recovered from $b_{it} = W_{it} - V_{it}$.
- Tax deductibility of interest on debt: $\tau r b_{it} = \tau r (W_{it} - V_{it})$, which yields:
 - Adjusted discount rate for the firm: $1/(1 + (1 - \tau)r)$ instead of $1/(1 + r)$
 - Penalty for foregone tax deductions on debt: $\tau r V_{it}$

5.5.1.2 Diversion function

$$\mathcal{D}(k_{it}, z_{it}, \eta_{it}, \hat{\eta}_{it})$$

- $\hat{\eta}_{it}$: shareholders' (potentially misreported) report of η_{it}
- Most straightforward specification under the pre-tax π convention used in this document: $\mathcal{D} = \lambda(1 - \tau) [\pi(k_{it}, z_{it}, \eta_{it}) - \pi(k_{it}, z_{it}, \hat{\eta}_{it})]$.

- λ : diversion parameter; $1 - \lambda$ captures potential losses in cash flow diversion

5.5.1.3 Firm value function

$$\begin{aligned}
 W(k_{it-1}, V_{it-1}, z_{it-1}) = & \max_{k_{it}, V_{z_{it}, \eta_{it}}, d_{z_{it}, \eta_{it}}} \frac{1}{1 + (1 - \tau)r} \\
 & \times \left[-k_{it} - \Psi(k_{it}, k_{it-1}) + (1 - \delta)k_{it} + \tau \delta k_{it} - r\tau V_{it-1} \right. \\
 & \left. + E_{t-1}[(1 - \tau)\pi(k_{it}, z_{it}, \eta_{it}) + W(k_{it}, V_{z_{it}, \eta_{it}}, z_{it})] \right]
 \end{aligned}$$

subject to:

Promise-keeping constraint:

$$V_{it-1} = \frac{1}{1 + r} E_{t-1}[d_{z_{it}, \eta_{it}} + V_{z_{it}, \eta_{it}}] \quad (8)$$

Incentive compatibility constraints:

$$d_{z_{it}, \eta_{it}} + V_{z_{it}, \eta_{it}} \geq d_{z_{it}, \hat{\eta}_{it}} + V_{z_{it}, \hat{\eta}_{it}} + \mathcal{D}(k_{it}, z_{it}, \eta_{it}, \hat{\eta}_{it}), \quad \forall z_{it}, \forall \hat{\eta}_{it} \quad (9)$$

Limited liability constraints (non-negativity):

$$d_{z_{it}, \eta_{it}} \geq 0, \quad \forall z_{it}, \forall \eta_{it} \quad (10)$$

$$V_{z_{it}, \eta_{it}} \geq 0, \quad \forall z_{it}, \forall \eta_{it} \quad (11)$$

5.5.1.4 Payments to the lender (recovered from resource constraint)

$$p_{it} = -k_{it} - \Psi(k_{it}, k_{it-1}) + (1 - \delta)k_{it} + \tau \delta k_{it} + \tau r(W_{it} - V_{it}) + (1 - \tau)\pi(k_{it}, z_{it}, \eta_{it}) - d_{it}$$

Variables (moral hazard model):

- V_{it-1} : equity value at the end of period $t - 1$
- $d_{z_{it}, \eta_{it}}$: state-contingent dividend payment
- $V_{z_{it}, \eta_{it}}$: state-contingent continuation equity value
- p_{it} : state-contingent payment to lender

5.5.2 LP Solution Method

5.5.2.1 Primitives and grids

State $S = K \times \mathcal{V} \times Z$ with discrete grids $K = \{k_1, \dots, k_{n_k}\}$, $\mathcal{V} = \{V_1, \dots, V_{n_v}\} \subset \mathbb{R}_+$ with $V_1 = 0$, and $Z = \{z_1, \dots, z_{n_z}\}$. Shock $\eta \in \{+\bar{\eta}, -\bar{\eta}\}$ with $P(\eta = +\bar{\eta}) = \kappa$. Persistent transition $Q(z' | z)$. Additional parameter: λ (diversion fraction).

State (k, V, z) : inherited capital, promised equity value, and current public persistent shock.

Discount in the firm Bellman: $1/(1+(1-\tau)r)$ (firm-side, with the tax shield on the debt component already embedded). Discount in promise-keeping: $1/(1+r)$ (equity-holder side, no debt tax shield).

Under the pre-tax π convention used in this document, the diversion function is

$$\mathcal{D}(k', z', \eta', \hat{\eta}') = \lambda(1 - \tau)[\pi(k', z', \eta') - \pi(k', z', \hat{\eta}')].$$

5.5.2.2 Minimal finite-action baseline

To keep the baseline a true LP, use a **finite contract-action menu**. For each current state (k, V, z) and candidate next capital $k' \in K$, a contract action is a fixed collection

$$a = \{V'_{z', \eta'}, d_{z', \eta'}\}_{z' \in Z, \eta' \in N},$$

where each $V'_{z', \eta'} \in \mathcal{V}$ and each $d_{z', \eta'} \in D$, with $D = \{d_1, \dots, d_{n_d}\} \subset \mathbb{R}_+$ a nonnegative dividend grid. Because the contract components are fixed before a Bellman inequality is added, the continuation terms $W(k', V'_{z', \eta'}, z')$ enter linearly with known coefficients.

Do **not** include convex-combination weights such as $\mu_j W(k', V_j, z')$ as free variables in the master LP. That would be bilinear, not linear. Continuous interpolation can be added later only through constraint generation, where interpolation weights are fixed constants when each Bellman inequality is added.

5.5.2.3 Step 1: Build feasible contract-action lists

For every (k, V, z, k') , enumerate candidate contract actions $a = \{V'_{z', \eta'}, d_{z', \eta'}\}$. Keep only actions satisfying all constraints below.

- **Promise keeping (Eq. 8):**

$$V = \frac{1}{1+r} \sum_{z', \eta'} Q(z' | z) P(\eta') [d_{z', \eta'} + V'_{z', \eta'}].$$

In code, because D and $\mathcal{V}$ are discrete, use a tight tolerance `pk_tol`:

$$\left| \frac{1}{1+r} \sum_{z', \eta'} Q(z' | z) P(\eta') [d_{z', \eta'} + V'_{z', \eta'}] - V \right| \leq \text{pk_tol}.$$

- **Incentive compatibility (Eq. 9):** for each z' and each pair $(\eta', \hat{\eta}')$ with $\eta' \neq \hat{\eta}'$,

$$d_{z', \eta'} + V'_{z', \eta'} \geq d_{z', \hat{\eta}'} + V'_{z', \hat{\eta}'} + \mathcal{D}(k', z', \eta', \hat{\eta}').$$

With binary η' , there are two non-trivial IC inequalities per z' .

• **Limited liability (Eqs. 10, 11):**

$$d_{z',\eta'} \geq 0, \quad V'_{z',\eta'} \geq 0.$$

A tuple (k, V, z, k') contributes no Bellman constraint if no feasible contract action exists for that particular k' . However, every state (k, V, z) must have at least one feasible action across all k' . If a state has no feasible action, treat this as a grid-design failure: enlarge the dividend grid D , enlarge/shift the promised-equity grid $\mathcal{V}$, relax only the numerical tolerance if the miss is purely rounding error, or remove the state from the admissible state grid. Do not solve the LP with states that have an empty feasible action correspondence.

The minimal baseline is intended for small grids only. A faster second-stage implementation can replace enumeration with constraint generation or an auxiliary search routine, but the master LP must still receive only fixed actions.

5.5.2.4 Step 2: Master LP

Variables. $W(k, V, z)$ for every $(k, V, z) \in S$. Count: $n_k n_V n_z$.

Objective.

$$\min_W \sum_{(k,V,z) \in S} W(k, V, z).$$

Bellman constraints. For every state (k, V, z) and every feasible fixed action (k', a) :

$$W(k, V, z) \geq \frac{1}{1 + (1 - \tau)r} \left[-k' - \Psi(k', k) + (1 - \delta)k' + \tau \delta k' - r\tau V + \sum_{z', \eta'} Q(z' | z) P(\eta') \left((1 - \tau)\pi(k', z', \eta') + W(k', V'_{z', \eta'}, z') \right) \right].$$

5.5.2.5 Step 3: Recover the policy

After the LP returns W^* , choose the feasible fixed action that maximizes the RHS at each state:

$$(k', a)^*(k, V, z) = \arg \max_{(k', a) \in \mathcal{A}_{MH}(k, V, z)} \text{RHS}_{MH}(k, V, z, k', a; W^*).$$

The policy consists of $k'^*(k, V, z)$ and the associated state-contingent contract $\{V'^*_{z', \eta'}, d^*_{z', \eta'}\}$.

Payment to the lender is recovered after the LP solve as the residual from the resource constraint. Under the pre-tax π convention, for each realized (z', η') :

$$p^*_{z', \eta'} = -k'^* - \Psi(k'^*, k) + (1 - \delta)k'^* + \tau \delta k'^* + \tau r (W^*(k'^*, V'^*_{z', \eta'}, z') - V'^*_{z', \eta'}) + (1 - \tau)\pi(k'^*, z', \eta') - d^*_{z', \eta'}.$$

This recovered payment is for accounting and simulated policy measurement. It should not be reintroduced into the master LP as an additional constraint involving $W - V$, because the Bellman equation already embeds the tax-shield logic through the adjusted discount factor and the $-r\tau V$ term.

5.5.2.6 Implementation notes

- **Baseline:** enumerate finite contract actions on small $\mathcal{V}$ and D grids, filter by promise keeping, IC, and limited liability, then assemble the sparse LP.
- **Correctness condition:** each Bellman inequality must correspond to a fixed feasible action. The continuation coefficient on each $W(k', V_j, z')$ is therefore a known probability weight, not a decision variable.
- **Scaling upgrade:** if enumeration becomes too large, use constraint generation. Given a current value vector $W^{(m)}$, solve a separate action-search problem for each state, freeze the selected action, add the corresponding linear Bellman inequality to the master LP, and repeat until no violated constraints remain.

5.5.2.6.1 Future scaling path: paper-style constraint generation

The current implementation target is a modest local baseline: TO is solved by full finite-action LP, while LE and MH use small finite menus of complete state-contingent contracts. This is sufficient for a clean conceptual implementation and for verifying timing, feasibility, and policy recovery. It is not intended to be paper-scale for LE/MH.

For larger LE/MH grids, follow the paper’s LP plus constraint-generation logic. The master problem contains only an active subset of Bellman inequalities, each corresponding to a fixed feasible action. After solving the relaxed master LP, use a separation oracle to search, state by state, for a feasible contract action whose Bellman RHS violates the current value function by more than tolerance. If such an action is found, freeze that complete action (including any interpolation weights, if used), add the resulting linear Bellman inequality to the master LP, and repeat until no violated constraints remain. The paper implements the action-search/separation step with mixed-integer programming because the state-contingent contracting action space is large. A future scalable implementation can therefore add a `constraint_generation` module and an optional `separation_oracle` module, likely using a stronger optimization backend such as Gurobi or CPLEX for the oracle. At the current baseline stage, these modules are documented but not implemented.

5.5.2.6.2 Why LP instead of NN based methods?

Models like TO, LE, and MH are complex because they (i) do not have closed-form Euler equations, (ii) featured nested fixed points in equilibrium, and (iii) have different equality and inequality constraints. There is generally no mathematical theorem proving that NN-based training can converge to a *unique fixed point*.

In contrast, grid-based numerical methods like VFI and LP are guaranteed to converge to the unique fixed point of a finite discounted dynamic programming problem under a set of conditions (e.g., contraction mapping). Practically, we need the problem to satisfy:

1. The state grid is finite.
2. Every Bellman inequality corresponds to a fixed feasible action with fixed continuation-state indices and fixed probability weights.
3. The relevant discount factor is strictly below one: $1/(1+r) < 1$ for TO/LE and $1/[1+(1-\tau)r] < 1$ for MH.
4. The feasible action set is nonempty at every state.
5. TO pricing $\Delta(k_{\text{choice}}, b_{\text{old}}, z_{\text{old}})$ is well-defined for every grid point, including the $b = 0$ case.
6. The resulting LP is feasible and bounded below.

Under these conditions, the Bellman operator for the finite discretized model is a contraction, and the standard LP formulation with objective $\min \sum_s W(s)$ and constraints $W(s) \geq T_a W(s)$ for all feasible state-action pairs recovers the unique fixed point. If continuous interpolation or auxiliary action searches are added later, the master LP remains valid only if each added Bellman inequality freezes the selected action and interpolation weights as constants.

5.6 Structural Estimation and Model Comparison

5.6.1 Policy Functions

The model solutions are the optimal policy function and value function. The **optimal policy function** maps state to actions and can be written as:

$$a_t = \varphi(s_t)$$

where the action vector is defined as $a_t \equiv \{k_{t+1}, b_{t+1}, d_{t+1}\}$ and the state vector is $s_t \equiv \{k_t, b_t, z_t\}$.

Because we use a grid-based LP solution, the theoretical policy function is stored as lookup tables over the discrete state grid. For example, at each grid point (k_t, b_t, z_t) , the TO and LE solvers store the optimal next capital k_{t+1} . The policy table is therefore a discrete numerical approximation to the model policy function.

For empirical estimation and model comparison, the primitive theoretical variables are transformed into observable counterparts. The **empirical policy function** is written as

$$y_{it} = P(x_{it}) + u_{it}, \quad E[u_{it} | x_{it}] = 0,$$

where the *observable states* are

$$x_{it} = \left(\log k_{it}, \frac{\pi(k_{it}, z_{it}, \eta_{it})}{k_{it}}, \frac{b_{it}}{k_{it}} \right)$$

and the *observable actions* are

$$y_{it} = \left(\frac{i_{it}}{k_{it}}, \frac{b_{i,t+1}}{k_{i,t+1}}, \frac{d_{i,t+1}}{k_{i,t+1}} \right)$$

For each action variable y_{it}^n with n indexing each element in vector y_{it} , the empirical policy function is estimated semi-parametrically using a series approximation

$$P^n(x_{it}) \approx \sum_{j=1}^J h_j^n p_j(x_{it}),$$

where $p_j(x_{it})$ are basis functions and h_j^n are coefficients estimated from regression:

$$\min_h \sum_{i,t} \left(y_{it}^n - \sum_{j=1}^J h_j^n p_j(x_{it}) \right)^2.$$

To reproduce the **model comparison figure**, I follow these steps:

- For a given model (e.g., TO), solve the raw policy function φ , use it to simulate a data of firm-year panel
- From the simulated data, construct observable states and actions (y_{it}, x_{it})

- Fit the series approximation above to obtain empirical policy function $P^n(x_{it})$ and overlay it in twoway plot of y on x

For the real-world data, we can directly fit (y_{it}, x_{it}) to estimate another empirical policy function.

I plot the fitted empirical policy functions $P^n(x_{it})$ as twoway slices of observable actions y_{it} against states x_{it} . Each slice plot fix other states at sample median. These six panels can be directly compared to the Figure 2 and 3 in Nikolov, Schmid, and Steri (2021).

- Investment vs log size
- Future leverage vs current leverage
- Investment vs current leverage
- Payout vs profitability
- Future leverage vs profitability
- Investment vs profitability

TO empirical policy: partial dependence vs binned data mean

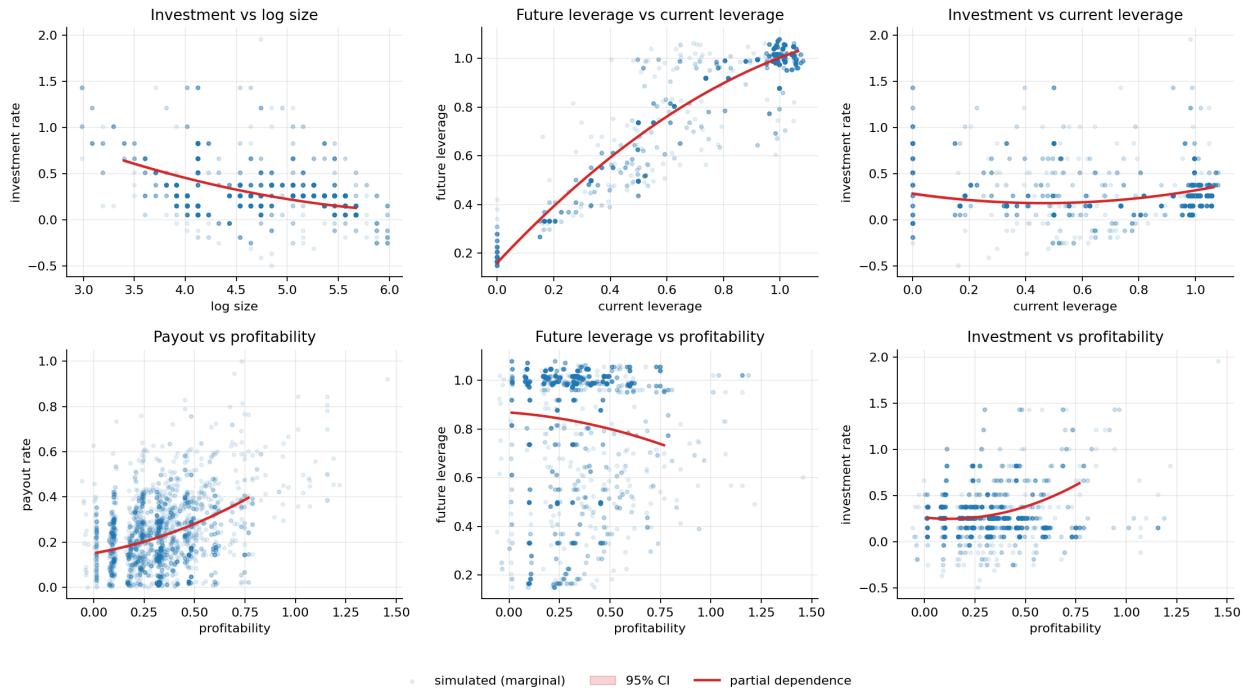


Figure 5.1: TO fitted empirical policy functions (panel simulated from the solved TO model)

The figure shows the empirical policy function fitted on a panel simulated from the solved TO model: each panel plots one observable action against one observable state, with the red line the fitted (partial-dependence) policy and the points the simulated firm-years. The estimated slopes line up with standard theory and with Nikolov21's TO results: investment falls with firm size and is roughly flat-to-declining in leverage, payout and investment both rise with profitability, future leverage is strongly increasing in current leverage (leverage is persistent), and future leverage falls with profitability (more profitable firms lever less). Because these signs match the theoretical mechanisms and the patterns reported for the TO model in Nikolov, Schmid, and Steri (2021), I treat this as a successful reproduction of the TO model's policy behavior. The figure is shown here as an illustrative example for the TO model; the same fitted

coefficients are the auxiliary moments used in the indirect inference below.

5.6.2 Indirect inference

The empirical policy function estimated above serves as the auxiliary model. The structural model is not estimated by matching individual firm outcomes one by one. Instead, for a candidate structural parameter vector β , we solve the model, simulate firm panels from the model, estimate the same empirical policy functions on the simulated data, and choose β so that the simulated policy functions are close to the empirical policy functions estimated from real data.

Let $v_{it} \equiv (y_{it}, x_{it})$ denote one observation in the real firm panel, where i indexes firms and t indexes time. The full real-data panel is

$$\mathcal{D}_{\text{data}} \equiv \{v_{it}\}_{i,t}.$$

For a candidate structural parameter vector β , let

$$v_{it}^{(r)}(\beta) \equiv \left(y_{it}^{(r)}(\beta), x_{it}^{(r)}(\beta) \right)$$

Denote one observation in simulated panel r , where $r = 1, \dots, R$ indexes independent simulation replications. The simulated i, t indices are artificial firm and time indices generated by the model. They are used only to construct a simulated panel with the same observable variables as the real data. The full simulated panel in replication r is

$$\mathcal{D}_{\text{sim}}^{(r)}(\beta) \equiv \{v_{it}^{(r)}(\beta)\}_{i,t}.$$

The vector β contains the structural parameters estimated by indirect inference, including technology parameters, shock-process parameters, adjustment costs, and the model-specific financial-friction parameter. In the paper, the main estimated parameters include

$$(\alpha, f, \rho_z, \sigma_z, \delta, \psi, \eta),$$

and the friction parameter is model-specific: ξ (TO), θ (LE), and λ (MH). Some parameters, such as r, τ, κ , are calibrated and fixed outside the inference.

For any panel dataset $\mathcal{D}$, define $h(\mathcal{D})$ as the vector collecting all estimated coefficients from the empirical policy-function regressions. In our notation, $h(\mathcal{D})$ stacks the coefficients h_j^n across all action variables n and all basis functions j . Therefore, $h(\mathcal{D})$ summarizes the estimated mapping from observable states x_{it} to observable actions y_{it} in that panel.

The real-data auxiliary coefficient vector is $h_{\text{data}} = h(\mathcal{D}_{\text{data}})$. For candidate parameter vector β , the simulated auxiliary coefficient vector is the average across simulated panels:

$$h_{\text{sim}}(\beta) = \frac{1}{R} \sum_{r=1}^R h\left(\mathcal{D}_{\text{sim}}^{(r)}(\beta)\right).$$

The indirect-inference moment is the difference between the empirical auxiliary coefficients and the model-implied auxiliary coefficients:

$$g(\beta) = h_{\text{data}} - h_{\text{sim}}(\beta).$$

The structural estimator chooses β to minimize the weighted distance between these two coefficient vectors:

$$\beta^* = \arg \min_{\beta} g(\beta)' W g(\beta),$$

where W is a positive definite weighting matrix. The paper's key idea is that a model fits well if, for some β , its simulated investment, leverage, and payout policies generate auxiliary policy-function coefficients close to those estimated from the real firm panel.

The estimation loop is:

1. Choose a candidate parameter vector β .
2. Solve the structural model by LP and recover the policy lookup tables.
3. Simulate R firm panels using the recovered policy functions and shock processes.
4. For each simulated panel, transform primitive model variables into the same observable variables $x_{it}^{(r)}(\beta)$ and $y_{it}^{(r)}(\beta)$ used in the real data.
5. Estimate the same semi-parametric empirical policy-function regressions on each simulated panel.
6. Compute $h_{\text{sim}}(\beta)$ by averaging the simulated coefficient vectors across R replications.
7. Compute

$$g(\beta) = h_{\text{data}} - h_{\text{sim}}(\beta).$$

8. Set W and search over β to minimize

$$g(\beta)' W g(\beta).$$

The same indirect-inference procedure is applied separately to TO, LE, and MH. Because the three models share the same observable policy variables even though their internal financial frictions differ, their fit can be compared by asking which model generates simulated policy-function coefficients closest to the empirical coefficients.

Variance estimation To construct

5.6.3 Data and Sample Construction

I construct a sample of listed companies in Hong Kong between 1999 and 2024 from the [Compustat Global database on Wharton Research Data Services \(WRDS\)](#) subscribed by UBC library.

Sample construction. I drop firms in utilities and financial services sector, and those that are highly unbalanced or with key variables missing. The final sample consists of 2,080 firm-years observations, with 116 firms in a balanced panel. I winsorized the top and bottom 1%. When constructing the moment conditions I fit the empirical policy functions on the raw observables and match only their slope and curvature coefficients, dropping each regression's intercept. I drop the intercept because the model and the

data differ in level mainly through the deflator (the model normalizes by capital while the data normalizes by total assets), a gap no structural parameter can close, so leaving the intercept out absorbs that level offset and lets the estimator target the comparable shape of the policy functions.

The table below summarizes the key observables used to fit the empirical policy function and to construct the moment conditions for inference. Note that leverage and future leverage are measured gross, $(DLTT + DLC)/AT$, and are therefore non-negative, which matches the model's $b/k \geq 0$ and is the support the structural policies can reproduce. For Hong Kong firms, market value is not available so I do not use market-to-book for estimation.

Variable	Role in Policy Function	Theory	Data (Compustat)
Investment rate	Action	$\frac{k_{t+1} - (1-\delta)k_t}{k_t}$	CAPX/AT
Book leverage (current)	State	b_t/k_t	$(DLTT+DLC)/AT$
Future leverage	Action	b_{t+1}/k_{t+1}	$(DLTT+DLC)/AT$
Profitability	State	$\frac{\pi(k_t, z_t, \eta_t)}{k_t} = \frac{(z_t + \eta_t)k_t^\alpha - f}{k_t}$	OIBDP/AT
Dividends payouts	Action	$\frac{d_t}{k_t}$	$(DVT+PSSTKC-PSTKRV)/AT$
Firm size (log)	State	$\log k_t$	$\log(PPENT)$
Market-to-book	State	W_t/k_t	$(DLTT+DLC+PRCCF \times CHSO)/AT$

5.6.3.1 Sample construction summary

Step	Firm-years	Firms
Raw quarterly records	21,706	312
Fiscal-Q4 (annual) records	5,942	311
HKD currency	4,866	242
Industrial, consolidated, standard format	2,561	131
Exclude financials and utilities	2,339	122
Positive assets and PP&E	2,333	122
Finite flow variables	2,205	122
Has next-period leverage	2,083	119
At least 3 firm-years per firm	2,080	116

The binding cuts are the annual (fiscal-Q4) filter and the industrial/consolidated/standard-format requirement, which together account for almost all of the attrition from 21,706 raw records; the sector exclusions and the positivity/finiteness screens drop comparatively few observations. The final panel is 2,080 firm-years for 116 firms and is close to balanced (mean 18, median 20 annual observations per firm).

5.6.3.2 Missing data and imputation

Debt and payout items are frequently missing in Compustat Global: long-term debt (DLTT) is missing for about 25% of firm-years, current debt (DLC) for 12%, dividends (DVT) for 46%, and repurchases (PRSTKC) for 88%. These fields are imputed to zero, interpreted as no debt or no payout, and flagged, so the corresponding leverage and payout values for those firm-years are zero by construction. The core fields (assets, PP&E, profitability, investment) are essentially complete.

5.6.3.3 Variable distributions

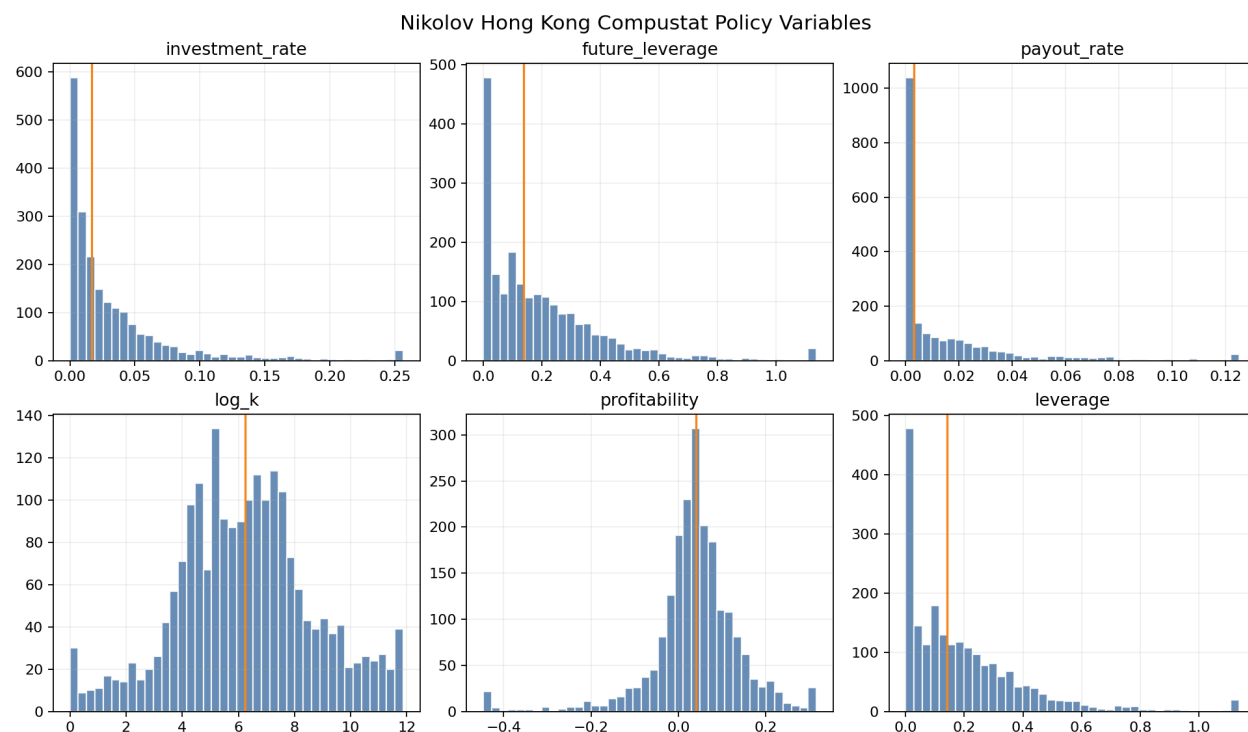


Figure 5.2: Hong Kong Compustat policy-variable distributions

Investment, payout, and leverage are right-skewed with a large mass at zero: many Hong Kong firms invest little, pay nothing, and carry little or no debt. Profitability is roughly symmetric around a low mean and is occasionally negative. Firm size (log capital) is widely dispersed. Orange lines mark medians; the small bars at the right edges are the 1% winsorization caps.

5.6.3.4 Summary Statistics

Variable	Mean	SD	p1	Median	p99
investment_rate	0.033	0.045	0.000	0.017	0.253
future_leverage	0.196	0.207	0.000	0.139	1.097
payout_rate	0.015	0.024	0.000	0.003	0.125
log_k	6.258	2.463	0.052	6.253	11.850
profitability	0.039	0.105	-0.438	0.040	0.316
leverage	0.194	0.204	0.000	0.141	1.090

A typical firm invests roughly 2 to 3% of capital per year (median 1.7%), holds modest leverage (median 14%), and pays out little (median 0.3%); profitability is low (median 4%) and can be negative.

5.6.4 Estimation Results

I now provide results from solving and estimating the three models. The main purpose is to show end-to-end model solve and estimation pipeline. Limited by the CPU capacity (M1), I use smaller grid density than

the paper's specification ($[k, b, z] = [21, 17, 5]$) so the results should be interpreted as rough preliminary estimates. Future work would require better CPU or improving the efficiency of LP solver by adding features such as constraint generation.

5.6.4.1 Trade-off (TO) Model

The table reports the parameter estimates from indirect inference on the TO Model, solved by LP method on grid with density ($[k, b, z] = [15, 12, 5]$). To conduct the minimization, I use Nelder-Mead as global method to search for the optimal parameter vector. The system is over-identified with 27 moments and 8 parameters. I use identity weight matrix to construct the standard SMM variance. Unfortunately, because simulated moment covariance is near-singular, I'm unable to conduct a valid over-identification (J) test for this run. The full estimation took about 2 hours on Apple M1. Reference Estimate reported in the second column are the initiation value, calibrated to the estimates on Large US public-listed firms reported by Nikolov, Schmid, and Steri (2021).

One notable estimate is the large capital adjustment cost $\hat{\psi} = 1.036$, which is not obviously realistic. This is consistent with the fact I show earlier that the Compustat-HK firms on average have much lower investment rate compared with Compustat-US firms. The i/k moments are poorly matched (see appendix) and worth examining whether the current measurement for investment need to be refined if, for example, the variable are defined and constructed differently in Hong Kong due to different regulation.

Parameter	Ref Estimate	Estimate $\hat{\beta}$	SE
α (capital share)	0.75	0.613	0.089
f (fixed cost)	0.70	0.631	0.231
ρ_z (shock persistence)	0.80	0.604	0.029
σ_z (shock volatility)	0.30	0.296	0.073
δ (depreciation)	0.15	0.175	0.072
ψ (adjustment cost)	0.15	1.036	0.024
$\bar{\eta}$ (i.i.d. shock size)	0.30	0.265	0.116
ξ (recovery rate)	0.60	0.734	0.134

The figure below reproduces the empirical policy comparison plot from Nikolov21's Figure 2 and 3. Each panel is a partial-dependence slice: it varies one observable state, holds the others at their sample median, and plots the fitted empirical policy. The black line is fit on the real Hong Kong panel and the red line is fit on a panel simulated from the solved TO model at the estimated parameters.

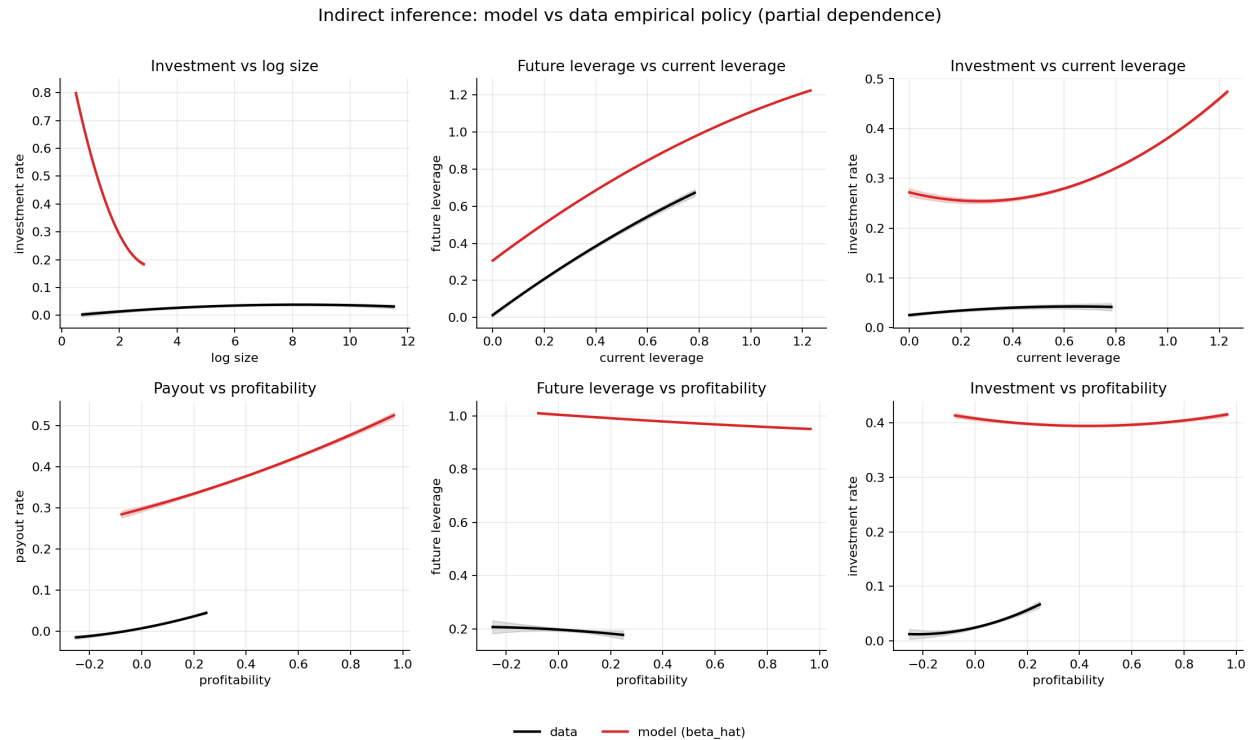


Figure 5.3: TO indirect-inference policy overlay: the empirical policy fit on the real Hong Kong panel (black) versus the same regressions refit on a panel simulated from the solved TO model at the estimated parameters (red)

Because intercepts are dropped, a good match means the two lines share the same slope and curvature, not the same level. Here the lines clearly diverge: the simulated policy sits well above the data in most panels, and the shapes agree only in part (for instance, future leverage rises with current leverage in both, but the model line is much steeper and higher). So the estimator could not bring the model's policy close to the data from the current preliminary results.

Why the fit is poor. I rank the likely causes from most to least actionable.

1. **Model solve quality (most likely the main cause).** The reported run solves the TO model on a coarse grid ($k = 15$, $b = 12$, $z = 5$) for tractability, because indirect inference re-solves the model once per candidate parameter vector, hundreds of times in total. A coarse grid turns the policy into a crude step function over a handful of states, so the simulated panel carries little granular variation and the empirical policy refit on it is shaped by discretization rather than by economics. This is plausibly the dominant problem. The clear next step is to refine the grid (denser capital, debt, and shock points), which lowers grid approximation error and lets the simulation reproduce the smooth policy variation the regressions are trying to recover. Only once the solve is accurate can we trust the simulated moments.
2. **Observable and normalization disagreement.** Even with an accurate solve, the model and the data describe firm flows on different scales. The model normalizes every flow by capital k , while the Compustat observables normalize by total assets, so the two policies can differ in level for reasons no structural parameter can fix. I already drop each regression's intercept to absorb this level offset and match only slope and curvature. A more careful version is the firm-fixed-effects adjustment used in

Nikolov, Schmid, and Steri (2021): estimate firm fixed effects from the real panel, drop the intercept of the simulated empirical policy, then add the real-data firm effects back onto the simulated policy. The purpose is to strip out persistent, firm-specific level differences (size, accounting deflator, unmodeled heterogeneity) that the structural model was never meant to explain, so the estimator compares the part of the policy that is actually comparable, its shape. The problem it solves is keeping a nuisance level gap from looking like a structural misfit.

3. **A genuine model rejection (only after 1 and 2 are ruled out).** If the solve is accurate, the observables are aligned, and the simulated policy still cannot approach the data, then the fit is truly poor and the model is rejected on its own terms. Confirming this needs the formal model-fit test from Nikolov, Schmid, and Steri (2021), the over-identification test on the moment gap. That test is expensive: building its variance requires re-solving the model many times to estimate the moment covariance and the sensitivity of the moments to the parameters. The current run could not even form a valid J statistic (identity weighting plus a near-singular moment covariance), so a formal rejection is not yet warranted. The honest reading is that causes 1 and 2 must be cleared first, and only then is the expensive formal test worth running.

Appendix A

Solution Methods to Dynamic Models

Supporting material for Chapter 1 (Solving Dynamic Models).

A.1 Value and Policy Function Iterations

Value function iteration (VFI) and policy function iteration (PFI) are the most widely used methods to solve discrete-time dynamic programming problems. In their simplest form, these methods discretize the continuous state space into a finite grid $\mathcal{S}_{\text{grid}}$ and iterate on the Bellman equation until convergence.

VFI exploits the property that the Bellman operator is a contraction mapping with unique fixed point V^* , so repeatedly applying the operator to any initial V_0 converges to V^* . Each iteration applies a single Bellman backup across all grid points and selects the maximizing action, but does not explicitly maintain a policy until convergence.

PFI separates each iteration into two steps: (i) **policy evaluation**, which solves for the exact on-policy value function V^{π_j} given a fixed policy π_j , and (ii) **policy improvement**, which updates the policy by maximizing the Bellman right-hand side using V^{π_j} . The Policy Improvement Theorem guarantees $V^{\pi_{j+1}}(s) \geq V^{\pi_j}(s)$ for all s . PFI typically converges in fewer outer iterations than VFI because each iteration performs exact policy evaluation rather than a single Bellman backup, though each iteration is more expensive.

A.1.1 Algorithm: Value Function Iteration (VFI)

Input: Grid $\mathcal{S}_{\text{grid}}$, reward r , dynamics f , discount γ , tolerance $\delta > 0$

Output: V^*, π^*

1. Initialize $V_0(s) = 0$ for all $s \in \mathcal{S}_{\text{grid}}$
2. **For** $j = 0, 1, 2, \dots$ **do**
3. **For** each $s \in \mathcal{S}_{\text{grid}}$ **do**
4. $V_{j+1}(s) = \max_a \{r(s, a) + \gamma \mathbb{E}_\epsilon [V_j(f(s, a, \epsilon))]\}$
5. **End for**
6. **If** $|V_{j+1} - V_j|_\infty < \delta$ **then break**
7. **End for**
8. $\pi^*(s) = \arg \max_a \{r(s, a) + \gamma, \mathbb{E}_\epsilon [V^*(f(s, a, \epsilon))]\}$ for all $s \in \mathcal{S}_{\text{grid}}$
9. **Return** V^*, π^*

A.1.2 Algorithm: Policy Function Iteration (PFI)

Input: Grid $\mathcal{S}_{\text{grid}}$, reward r , dynamics f , discount γ , tolerance $\delta > 0$

Output: V^*, π^*

1. Initialize $\pi_0(s)$ arbitrarily for all $s \in \mathcal{S}_{\text{grid}}$
2. **For** $j = 0, 1, 2, \dots$ **do**
3. **Policy evaluation:** Solve for V^{π_j} satisfying
4. $V^{\pi_j}(s) = r(s, \pi_j(s)) + \gamma \mathbb{E}_\epsilon [V^{\pi_j}(f(s, \pi_j(s), \epsilon))] , \quad \forall s \in \mathcal{S}_{\text{grid}}$
5. **Policy improvement:** Update policy
6. $\pi_{j+1}(s) = \arg \max_a \{r(s, a) + \gamma \mathbb{E}_\epsilon [V^{\pi_j}(f(s, a, \epsilon))]\} , \quad \forall s \in \mathcal{S}_{\text{grid}}$
7. **If** $|\pi_{j+1} - \pi_j|_\infty < \delta$ **then break**
8. **End for**
9. $V^* = V^{\pi_j}, \pi^* = \pi_j$
10. **Return** V^*, π^*

The common limitation of both methods is the reliance on discretization: the computation cost scales with the number of grid points, which grows exponentially in the dimension of the state space, i.e., the so-called “curse of dimensionality”.

A.2 Lifetime Reward Maximization

The Lifetime Reward Maximization (LRM) method directly maximizes expected discounted lifetime rewards by simulating trajectories under the current policy. Given initial state s_0 and a shock sequence $\{\epsilon_1, \dots, \epsilon_T\}$, the policy π_θ generates a trajectory $\tau = (s_0, a_0, s_1, a_1, \dots, s_{T-1}, a_{T-1}, s_T)$ where $a_t = \pi_\theta(s_t)$ and $s_{t+1} = f(s_t, a_t, \epsilon_{t+1})$.

True objective. The infinite-horizon value under policy π_θ starting from s_0 is:

$$V^\pi(s_0) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi_\theta(s_t)) \right]$$

Splitting at a finite horizon T gives an exact decomposition:

$$V^\pi(s_0) = \mathbb{E} \left[\sum_{t=0}^{T-1} \gamma^t r(s_t, \pi_\theta(s_t)) \right] + \gamma^T \mathbb{E} [V^\pi(s_T)]$$

where the second term is the discounted expected continuation value from the terminal state s_T onward.

Truncated objective. Maliar, Maliar, and Winant (2021) approximate the true objective by dropping the continuation term, setting $\hat{V}^{\text{term}}(s_T) = 0$:

$$\max_{\theta} J_T(\theta) = \mathbb{E}_{(s_0, \epsilon_1, \dots, \epsilon_T)} \left[\sum_{t=0}^{T-1} \gamma^t r(s_t, \pi_\theta(s_t)) \right]$$

This is valid when T is large enough that $\gamma^T V^\pi(s_T) \approx 0$. However, the discount factor contracts this term slowly: with $\gamma = 0.96$, keeping the truncation bias below 1% of the true value requires $T \geq \lceil \log(0.01) / \log(0.96) \rceil = 113$ periods. BPTT through such a long chain is computationally prohibitive because gradient memory scales linearly in T , and vanishing or exploding gradients compound across the chain.

Back-propagation through time (BPTT) The core mechanics of the LRM method is known as BPTT in machine learning. The idea is to exploit the end-to-end differentiability of the reward function r and the state transition function f and obtain the precise gradient flow $\nabla_{\theta} J(\theta)$ backward through the entire trajectory generated by π_{θ} to improve the policy. The main issue of BPTT is the trade-off between truncation bias and computational cost:

- When T is large, computation is slow and expensive, and the gradient may blow up
- When T is moderate-to-small, policy π_{θ} suffers from nontrivial truncation bias

True value function. The true continuation value from the terminal state s_T onward is:

$$V^{\pi}(s_T) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_{T+t}, \pi(s_{T+t})) \mid s_T \right]$$

which integrates over all future shock realizations and the policy's dynamic response to them.

Terminal value approximation. The terminal value approximation formula approximates it by exploiting the endogenous-exogenous state decomposition and using a *deterministic* perpetuity following optimal policy at mean value and realized shocks. This approximation exploits the maximum information of the model's structure, but there remains an approximation error gap that cannot be closed. Specifically, I replace the terminal value using a geometric perpetuity:

$$\hat{V}^{\text{term}}(s_T^{\text{endo}}) = \frac{r(\bar{s}, \bar{a})}{1 - \gamma}$$

where state and action variables are set to

$$\bar{s} = [s^{\text{endo}} \mid \bar{s}^{\text{exo}}], \quad \bar{a} = \bar{a}(s^{\text{endo}})$$

where $\bar{s}^{\text{exo}} = \mathbb{E}[s^{\text{exo}}]$ is the stationary mean of the exogenous process, and $\bar{a}(s_T^{\text{endo}})$ is the action satisfying $f^{\text{endo}}(s_T^{\text{endo}}, \bar{a}) = s_T^{\text{endo}}$, the steady-state action that holds the endogenous state constant. Both are functions of s_T^{endo} alone and are model constants provided by the environment. The approximation replaces the stochastic future with a deterministic steady state in which the exogenous state is frozen at its mean and the agent repeats the stationary action forever. The continuation then reduces to a geometric perpetuity. During training, this is evaluated at s_T^{endo} , the terminal endogenous state obtained by rolling out the current policy π_{θ} for T steps. The formula is a fixed function of s_T^{endo} that does not depend on policy network parameters θ .

Using the basic investment model as an concrete example: exogenous state variable is AR(1) shock $\bar{z} = \mathbb{E}[z] = \mu$, endogenous state variable is set to steady state capital k_{ss} , and action variable at steady state is $I = \delta \cdot k_{ss}$. Because k_{ss} is unknown, the best we can do is to rollout T periods using π_{θ} and assume that $k_T \approx k_{ss}$. Early in training, the policy has not converged, so the rollout may not reach the steady state even when T is adequate. The terminal value is then a rough approximation. But as the policy improves, rollout trajectories increasingly reach the neighborhood of the steady state, making the terminal value more accurate, which in turn provides a better gradient signal.

Approximation error. The error of the perpetuity relative to the true continuation is:

$$\hat{V}^{\text{term}}(s_T^{\text{endo}}) - V^{\pi}(s_T) = \frac{r(\bar{s}, \bar{a})}{1 - \gamma} - \mathbb{E}_{\epsilon} \left[\sum_{t=0}^{\infty} \gamma^t r(s_{T+t}, \pi(s_{T+t})) \mid s_T \right]$$

To understand its magnitude, consider the idealized case where s_T^{endo} is at the optimal steady state. The perpetuity gives the reward at the deterministic steady state, while V^π accounts for the agent's optimal response to future stochastic shocks. By the envelope theorem, the first-order effect of small shocks on the value function vanishes: the agent is already optimizing, so marginal perturbations in the exogenous state are absorbed by optimal policy adjustment. The approximation error is therefore **second-order in the exogenous volatility** $O(\sigma_\epsilon^2)$ and it has two components:

1. **Jensen's correction.** The value function is generally concave in the exogenous state. Replacing the stochastic s^{exo} with its mean overstates the value.
2. **Precautionary motive.** A firm facing adjustment costs benefits from the *option to respond* to future shocks. The perpetuity assumes a fixed action forever, which ignores this option value.

It is important to note that further reducing the approximation error would require explicitly learning V^π using a value network. At that point the algorithm becomes a critic-based method which is a fundamentally different algorithm. This is exactly the solution method of Short-Horizon Actor Critic that I explore in later section.

Why this matters? Without a terminal value $\hat{V}^{\text{term}} \approx V^\pi(s_T)$, the learned policy π_θ has a nontrivial truncation bias. For example, in most economic/finance models, the optimal investment policy would be systematically under-estimated in the absence of $\hat{V}^{\text{term}}$ because the long-run return to investment is ignored.

Loss function. The SGD loss with terminal value correction, evaluated over a mini-batch $\mathcal{B}$:

$$J(\theta) = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \left(\sum_{t=0}^{T-1} \gamma^t \cdot r(s_{it}, \pi_\theta(s_{it})) + \gamma^T \hat{V}^{\text{term}}(s_{iT}^{\text{endo}}) \right)$$

Setting $\hat{V}^{\text{term}} = 0$ recovers the truncated objective of Maliar, Maliar, and Winant (2021). In the BPTT computation, $\hat{V}^{\text{term}}$ should be differentiable with respect to s_T^{endo} so that gradients prevent the policy from de-investing near the horizon, but should not route gradients through the policy network at the terminal step to avoid $1/(1-\gamma)$ gradient amplification through the BPTT chain. The gap between $\hat{V}^{\text{term}}$ and the true continuation V^π arises because the perpetuity ignores future exogenous volatility and the firm's dynamic response to it.

A.2.1 Algorithm: Lifetime Reward Maximization

Input: Policy network π_θ , dynamics f , reward r , discount γ , horizon T , terminal value $\hat{V}^{\text{term}}$, learning rate η , convergence rule $\text{CONVERGED}(\theta, j)$

Output: Trained policy π_θ^*

1. Initialize policy parameters θ
2. **For** $j = 0, 1, 2, \dots$ **do**
3. Sample mini-batch $\mathcal{B}$ consisting of initial states $\{s_0\}_i$ and shock sequences $\{\epsilon_1, \dots, \epsilon_T\}_i$
4. **For** each observation $i \in \mathcal{B}$, rollout trajectory:
5. **For** $t = 0, \dots, T-1$: simulate $a_{i,t} = \pi_\theta(s_{i,t})$ and $s_{i,t+1} = f(s_{i,t}, a_{i,t}, \epsilon_{i,t+1})$
6. Compute loss: $J(\theta) = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \left(\sum_{t=0}^{T-1} \gamma^t \cdot r(s_{it}, \pi_\theta(s_{it})) + \gamma^T \hat{V}^{\text{term}}(s_{iT}^{\text{endo}}) \right)$
7. SGD update: $\theta \leftarrow \theta - \eta \cdot \nabla_\theta J(\theta)$
8. **If** $\text{CONVERGED}(\theta, j)$ **then break**
9. **End for**
10. **Return** π_{θ^*}

A.3 Euler Residual Minimization

The ER method minimizes the Euler equation errors that characterize optimality. It enforces an intertemporal first-order necessary condition between (s, a) and (s', a') at each observation independently.

Euler equation. At the optimum, the policy π_θ satisfies:

$$\mathbb{E}_\varepsilon [F(s, \pi_\theta(s), s', \pi_\theta(s'))] = 0$$

where $F : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the Euler residual function derived analytically from the first-order conditions of the Bellman equation, and $s' = f(s, \pi_\theta(s), \varepsilon)$ is computed using the state transition function f .

ERM is a one-step method: each observation is a single-step transition $(s, f(s, \cdot), s', f(s', \cdot))$, and the loss is computed independently per observation. ERM is significantly faster than LRM and SHAC because it does not require rolling out a full trajectory and BPTT. The optimality condition is given analytically by the Euler equation, and ERM directly search for θ^* that directly enforces it.

Target network. Both Maliar, Maliar, and Winant (2021) and Fernández-Villaverde (2025) suggest using a single policy network inside the loss function. However, computing $\pi_\theta(s')$ introduces a recursive dependency: the gradient of θ flows through both the current policy $\pi_\theta(s)$ and the next-period policy $\pi_\theta(s')$, creating a moving-target problem that prevents convergence. My implementation introduces a separate target network $\pi_{\theta-}$ for the next-period action, updated via Polyak averaging.

Loss function The objective is to minimize the squared Euler residual. Following Maliar, Maliar, and Winant (2021), I use the Monte Carlo cross product estimator with two independent shock draws for unbiased estimation of the squared expectation. Specifically, I first draw two random iid $(\varepsilon_1, \varepsilon_2)$, and use the AR(1) transition function to construct (s'_1, s'_2) :

$$\mathcal{L}_\theta = \frac{1}{\mathcal{B}} \sum_{i \in \mathcal{B}} F(s_i, a_i, s'_{i,1}, a'_{i,1}) \cdot F(s_i, a_i, s'_{i,2}, a'_{i,2})$$

where:

- $a_i = \pi_\theta(s_i)$ is current action from the trainable policy network (with gradient)
- $s'_{i,m} = f(s_i, \pi_\theta(s_i), \varepsilon_{i,m})$ are next states under iid shock draw $m \in \{1, 2\}$
- $a'_{i,m} = \pi_{\theta-}(s'_{i,m})$ is next action from the target policy network (no gradient)
- $\mathcal{B}$ denotes mini-batches and i denotes observation

Why do we need a target network? The original method in Maliar, Maliar, and Winant (2021) and Fernández-Villaverde (2025) uses a single policy network π_θ for both current-period and next-period Euler equation. This creates a **moving target problem**. In practice, this leads to oscillatory or divergent dynamics: the optimizer cannot further minimize the Euler equation error toward zero because every step that adjusts the Euler LHS also shifts the Euler RHS by a similar magnitude.

The problem with a single network. Take the basic investment model as an concrete example, the Euler residual has the structure:

$$F = 1 - \frac{1}{1+r} \cdot \frac{m(\pi_\theta(s'))}{\chi(\pi_\theta(s))}$$

where χ is the marginal cost of investment today (depends on the current action $a = \pi_\theta(s)$) and m is the marginal benefit of capital tomorrow (depends on the next-period action $a' = \pi_\theta(s')$). At optimum, $F = 0$: marginal cost equals discounted marginal benefit. When a single network π_θ supplies both a and a' , any update to θ simultaneously moves both sides of the equation. Consider a gradient step that increases investment everywhere:

- **Today** ($a = \pi_\theta(s)$): higher investment raises the marginal cost χ : the denominator increases.
- **Tomorrow** ($a' = \pi_\theta(s')$): higher investment also raises $\chi' = 1 + \partial\psi'/\partial k''$ via the next-period adjustment cost: the numerator m increases too.

Both sides of the ratio m/χ shift in response to the same parameter update. The gradient points toward the correct equilibrium, but the target it is aiming at (the RHS) moves by a comparable amount at each step.

This is not unique to investment models. In any Euler equation $MC(a) = \beta \cdot E[MB(a')]$, the marginal quantities on both sides are evaluated under the *same* policy. A parameter change that reduces the residual at the current θ does not guarantee a smaller residual at $\theta + \Delta\theta$, because the next-period side has shifted.

How the target network resolves this. The target network π_{θ^-} provides a fixed reference for the next-period action:

- $a = \pi_\theta(s)$: gradients flow through the current policy.
- $a' = \pi_{\theta^-}(s')$: **gradients stopped**; weights are frozen for this step.

Now the marginal benefit m is computed from π_{θ^-} , which moves slower per step via Polyak averaging. The optimizer sees a near-stationary target: it adjusts the current-period action to match the slowly-moving next-period reference, and the reference gradually tracks the improving policy. This converts the unstable simultaneous update into a stable fixed-point iteration.

A.3.1 Algorithm: Euler Residual Minimization

Input: Policy network π_θ , target policy π_{θ^-} , known state transition function f , analytical formula of Euler equation error F , flattened dataset $\{(s_i, \epsilon_{i,1}, \epsilon_{i,2})_{i=1}^N\}$, mini-batch B , Polyak rate τ_{polyak} , learning rate η , convergence rule $\text{CONVERGED}(\theta, j)$

Output: Trained policy network π_{θ^*}

1. Initialize policy parameters θ and create target network $\theta^- \leftarrow \theta$
2. Fit a shared StaticNormalizer on the full training dataset (identical statistics, separate copy per network)
3. **For** $j = 0, 1, 2, \dots$ **do**
4. Sample mini-batch $\mathcal{B} = \{(s_i, \epsilon_{i,1}, \epsilon_{i,2})_{i=1}^N\}$ from the flattened dataset
5. Use current policy θ to compute action: $a_i = \pi_\theta(s_i)$
6. **For** $m \in \{1, 2\}$: Rollout $s'_{i,m} = f(s_i, \pi_\theta(s_i), \epsilon_{i,m})$ using current policy weights θ
7. **For** $m \in \{1, 2\}$: Rollout $a'_{i,m} = \pi_{\theta^-}(s'_{i,m})$ using target policy weights θ^- (no gradient)
8. **For** $m \in \{1, 2\}$: Compute Euler residuals $F_{i,m} = F(s_i, a_i, s'_{i,m}, a'_{i,m})$
9. Compute loss function: $\mathcal{L}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} F_{i,m=1} \cdot F_{i,m=2}$
10. Update current policy: $\theta \leftarrow \theta - \eta \cdot \nabla_\theta \mathcal{L}(\theta)$
11. Update target policy: $\theta^- \leftarrow \tau_{\text{polyak}} \cdot \theta^- + (1 - \tau_{\text{polyak}}) \cdot \theta$
12. **If** $\text{CONVERGED}(\theta, j)$ **then break**
13. **End for**
14. **Return** π_{θ^*}

A.4 Short-Horizon Actor-Critic (SHAC)

SHAC solves infinite-horizon dynamic programming problems by combining short-horizon backpropagation through differentiable dynamics with a learned value function network. It builds on a modern RL algorithm developed by Xu et al. (2022). I adopt the core structure of windowed actor BPTT with on-policy continuation across windows, but replacing the critic update with a one-step Bellman target – similar to the Deep Deterministic Policy Gradient (DDPG) method – to improve stability in economic environments.

Core idea. The full T -step trajectory is divided into consecutive windows of length h . Within each window, the actor loss backpropagates through h exact dynamics steps, and a value function V bootstraps the continuation beyond the window boundary. Between windows, the endogenous state carries forward (detached via `stop_gradient`) so the trajectory remains on-policy. This avoids the gradient explosion/vanishing of full-trajectory BPTT while retaining exact policy gradients through known, differentiable dynamics.

Reward and Bellman normalization. Unlike in standard RL environments, economic models typically have large reward and value scales, yet SHAC’s default hyperparameters assume $O(1)$. To bring values into the right range and stabilize training, I rescale every reward by $\lambda_r = 1/|V^*|$ so that the critic learns values of $O(1)$. To obtain V^* , I use the environment’s baseline steady-state value that can be obtained analytically. For example, in basic investment model I use the first-order approximation value $V^* = \hat{V}^{\text{term}}$ as described in the [LRM appendix](#). The scaling is applied uniformly to r in both the actor loss and the critic Bellman target, which makes the rescaling mathematically equivalent to the unscaled algorithm: the critic learns $\lambda_r V^\pi$ in place of V^π , and the optimal policy is unchanged. Because λ_r is a numerical preconditioning factor (i.e., multiplied by a constant) rather than part of the algorithm’s logic, I omit it from the algorithm summary below.

Why this variant? The original SHAC uses an on-policy TD- λ critic trained on rewards from the actor’s own rollout. In economic environments with large reward scales, this creates a positive feedback loop: the critic overfits to the actor’s trajectory, the actor exploits the critic’s overestimates, and training diverges. Our variant decouples the critic from the current actor by using a 1-step Bellman target with separate target networks for both policy and value.

This method uses four separate NNs to separately approximate the policy and value functions:

Network	Notation	Output head	Role
Policy (actor)	π_θ	Affine + clip (see below)	Maps full state s to action a
Value (critic)	V_ϕ	Dense(1), linear	Maps full state s to scalar value
Target policy	$\bar{\pi}_\theta$	Same as π_θ	Polyak-averaged copy of π_θ ; separate weights
Target value	$\bar{V}_\phi$	Same as V_ϕ	Polyak-averaged copy of V_ϕ ; separate weights

A.4.1 Algorithm: Short-Horizon Actor-Critic

Input: Actor π_θ , target actor π_{θ^-} , critic V_ϕ , target critic V_{ϕ^-} , dynamics f , reward r , discount γ , mini-batch size B , total horizon T , window length h (with $T \bmod h = 0$), critic steps per window n_{critic} , Polyak rate τ_{polyak} , learning rates η_θ, η_ϕ , convergence rule $\text{CONVERGED}(\theta, \phi, j)$

Output: Trained actor π_θ^* and critic V_ϕ^*

1. Initialize θ, ϕ and target networks $\theta^- \leftarrow \theta, \phi^- \leftarrow \phi$; fit a shared StaticNormalizer on the full training dataset.
2. **For** training steps $j = 0, 1, 2, \dots$ **do**
3. Sample mini-batch $\mathcal{B}$ of trajectories with initial states $\{s_{i,0}\}$ and pre-simulated shock paths $\{\varepsilon_{i,1:T}\}$; set the rollout state $s_i \leftarrow s_{i,0}$.
4. **For** windows $w = 0, 1, \dots, T/h - 1$ **do** (with start time $t_0 = wh$):
5. **(a) Actor BPTT.** Set $s_{i,0} \leftarrow s_i$ (detached) and unroll h steps under the current actor:

$$a_{i,\ell} = \pi_\theta(s_{i,\ell}), \quad s_{i,\ell+1} = f(s_{i,\ell}, a_{i,\ell}, \varepsilon_{i,t_0+\ell+1}), \quad \ell = 0, \dots, h-1$$

6. **(b) Actor loss and update** (current critic V_ϕ as terminal bootstrap):

$$\mathcal{L}(\theta) = -\frac{1}{B} \sum_{i \in \mathcal{B}} \left[\sum_{\ell=0}^{h-1} \gamma^\ell r(s_{i,\ell}, a_{i,\ell}) + \gamma^h V_\phi(s_{i,h}) \right], \quad \theta \leftarrow \theta - \eta_\theta \nabla_\theta \mathcal{L}(\theta)$$

Carry $s_i \leftarrow \text{stop_gradient}(s_{i,h})$ into the next window.

7. **(c) Critic 1-step Bellman regression** on the detached set $\mathcal{D}_w = \{(s_{i,\ell}, \varepsilon_{i,t_0+\ell+1}) : i \in \mathcal{B}, \ell = 0, \dots, h-1\}$. Define the Bellman target under the target networks:

$$y(s, \varepsilon) = \text{stop_gradient} \left[r(s, \pi_{\theta^-}(s)) + \gamma V_{\phi^-}(f(s, \pi_{\theta^-}(s), \varepsilon)) \right]$$

For $u = 1, \dots, n_{\text{critic}}$:

$$\mathcal{L}(\phi) = \frac{1}{|\mathcal{D}_w|} \sum_{(s, \varepsilon) \in \mathcal{D}_w} \left(V_\phi(s) - y(s, \varepsilon) \right)^2, \quad \phi \leftarrow \phi - \eta_\phi \nabla_\phi \mathcal{L}(\phi)$$

8. **(d) Polyak update target networks:**

$$\theta^- \leftarrow \tau_{\text{polyak}} \theta^- + (1 - \tau_{\text{polyak}}) \theta, \quad \phi^- \leftarrow \tau_{\text{polyak}} \phi^- + (1 - \tau_{\text{polyak}}) \phi$$

9. **If** $\text{CONVERGED}(\theta, \phi, j)$ **then break**
10. **End for** (windows)
11. **End for** (training steps)
12. **Return** π_θ^*, V_ϕ^*

A.5 Bellman Residual Minimization

The Bellman Residual Minimization (BRM) method jointly trains a policy network π_θ and a value function network V_ϕ to satisfy the Bellman equation. The challenge is that the Bellman equation contains a max operator:

$$V(s) = \max_a \{r(s, a) + \gamma \mathbb{E}_\epsilon [V(s')]\}$$

Rather than solving the inner maximization directly, Maliar, Maliar, and Winant (2021) and Fernández-Villaverde (2025) suggest eliminating the max by adding the first-order necessary condition as auxiliary losses. This turn the loss function into a multitask objective that combines the Bellman residual, the first-order condition (FOC), and other model-specific constraints and optimality conditions with user-specified exogenous weights.

Bellman residual. For a given policy π_θ and value function V_ϕ , define the Bellman equation residual for each observation i :

$$F_{i,m}^{\text{BR}} = V_\phi(s_i) - r(s_i, a_i) - \gamma V_\phi(s'_{i,m})$$

where $a_i = \pi_\theta(s_i)$ and $s'_{i,m} = f(s_i, a_i, \epsilon_{i,m})$ for two Monte Carlo draws $m = 1, 2$. We state that the value function V^π satisfies the **on-policy** Bellman equation if and only if $\mathbb{E}_\epsilon[F^{\text{BR}}] = 0$. Note that this can be satisfied for any arbitrary policy π that is not optimal.

FOC residual. Differentiating the Bellman RHS with respect to the action a yields the necessary condition:

$$F_{i,m}^{\text{FOC}} = \nabla_a r(s_i, a)|_{a=a_i} + \gamma \nabla_{s'} V_\phi(s'_{i,m}) \cdot \nabla_a f(s_i, a, \epsilon_{i,m})|_{a=a_i}$$

The necessary condition for optimality is $\mathbb{E}_\epsilon[F^{\text{FOC}}] = 0$.

Envelope condition residual. Differentiating the Bellman equation with respect to the state s (applying the envelope theorem) gives:

$$F_i^{\text{Env}} = \nabla_s r(s_i, a_i) - \nabla_s V_\phi(s_i)$$

This condition involves no expectation over future shocks, so the loss uses a direct squared residual.

Feasibility constraints (Optional). When the model needs to satisfy feasibility constraints given as:

$$G(\cdot; \theta) \leq 0 \quad \text{and} \quad H(\cdot; \theta) = 0$$

where $G(\cdot)$ and $H(\cdot)$ can be either linear or non-linear functions over states, actions, or state-action pairs. There are different approaches to handle complementary constraints like Kuhn-Tucker (KT) conditions. Maliar, Maliar, and Winant (2021) uses additional NNs to approximate the Lagrangian multiplier on each of the constraint and construct separate loss that measures the empirical violations of the constraint. Let $\mathcal{L}^{\text{IC}}$ and $\mathcal{L}^{\text{EC}}$ denote the loss for the inequality and equality constraints, respectively, and they will be added into the total loss function with exogenous weight.

Total loss with AiO integration:

$$J(\theta, \phi) = \underbrace{\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} F_{i,1}^{\text{BR}} \cdot F_{i,2}^{\text{BR}}}_{\mathcal{L}^{\text{BR}}} + w_1 \underbrace{\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} F_{i,1}^{\text{FOC}} \cdot F_{i,2}^{\text{FOC}}}_{\mathcal{L}^{\text{FOC}}} + w_2 \underbrace{\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} (F_i^{\text{Env}})^2}_{\mathcal{L}^{\text{Env}}} + w_3 \mathcal{L}^{\text{IC}} + w_4 \mathcal{L}^{\text{EC}}$$

where $w_1, w_2 > 0$ are exogenous weights that must be tuned *manually and carefully* because $\mathcal{L}^{\text{BR}}$ is measured in value levels (squared Bellman residual), but $\mathcal{L}^{\text{FOC}}$ and $\mathcal{L}^{\text{Env}}$ are measured in derivatives and feasibility constraints $\mathcal{L}^{\text{IC}}$ and $\mathcal{L}^{\text{EC}}$ are measured in arbitrary units (depending on the model).

Fundamental defects. In practice, the BRM method is extremely sensitive to the choice of exogenous weights and the unit (scale) of each loss. Maliar, Maliar, and Winant (2021) recommend fine tuning of the weights in pre-training to make the magnitude of each loss roughly the same, but they still find this method obviously less accurate than the other methods.

Although pre-training and fine tuning helps, I find a more serious defect of this method that is overlooked in Maliar, Maliar, and Winant (2021): *minimizing the current multi-task objective function does not guarantee convergence to the optimal policy.*

This is because these multiple auxiliary losses are not all serving a shared goal, but instead creates conflicting gradients that lead to incorrect solutions. Only the FOC loss provides the correct gradient signals toward the optimal policy θ , while $\mathcal{L}^{\text{BR}}$ and other auxiliary losses can be flexibly minimized by a set of arbitrary NN weights θ, ϕ . As a result, BRM training often lead to spurious convergence where the joint-loss function is minimized but the learned θ, ϕ are obviously wrong.

This defect is further worsen by two mechanics: (1) FOC loss is much smaller than Bellman error and other losses, so early training typically focuses on minimizing $\mathcal{L}^{\text{BR}}$ and “ignoring” $\mathcal{L}^{\text{FOC}}$, which lead to a self-consistent Bellman for any arbitrary policy weight θ . (2) Bootstrap estimate of V_ϕ is lower than the true V^* due to NN initialization around zero.

Although $\mathcal{L}^{\text{FOC}}$ can provide correct gradient signals for θ , it is not sufficient to ensure the converged $\pi_\theta \approx \pi^*$. In practice, the BRM training usually plateau at a small loss where the NN find an arbitrary pair of (θ, ϕ) that satisfies on-policy Bellman but only weakly satisfies the FOC.

Potential solution: warm-start value network. I find that warm-start the value NN V_ϕ using a supervised regression can help training to learn the correct “shape” of the optimal policy, but the solution remains biased and the training is instable compared with other methods. The idea is use a baseline closed-form $\hat{V}$ as regression label to pre-train V_ϕ , so that in BRM training the initial V_ϕ already captures the correct “shape”, in such case, the algorithm is more likely to converge (but it is still not guaranteed).

Target Network. As in the ER method, the value network $V_\phi(s'_{i,m})$ introduces a recursive gradient dependency: the SGD update to ϕ changes both the current-state evaluation $V_\phi(s_i)$ and the target $V_\phi(s'_{i,m})$ simultaneously. Maliar et al. (2021) do not address this; the actor-critic method in the [SHAC appendix](#) resolves it via target networks and separated updates.

A.5.1 Algorithm: Bellman Residual Minimization

Input: Policy network π_θ , value network V_ϕ , dynamics f , reward r , discount γ , exogenous weights w , learning rate η , convergence rule $\text{CONVERGED}(\theta, \phi, j)$

Output: Trained policy π_θ^* , value function V_ϕ^*

1. Initialize policy parameters θ and value parameters ϕ
2. **For** $j = 0, 1, 2, \dots$ **do**
3. Sample mini-batch $\mathcal{B}$ of states s_i with two independent shock draws $\{\epsilon_{i,m}\}_{m=1}^2$
4. Compute actions: $a_i = \pi_\theta(s_i)$
5. Compute next states: $s'_{i,m} = f(s_i, a_i, \epsilon_{i,m})$ for $m = 1, 2$
6. Compute Bellman residuals: $F_{i,m}^{\text{BR}} = V_\phi(s_i) - r(s_i, a_i) - \gamma V_\phi(s'_{i,m})$ for $m = 1, 2$
7. Compute FOC residuals: $F_{i,m}^{\text{FOC}} = \nabla_a r|_{a_i} + \gamma \nabla_{s'} V_\phi(s'_{i,m}) \cdot \nabla_a f|_{a_i}$ for $m = 1, 2$
8. Compute envelope residuals: $F_i^{\text{Env}} = \nabla_s r(s_i, a_i) - \nabla_s V_\phi(s_i)$
9. Compute constraint losses (if any): $\mathcal{L}^{\text{IC}}$ and $\mathcal{L}^{\text{EC}}$

10. Compute combined loss: $J(\theta, \phi) = \mathcal{L}^{\text{BR}} + w_1 \mathcal{L}^{\text{FOC}} + w_2 \mathcal{L}^{\text{Env}} + w_3 \mathcal{L}^{\text{IC}} + w_4 \mathcal{L}^{\text{EC}}$
11. Update: $(\theta, \phi) \leftarrow (\theta, \phi) - \eta \nabla_{(\theta, \phi)} J(\theta, \phi)$
12. **If** CONVERGED(θ, ϕ, j) **then break**
13. **End for**
14. **Return** π_θ^*, V_ϕ^*

A.6 Nested VFI (Risky Debt)

The discrete benchmark solver uses nested iteration. Starting from a candidate risky-rate schedule $\tilde{r}^{(n)}$, an inner loop solves the firm's Bellman equation by value function iteration and produces the converged outer iterate $V^{(n)}$. The lender zero-profit condition then uses this $V^{(n)}$ to determine the default states and update the pricing schedule to $\tilde{r}^{(n+1)}$.

Setup: Discrete grids

- Capital: $k \in \mathcal{K} = \{k_1, \dots, k_{N_k}\}$
- Debt: $b \in \mathcal{B} = \{b_1, \dots, b_{N_b}\}$
- Shock: $z \in \mathcal{Z} = \{z_1, \dots, z_{N_z}\}$ with Markov transition matrix $g_{jl} \equiv p(z_l | z_j)$, where $\sum_{l=1}^{N_z} g_{jl} = 1$

Objects to solve for:

$V(k_i, b_m, z_j)$: equity value on the grid

$\tilde{r}(z_j, k'_i, b'_m)$: risky rate for each current shock and next-period choice

A.6.1 Algorithm

At outer iteration n , the pricing schedule $\tilde{r}^{(n)}$ is treated as fixed. The inner loop solves the Bellman equation under this fixed pricing schedule and produces the converged outer value iterate $V^{(n)}$. This $V^{(n)}$ determines the default states in next period, and those default states imply the updated pricing schedule $\tilde{r}^{(n+1)}$ through the lender's zero-profit condition. The outer loop therefore searches for a value function that is consistent with the pricing schedule implied by its own default states.

Step 0. Initialization

Set iteration counter $n = 0$. Initialize $\tilde{r}^{(0)}(z_j, k'_i, b'_m) = r$ for all (j, i, m) .

Set inner-loop counter $s = 0$. Initialize $V^{(s)}$ with all zeros.

Step 1. Solve the Bellman problem under fixed $\tilde{r}^{(n)}$

- Take a fixed $\tilde{r}^{(n)}$ as input. For each state (k_i, b_m, z_j) , evaluate:

$$V^{(s+1)}(k_i, b_m, z_j) = \max \left\{ 0, \max_{k'_i \in \mathcal{K}, b'_{m'} \in \mathcal{B}} \left[e^{(n)}(k_i, k'_i, b_m, b'_{m'}, z_j) - \Omega(e^{(n)}(\cdot)) + \frac{1}{1+r} \sum_{l=1}^{N_z} g_{jl} V^{(s)}(k'_i, b'_{m'}, z_l) \right] \right\}$$

where the function uses the **fixed** outer-loop interest rate $\tilde{r}^{(n)}$:

$$e^{(n)}(k_i, k'_i, b_m, b'_{m'}, z_j) = (1-\tau)\pi(k_i, z_j) - \psi(k'_i - (1-\delta)k_i, k_i) - (k'_i - (1-\delta)k_i) + \frac{b'_{m'}}{1 + \tilde{r}^{(n)}(z_j, k'_i, b'_{m'})} - b_m + \frac{\tau \tilde{r}^{(n)}(z_j, k'_i, b'_{m'}) b'_{m'}}{(1 + \tilde{r}^{(n)})(1+r)}$$

- Operationally, the solver first computes the Bellman RHS on the full (k', b') choice grid, then clamps the maximized value to zero if it is negative.

Repeat this value iteration, until $\|V^{(s+1)} - V^{(s)}\| < \epsilon_{\text{inner}}$.

When converged, store the result as $V^{(n)} \equiv V^{(s+1)}$. This is the inner-loop value iterate associated with the fixed pricing schedule $\tilde{r}^{(n)}$.

Step 2. Recover the default partition implied by $V^{(n)}$

For each $(k'_i, b'_{m'})$, partition the **future** shock space:

$$\mathcal{D}(k'_i, b'_{m'}) = \{z'_l \in \mathcal{Z} : V^{(n)}(k'_i, b'_{m'}, z'_l) = 0\}$$

$$\mathcal{S}(k'_i, b'_{m'}) = \{z'_l \in \mathcal{Z} : V^{(n)}(k'_i, b'_{m'}, z'_l) > 0\}$$

Step 3. Update the pricing schedule to $\tilde{r}^{(n+1)}$

For each current shock and next-period choice triple $(z_j, k'_i, b'_{m'})$, solve for $\tilde{r}^{(n+1)}$ from the lender's break-even condition:

$$b'_{m'}(1+r) = \sum_{l \in \mathcal{D}(k'_i, b'_{m'})} g_{jl} R(k'_i, z_l) + b'_{m'}(1 + \tilde{r}^{(n+1)}) \sum_{l \in \mathcal{S}(k'_i, b'_{m'})} g_{jl}$$

where $R(k', z') = (1 - c_d)((1 - \tau)\pi(k', z') + (1 - \delta)k')$.

Solving for $\tilde{r}^{(n+1)}$ explicitly:

$$\tilde{r}^{(n+1)}(z_j, k'_i, b'_{m'}) = \frac{(1+r) - \frac{1}{b'_{m'}} \sum_{l \in \mathcal{D}} g_{jl} R(k'_i, z_l)}{\sum_{l \in \mathcal{S}} g_{jl}} - 1$$

Note: if $\sum_{l \in \mathcal{S}} g_{jl} = 0$ (default in all states), the debt is worthless and set $\tilde{r}^{(n+1)} = \infty$ (or equivalently, the bond price $b'/(1 + \tilde{r}) = 0$, meaning no lender will fund this (k', b') pair).

Step 4. Outer-loop convergence

Resolve the Bellman problem in Step 1 under $\tilde{r}^{(n+1)}$ to obtain $V^{(n+1)}$, and stop when consecutive outer value iterates are close:

$$\|V^{(n+1)} - V^{(n)}\| < \epsilon_{\text{outer}} \quad \text{under} \quad \tilde{r}^{(n+1)}$$

Outer convergence is checked only after the Bellman solve under $\tilde{r}^{(n+1)}$ has itself converged. If not converged, set $n \leftarrow n + 1$ and go to Step 1 and repeat.

A.6.2 Remarks

In this method, the endogenous price $\tilde{r}^{(n+1)}$ is solved given the default/solvent partition $\mathcal{D}$ and $\mathcal{S}$, which in turn depends on the last converged $V^{(n)}$ from the inner loop. When both loops converged, the nested fixed point is reached.

The main cons of this method is computational cost. The object $\tilde{r}^{(n)}(z_j, k'_i, b'_{m'})$ is a **three-dimensional array** of size $N_z \times N_k \times N_b$ that must be stored and updated each outer iteration. Each outer iteration triggers a full VFI (many inner iterations). And the inner VFI itself is $O(N_k^2 \times N_b^2 \times N_z)$ per iteration because for each state (k_i, b_m, z_j) we search over all $(k'_i, b'_{m'})$.

Appendix B

Implementation Details

Supporting material for Chapter 1 (Solving Dynamic Models).

This appendix describes architecture-level implementation choices common to all deep-learning solvers (LRM, ERM, BRM, SHAC) in the codebase. The choices are not method-specific and are unchanged across solver classes.

B.1 Input Normalization

State variables in economic models span orders of magnitude (capital in the hundreds, log-productivity near zero, interest rates as fractions). Without normalization, the first-layer gradient scales with raw feature variances and the optimizer cannot make balanced progress across features. I apply a per-feature **static Z-score** to the input layer:

$$\hat{x}_d = \frac{x_d - \mu_d}{\sigma_d + \varepsilon},$$

where μ_d, σ_d are computed once from the full training dataset before any gradient steps and held fixed throughout training. No hidden-layer normalization (BatchNorm, LayerNorm) is applied.

Statistics for the exogenous component s^{exo} are fit on all $N \times T$ trajectory samples to capture the AR(1) ergodic distribution. Statistics for the endogenous component s^{endo} are fit on the N initial states drawn uniformly over the bounded state space. The normalizer does not need to be ergodically exact: every state visited during training falls within the bounded region by construction, so its purpose is to map inputs to an $O(1)$ range that conditions the first-layer gradient, nothing more. Online Z-score normalizers common in RL add no information here because the full dataset is available before training begins.

B.2 Hidden-Layer Activation

The hidden layers use the **SiLU** (Sigmoid-weighted Linear Unit, also known as Swish) activation:

$$\text{SiLU}(h) = h \cdot \sigma(h), \quad \text{SiLU}'(h) = \sigma(h)(1 + h(1 - \sigma(h))).$$

ReLU is the standard alternative but has zero gradient for $h < 0$. Any neuron whose pre-activation is negative for all training samples never recovers, a “dead neuron”. With centered inputs (after the static Z-score above), roughly half of pre-activations are negative on average, so the dead-neuron risk is concrete rather than theoretical. SiLU’s gradient is nonzero everywhere, eliminates dead neurons, and is smooth, which matches the smooth, concave objectives typical in economic models.

B.3 Output Head Transformation

The policy network outputs a continuous action constrained to box bounds $[a_{\min}, a_{\max}]$ (e.g., investment $I \in [I_{\min}, I_{\max}]$). The standard RL choice is a tanh squashing function. I instead use a **linear output head followed by clipping**:

$$\hat{y} = \mathbf{w}^\top \mathbf{a}^L + b, \quad a = \text{clip}(\hat{y}, a_{\min}, a_{\max}).$$

The motivation is gradient quality near the bounds. For tanh (or any differentiable bijection $\mathbb{R} \rightarrow (a_{\min}, a_{\max})$), $\partial a / \partial \hat{y} \rightarrow 0$ as a approaches either bound, a topological necessity for a bounded smooth function. In standard RL benchmarks the optimal policy is rarely near the bounds and the saturation is harmless. In economic models the optimal policy is often near the upper bound when productivity is high, and the per-period reward also has diminishing marginal returns in that region. The two effects compound: $\partial \mathcal{L} / \partial \theta$ becomes small in exactly the region where the policy needs the most learning signal, and the trained policy systematically deviates from the analytical benchmark at the boundaries.

The linear-plus-clip head avoids this. Inside the feasible region the output is identity and $\partial a / \partial \hat{y} = 1$ uniformly; outside, the gradient is zero but the action is correctly snapped. The interior gradient is independent of distance to the boundary, so the policy learns boundary-pushing behavior without saturation. The same design is used by TD-MPC2’s MPPI planner, PPO with clipped actions, and DDPG with bounded action spaces.

Appendix C

Structural Estimation Methods

Supporting material for Chapter 2 (Structural Estimation).

C.1 Generalized Method of Moments (GMM)

GMM estimates structural parameters from moment conditions that are closed-form functions of observables and parameters. Unlike SMM, GMM does not require solving the model: it applies whenever the model produces structural restrictions (e.g., Euler equations) that can be evaluated directly from data and a candidate β .

Symbol	Definition
β^*	True structural parameters. Unknown.
β	A candidate parameter vector.
$\hat{\beta}$	The GMM estimate that minimizes $Q(\beta)$.
K	Number of parameters to estimate.
R	Total number of moment conditions ($R \geq K$).
N	Number of cross-sectional units.
T	Number of time periods.
$e_{it}(\beta)$	Structural residual for observation i, t . At β^* , $E_t[e_{it}(\beta^*)] = 0$.
Z_{it}	Instrument vector: variables known at time t , uncorrelated with $e_{it}(\beta^*)$.
$g(\beta)$	$R \times 1$ sample moment vector. At β^* , $E[g(\beta^*)] = 0$.
W	$R \times R$ positive-definite weighting matrix.
$g_{it}(\beta)$	Per-observation moment contribution: $e_{it}(\beta) \cdot Z_{it}$.
$\hat{\Omega}$	Long-run variance-covariance matrix of g_{it} .
D	$R \times K$ Jacobian: $D_{rk} = \partial g_r / \partial \beta_k _{\hat{\beta}}$.
V	$K \times K$ asymptotic variance-covariance matrix of $\hat{\beta}$.

The conditional restriction $E_t[e_{it}(\beta^*)] = 0$ implies $E[e_{it}(\beta^*) \cdot Z_{it}] = 0$ for any time- t instrument Z_{it} . Stacking instrument interactions gives the sample moment vector

$$g(\beta) = \frac{1}{NT} \sum_{i=1}^N \sum_{t=1}^T e_{it}(\beta) \cdot Z_{it}.$$

Identification requires $R \geq K$ and that the instruments be relevant (correlated with the endogenous variables in e_{it}) and exogenous (uncorrelated with $e_{it}(\beta^*)$).

C.1.1 Estimator and inference

The estimator minimizes $Q(\beta) = g(\beta)^\top W g(\beta)$ for a positive-definite W . Each evaluation is arithmetic on the data, so a standard local optimizer (`scipy.optimize.minimize` with Powell or L-BFGS-B) suffices.

Two-step weighting. The optimal W depends on β^* , so estimation is iterative.

1. Set $W = I_R$ and minimize to obtain $\hat{\beta}_1$.
2. At $\hat{\beta}_1$, compute the per-observation contributions $g_{it}(\hat{\beta}_1) = e_{it}(\hat{\beta}_1) \cdot Z_{it}$ and estimate $\hat{\Omega}$. For dynamic models, the moment contributions are serially correlated within firm because consecutive observations share persistent state variables, so a HAC estimator is required:

$$\hat{\Omega}_{\text{HAC}} = \hat{\Gamma}_0 + \sum_{l=1}^L w(l)(\hat{\Gamma}_l + \hat{\Gamma}_l^\top), \quad \hat{\Gamma}_l = \frac{1}{NT} \sum_{i,t>l} g_{it} g_{i,t-l}^\top,$$

with Bartlett kernel weights $w(l) = 1 - l/(L+1)$ and bandwidth $L = \lfloor T^{1/3} \rfloor$. Cross-sectional independence across firms is assumed. The i.i.d. estimator $\hat{\Omega}_{\text{iid}} = \frac{1}{NT} \sum_{i,t} g_{it} g_{it}^\top$ should only be used when serial correlation is absent, since it underestimates the true variance otherwise.

3. Set $W = \hat{\Omega}^{-1}$ and warm-start from $\hat{\beta}_1$ to obtain $\hat{\beta}$.

Inference at $\hat{\beta}$. With $W = \hat{\Omega}^{-1}$, the asymptotic variance is

$$V = (D^\top \hat{\Omega}^{-1} D)^{-1}, \quad \text{se}(\hat{\beta}_k) = \sqrt{V_{kk}/(NT)},$$

where the Jacobian D is computed analytically or by centered finite differences. The t-statistic $t_k = (\hat{\beta}_k - \beta_k^0)/\text{se}(\hat{\beta}_k)$ tests $H_0 : \beta_k^* = \beta_k^0$. The overidentification test uses

$$J = NT \cdot Q(\hat{\beta}) \xrightarrow{d} \chi^2(R - K),$$

requiring $R > K$ and $W = \hat{\Omega}^{-1}$. Reject at level α if $J > \chi_{1-\alpha}^2(R - K)$; rejection indicates misspecification.

C.1.2 Application to the basic investment model with convex cost

I apply GMM to the basic investment model with convex adjustment costs in Strebulaev and Whited (2012, sec. 3.1), which has a closed-form Euler equation. The structural primitives are:

- Production $\pi(k, z) = zk^\alpha$ with $\alpha \in (0, 1)$.
- Convex adjustment cost $\psi(I, k) = \frac{\psi_1}{2} I^2/k$ with $I_t = k_{t+1} - (1 - \delta)k_t$.
- AR(1) productivity $\ln z_{t+1} = \rho \ln z_t + \varepsilon_{t+1}$, $\varepsilon \sim \mathcal{N}(0, \sigma_\varepsilon^2)$.
- Estimated parameters $\beta = (\alpha, \psi_1, \rho, \sigma_\varepsilon)$, $K = 4$. Calibrated: r, δ .

Observables. For firm i at time t , the observable variables are $\pi_{it} = z_{it}k_{it}^\alpha$ (operating income), k_{it} (book capital), and $I_{it} = k_{i,t+1} - (1 - \delta)k_{it}$ (investment). The productivity z_{it} is latent and recovered at a candidate

α via $\ln z_{it}(\alpha) = \ln \pi_{it} - \alpha \ln k_{it}$. Every term in the residuals below is a known function of (π, k, I) and the candidate β .

Structural residuals. Eliminating V via the envelope condition, the Euler equation is

$$1 + \psi_1 \frac{I_t}{k_t} = \frac{1}{1+r} \mathbb{E}_t \left[\alpha \frac{\pi_{t+1}}{k_{t+1}} + \frac{\psi_1}{2} \left(\frac{I_{t+1}}{k_{t+1}} \right)^2 + (1-\delta) \left(1 + \psi_1 \frac{I_{t+1}}{k_{t+1}} \right) \right].$$

The marginal product of capital $\alpha z_{t+1} k_{t+1}^{\alpha-1} = \alpha \pi_{t+1} / k_{t+1}$ is directly computable from observables. The Euler residual replaces the conditional expectation with realized values:

$$e_{it}^u(\beta) = \alpha \frac{\pi_{i,t+1}}{k_{i,t+1}} + \frac{\psi_1}{2} \left(\frac{I_{i,t+1}}{k_{i,t+1}} \right)^2 + (1-\delta) \left(1 + \psi_1 \frac{I_{i,t+1}}{k_{i,t+1}} \right) - (1+r) \left(1 + \psi_1 \frac{I_{it}}{k_{it}} \right).$$

This block identifies α and ψ_1 . The AR(1) residual $e_{it}^v(\beta) = \ln z_{i,t+1}(\alpha) - \rho \ln z_{it}(\alpha)$ identifies ρ (and α through $\ln z$). The variance condition $e_{it}^w(\beta) = (e_{it}^v)^2 - \sigma_\varepsilon^2$ identifies σ_ε .

Instruments: lagged only. I use strictly lagged variables (time $t-1$ and earlier) as instruments. Current-period variables (I_t/k_t , π_t/k_t , $\ln z_t$) appear directly in the residuals; using them as instruments would create mechanical second-moment terms like $\psi_1 (I_t/k_t)^2$ that produce pathologically small standard errors. The Euler block uses

$$Z_{it}^u = (1, I_{i,t-1}/k_{i,t-1}, \pi_{i,t-1}/k_{i,t-1})^\top.$$

The shock block uses $Z_{it}^v = (1, \ln z_{i,t-1}(\alpha))^\top$, which depends on α through the recovery formula and is re-evaluated at each candidate. The variance block uses only the constant. Validity holds because all instruments are known at time $t-1$ and the AR(1) innovations are i.i.d.

Stacked moment vector. The $R \times 1$ sample moment vector with $R = 6$ is

$$g(\beta) = \frac{1}{NT} \sum_{i,t} \begin{pmatrix} e_{it}^u(\beta) \cdot Z_{it}^u \\ e_{it}^v(\beta) \cdot Z_{it}^v \\ e_{it}^w(\beta) \end{pmatrix}.$$

Block	Residual	Instruments	Conditions	Identifies
Euler equation	e_{it}^u	Z_{it}^u (3×1)	3	α, ψ_1
Shock process	e_{it}^v	Z_{it}^v (2×1)	2	ρ (and α via $\ln z$)
Variance	e_{it}^w	constant	1	σ_ε
Total			6	$K = 4$, overid = 2

Table C.2: Moment block summary for the basic investment model.

The optimal weight $\hat{\Omega}$ is computed with the HAC estimator above. This GMM design requires a closed-form Euler equation; fixed costs, default options, and other non-differentiabilities break it and require switching to SMM.

C.2 Simulated Method of Moments (SMM)

I estimate the parameters of the risky debt model in Strebulaev and Whited (2012, sec. 3.6) using SMM. Each candidate β requires a fresh model solve (VFI / PFI / NN method), so wall time is dominated by the optimizer's inner loop.

Symbol	Definition
β^*	True structural parameters. Unknown.
β	A candidate parameter vector.
$\hat{\beta}$	The SMM estimate that minimizes $Q(\beta)$.
x	Real-world dataset (one panel).
K	Number of parameters to estimate.
R	Number of moments ($R \geq K$).
S	Number of independently simulated panels per evaluation of Q .
$M(x)$	$R \times 1$ moment vector from the real data.
$m_s(\beta)$	$R \times 1$ moment vector from simulated panel s .
$\bar{m}(\beta)$	$R \times 1$ averaged simulated moments: $\bar{m} = \frac{1}{S} \sum_s m_s$.
W	$R \times R$ positive-definite weighting matrix.

C.2.1 Estimator and inference

1. **Setup (once).** Compute target moments $M(x)$ from the real panel. Fix simulation shocks via a master seed: at every evaluation of Q , the same uniform draws $u \sim U(0, 1)$ are converted to model-specific shocks (e.g., $\varepsilon = \Phi^{-1}(u)$ for AR(1)). Common random numbers across β candidates make $Q(\beta)$ smooth and reduce optimizer iterations.
2. **Evaluate $Q(\beta)$.** Solve the model at β , simulate S panels using the fixed shocks, compute moments per panel $m_s(\beta)$, average to $\bar{m}(\beta) = \frac{1}{S} \sum_s m_s(\beta)$, form the error vector $e(\beta) = \bar{m}(\beta) - M(x)$, and return

$$Q(\beta) = e(\beta)^\top W e(\beta).$$

Retain $\{m_s(\beta)\}_{s=1}^S$ for later $\hat{\Omega}$ construction. Level deviations are the default; percent deviations $e_r = (\bar{m}_r - M_r)/M_r$ are available but only safe when moments have comparable magnitudes (small M_r inflates the $\hat{\Omega}$ condition number by $1/M_r^2$).

3. **Two-step weighting.** Run the optimizer with $W = I_R$ to obtain $\hat{\beta}_1$. At $\hat{\beta}_1$, form the per-panel error vectors $E_s = m_s(\hat{\beta}_1) - M(x)$ and estimate $\hat{\Omega} = \frac{1}{S} \sum_s E_s E_s^\top$ ($S > R$ is required for full rank; $S \gg R$ in practice). Set $W = \hat{\Omega}^{-1}$, warm-start from $\hat{\beta}_1$, and re-run the local optimizer to obtain $\hat{\beta}_2$.
4. **Inference.** Compute the Jacobian by centered finite differences, $D_{rk} \approx [e_r(\hat{\beta} + h_k \mathbf{e}_k) - e_r(\hat{\beta} - h_k \mathbf{e}_k)] / (2h_k)$ with $h_k = \max(10^{-4}|\hat{\beta}_k|, 10^{-8})$. Each entry requires a full evaluation; the full Jacobian costs $2K$ solves. The error vector has variance $(1 + 1/S)\Omega$ (combining target noise and simulation noise), so by the delta method

$$V = \left(1 + \frac{1}{S}\right) (D^\top W D)^{-1}, \quad \text{se}(\hat{\beta}_k) = \sqrt{V_{kk}}.$$

The t-statistic for $H_0 : \hat{\beta}_k = \beta_k^0$ is $t_k = (\hat{\beta}_k - \beta_k^0) / \text{se}(\hat{\beta}_k)$. The overidentification test uses

$$J = \frac{S}{S+1} Q(\hat{\beta}) \xrightarrow{d} \chi^2(R-K),$$

where the $S/(S + 1)$ factor corrects for the target being a single random panel. Failing to reject is consistent with correct specification but does not prove it.

C.2.2 Optimization

Each $Q(\beta)$ evaluation requires a full model solve. With hundreds to thousands of optimizer iterations, total wall time is dominated by the solve count. I use `scipy.optimize` rather than `tf.keras.optimizers` because the model solve (VFI, discrete default decisions) is not differentiable through standard automatic differentiation. SciPy supports bounds, global search, and finite-difference gradients; the NumPy / TF conversion overhead is negligible relative to the solve cost.

The optimizer runs in two phases. Stage 1 uses `dual_annealing` for stochastic global search over the bounded parameter space, following Hennessy and Whited (2007). After `dual_annealing` exhausts its `maxiter` budget, Powell is run from the best point found to refine to $\hat{\beta}_1$. Stage 2 reuses Powell from $\hat{\beta}_1$ with $W = \hat{\Omega}^{-1}$. Powell is derivative-free and treats Q as a black box; it converges linearly but is robust when the objective is non-smooth in β .

Phase	Method	Input	Output
Stage 1 global	<code>dual_annealing</code>	initial guess, bounds	coarse basin
Stage 1 polish	Powell	best point above	$\hat{\beta}_1$
Stage 2	Powell	$\hat{\beta}_1, W = \hat{\Omega}^{-1}$	$\hat{\beta}_2$

C.2.3 Validation metrics

For each parameter k , I compute the following diagnostics across J Monte Carlo replications (the validation procedure itself is described in the [GMM and SMM section](#)):

Metric	Formula	Target	Interpretation
Bias	$\frac{1}{J} \sum_j \hat{\beta}_k^{(j)} - \beta_{0,k}$	≈ 0	A biased parameter implies its identifying moment is poorly computed or weak
SD	$\text{sd}(\{\hat{\beta}_k^{(j)}\})$	small	Variability across replications
Avg SE	$\frac{1}{J} \sum_j \text{se}_k^{(j)}$	small	Used for per-replication t-tests
RMSE	$\sqrt{\frac{1}{J} \sum_j (\hat{\beta}_k^{(j)} - \beta_{0,k})^2}$	small	Combined bias and variance
J -test size	Fraction with $J^{(j)} > \chi_{0.95}^2(R - K)$	≈ 0.05	Should match the nominal 5% size

Under sufficient optimizer budget, $\text{SD} \approx \text{Avg SE}$; with limited budget, optimizer noise inflates SD relative to SE.

C.2.4 Application to the basic investment model

I follow Hennessy and Whited (2007) and calibrate $r = 0.04$, $\delta = 0.15$ externally. Two SMM specifications are used: the frictionless model for clean validation (analytical policy, no solver inside the loop) and the frictional model for end-to-end testing.

Frictionless validation ($\psi_0 = \psi_1 = 0$). The optimal policy $k^*(z)$ is closed-form, so any error must be in SMM rather than in the model solve. Four moments identify $K = 3$ parameters ($\alpha, \rho, \sigma_\varepsilon$):

#	Moment	Definition	Identifies
1	Mean I/k	$E[I_{it}/k_{it}]$	α
2	Var I/k	$\text{Var}[I_{it}/k_{it}]$	$\alpha, \sigma_\varepsilon$
3	Serial corr I/k	$\text{Corr}(I_{it}/k_{it}, I_{i,t-1}/k_{i,t-1})$	ρ
4	AR(1) resid std	$\hat{\sigma}_u$ from panel AR(1) on I/k	σ_ε

Frictional application ($\psi_1 > 0$). No closed-form policy, so the model is solved via PFI or ER inside the optimizer loop. Five moments identify $K = 4$ parameters ($\alpha, \psi_1, \rho, \sigma_\varepsilon$):

#	Moment	Definition	Identifies
1	Mean I/k	$E[I_{it}/k_{it}]$	α
2	Var I/k	$\text{Var}[I_{it}/k_{it}]$	α, ψ_1
3	Serial corr I/k	$\text{Corr}(I_{it}/k_{it}, I_{i,t-1}/k_{i,t-1})$	ψ_1, ρ
4	AR(1) persistence	$\hat{\beta}_1$ from panel AR(1) on π/k	ρ, ψ_1
5	AR(1) resid std	$\hat{\sigma}_u$ from same regression	σ_ε

C.2.5 Application to the risky debt model

I follow Hennessy and Whited (2007)'s moment selection. The active set has 11 moments identifying up to $K = 7$ parameters ($\alpha, \psi_1, \eta_0, \eta_1, c_{\text{def}}, \rho, \sigma_\varepsilon$), leaving 4 overidentifying restrictions when all parameters are estimated. When parameters are calibrated externally (e.g., c_{def}), moments tagged solely to those parameters are auto-dropped. The equity issuance cost is $\Omega(e) = (\eta_0 + \eta_1|e|) \mathbf{1}\{e < 0\}$, separating a fixed cost (η_0 , gating issuance frequency) from a proportional cost (η_1 , shaping the pecking order).

#	Block	Moment	Definition	Identifies
1	Issuance	Avg equity issuance / assets	$E[\max(0, -e_{it})/k_{it}]$	η_0
2	Issuance	Frequency of issuance	$\Pr(e_{it} < 0)$	η_0
3	Issuance	Corr(issuance, investment)	$\text{Corr}(\max(0, -e_{it})/k_{it}, I_{it}/k_{it})$	η_0, η_1
4	Leverage	Book leverage	$E[b'_{it}/k_{it}]$	c_{def}
5	Leverage	Cov(leverage, investment)	$\text{Cov}(b'_{it}/k_{it}, I_{it}/k_{it})$	c_{def}
6	Investment	Var I/k	$\text{Var}[I_{it}/k_{it}]$	α, ψ_1

#	Block	Moment	Definition	Identifies
7	Investment	Serial corr I/k	$\text{Corr}(I_{it}/k_{it}, I_{i,t-1}/k_{i,t-1})$	ψ_1
8	Real	Mean I/k	$E[I_{it}/k_{it}]$	α
9	Real	AR(1) persistence	$\hat{\beta}_1$ from panel IV on $\Delta \log y_{it}$	ρ
10	Real	AR(1) shock std dev	$\hat{\sigma}_u$ from same regression	σ_ε
11	Default	Default frequency	$\Pr(\text{default}_{it+1} \mid \text{state}_{it})$	c_{def}

Table C.8: Active SMM moment set for the risky debt model ($R = 11$).

I depart from H&W's covariance form for the issuance-investment pecking-order channel and use the correlation instead, since $\text{Cov}(\text{Iss}/k, I/k)$ has a population variance of $\sim 10^{-8}$ (both terms are near-zero for most firm-years), which would make $\hat{\Omega}$ singular in finite samples. The correlation is bounded in $[-1, 1]$ and conditioning is comparable to other moments. Each candidate β requires a full nested VFI solve (see the [nested VFI appendix](#)); computing all 11 moments per simulated panel adds negligible overhead relative to the solve.

For Monte Carlo validation, I fix the truth at:

Parameter	β_0	Role
α	0.7	Production technology
ψ_1	0.05	Adjustment cost
η_0	0.6	Fixed issuance cost
η_1	0.1	Proportional issuance cost
c_{def}	0.5	Deadweight default cost
ρ	0.7	AR(1) persistence
σ_ε	0.15	AR(1) shock std dev

Appendix D

Moment fit: TO Model

Supporting material for Chapter 5 (Nikolov, Schmid, and Steri models).

The moment vector stacks the slope and curvature coefficients of the three policy regressions (intercepts dropped). Actions are written as i/k (investment rate), b'/k' (future leverage), and d/k (payout); regressors are $\log k$, profitability (prof), and current leverage (lev), together with their squares and interactions. The gap is $g = h_{\text{data}} - h_{\text{sim}}$.

Action	Regressor	Data h	Sim h	Gap g
i/k	$\log k$	0.009	-0.688	0.697
i/k	prof	0.007	0.170	-0.163
i/k	lev	0.027	-0.434	0.461
i/k	$(\log k)^2$	-0.001	0.091	-0.092
i/k	$\log k \cdot \text{prof}$	0.019	-0.089	0.108
i/k	$\log k \cdot \text{lev}$	0.005	0.193	-0.189
i/k	prof^2	0.248	0.075	0.173
i/k	$\text{prof} \cdot \text{lev}$	-0.066	-0.095	0.028
i/k	lev^2	-0.042	0.236	-0.278
b'/k'	$\log k$	-0.011	-0.210	0.199
b'/k'	prof	-0.186	-0.002	-0.185
b'/k'	lev	0.820	0.774	0.046
b'/k'	$(\log k)^2$	0.000	-0.010	0.010
b'/k'	$\log k \cdot \text{prof}$	0.025	-0.011	0.037
b'/k'	$\log k \cdot \text{lev}$	0.033	0.165	-0.132
b'/k'	prof^2	-0.083	0.018	-0.101
b'/k'	$\text{prof} \cdot \text{lev}$	-0.160	-0.063	-0.098
b'/k'	lev^2	-0.227	-0.245	0.017
d/k	$\log k$	0.002	-0.024	0.026
d/k	prof	0.073	0.358	-0.285
d/k	lev	-0.078	-0.135	0.057
d/k	$\log k^2$	-0.000	-0.054	0.054
d/k	$\log k \cdot \text{prof}$	0.012	-0.088	0.100
d/k	$\log k \cdot \text{lev}$	0.003	0.091	-0.088
d/k	prof^2	0.128	0.080	0.048

Action	Regressor	Data h	Sim h	Gap g
d/k	prof · lev	-0.131	-0.059	-0.072
d/k	lev ²	0.040	-0.076	0.116

References

- DeAngelo, Harry. 2022. “The Capital Structure Puzzle: What Are We Missing?” *Journal of Financial and Quantitative Analysis* 57 (2): 413–54. <https://doi.org/10.1017/S002210902100079X>.
- Duarte, Victor, Diogo Duarte, and Dejanir H Silva. 2024. “Machine Learning for Continuous-Time Finance.” *The Review of Financial Studies* 37 (11): 3217–71.
- Fernández-Villaverde, Jesús. 2025. “Deep Learning for Solving Economic Models.” National Bureau of Economic Research.
- Graham, John R. 2022. “Presidential Address: Corporate Finance and Reality.” *The Journal of Finance* 77 (4): 1975–2049.
- Hennessy, Christopher A, and Toni M Whited. 2007. “How Costly Is External Financing? Evidence from a Structural Estimation.” *The Journal of Finance* 62 (4): 1705–45.
- Korteweg, Arthur. 2010. “The Net Benefits to Leverage.” *The Journal of Finance* 65 (6): 2137–70.
- Maliar, Lilia, Serguei Maliar, and Pablo Winant. 2021. “Deep Learning for Solving Dynamic Economic Models.” *Journal of Monetary Economics* 122 (September): 76–101. <https://doi.org/10.1016/j.jmoneco.2021.07.004>.
- Nikolov, Boris, Lukas Schmid, and Roberto Steri. 2021. “The Sources of Financing Constraints.” *Journal of Financial Economics* 139 (2): 478–501. <https://doi.org/10.1016/j.jfineco.2020.07.018>.
- Strebulaev, Ilya A, and Toni M Whited. 2012. “Dynamic Models and Structural Estimation in Corporate Finance.” *Foundations and Trends in Finance*.
- Vihola, Matti. 2012. “Robust Adaptive Metropolis Algorithm with Coerced Acceptance Rate.” *Statistics and Computing* 22 (5): 997–1008.
- Xu, Jie, Viktor Makoviychuk, Yashraj Narang, Fabio Ramos, Wojciech Matusik, Animesh Garg, and Miles Macklin. 2022. “Accelerated Policy Learning with Parallel Differentiable Simulation.” <https://arxiv.org/abs/2204.07137>.